On Intelligence

and Its Specifications

Alex Towell

On Intelligence and Its Specifications

First edition, 2026.

Published independently via Kindle Direct Publishing.

For the people who will live in the world

being built from this mathematics.

Contents

Preface

We taught sand to think.

That is not a figure of speech. We took silicon, the most common and least regarded material on the planet, arranged it with great care, and taught it to predict. Out of prediction came something that, in every sense the mathematics in this book can make precise, is thought. Not an imitation of thought hung on the outside of a machine. Thought, done by the machine, because prediction carried far enough stops being a model of the thing and becomes the thing.

I want to persuade you of two claims that sit badly together. The first is that this is the most remarkable thing our species has done. The second is that we did it in the wrong order. We learned to build the optimizer before we learned to write down what it should optimize. The capability arrived first. The specification has not arrived at all. The distance between a machine that does exactly what we said and a world that needed it to do what we meant is where the safety of everything ahead will be decided, and in 2026 it is not yet decided.

Most writing about AI sits at one of two poles. One treats every new model as the last step before a digital god. The other treats the whole subject as a bubble, a parlor trick, a way to sell graphics cards. Both are ways of not looking. This book looks, with the one instrument that actually sharpens the view: the mathematics.

Here is the part almost no one tells you. There is a real, settled, genuinely beautiful theory of what intelligence is. Bayes' theorem. Kraft's inequality. Solomonoff's universal prior. Expected utility. AIXI as the synthesis. None of it is in dispute. All of it has sat in the technical literature for decades, optimal and uncomputable, describing exactly what an ideal mind would do. We have never been able to build it. What we have built instead, the large language

models that anyone reading this has used, are crude, bounded, partial shadows of that ideal. Not the theory. The closest thing to it anyone has made, and close enough that the resemblance is not a metaphor.

The gap between the clean theory and the crude practice is the subject of this book, because the gap is where the danger lives. Reward hacking, deceptive alignment, the whole catalog of what can go wrong: these are not separate problems. Each has coordinates on that gap. To see them you need both sides, the theory of what intelligence is when it is optimal and the honest picture of what we have actually built. Most people have neither. By the last page you will have both.

The book builds the theory carefully, so the gap can be named without hand-waving.

Part I is about prediction: Bayes, the prior problem, description and probability, Solomonoff induction.

Part II is about decision: the agent, reinforcement learning, generalization, AIXI.

Part III is about specification: reward modeling, inner alignment, why optimization is dangerous, the mitigations and their limits.

Part IV is about reality: what large language models are, how they are trained, the gap between them and the theory, where the trajectory is going, and what it all means.

Every chapter develops one idea in plain language, with diagrams carrying as much of the work as the prose. The mathematics is real, and it is explained conceptually before it is formalized. You do not need a degree. You need a willingness to think about one thing at a time, and to keep going.

A word about what kind of book this is. It is not neutral, and I am not going to pretend it is. For most of this field's history the burden of proof sat on anyone who claimed that serious machine intelligence was close. That burden has shifted. The trajectory is real, the forces driving it are nowhere near spent, and the comfortable assumption that all of this will conveniently stall is now the claim that owes you an argument. I do not give you a date, because dates

are how forecasters embarrass themselves. I give you the structure, and a stand the mathematics earns. Where the picture is contested, I say so. Where my own reading runs ahead of what most researchers will commit to, I say so. You do not have to agree with me. You have to think clearly. If the book works, you will think more clearly at the end than at the beginning about what is probably the most consequential thing happening in your lifetime.

I wrote the book I wish had existed when I began taking these questions seriously. The mathematics is real. The systems are real. The gap between them is real, and you, and everyone you love, will live in the world that gap decides. That is reason enough to understand it.

Part I

Prediction

Chapter 1

Bayes

The actual science of logic is conversant at present only with things either certain, impossible, or entirely doubtful, none of which (fortunately) we have to reason on. Therefore the true logic for this world is the calculus of probabilities.

James Clerk Maxwell, 1850

Suppose a friend tells you they flipped a coin five times and every flip came up heads.

What should you believe?

There are at least two hypotheses worth taking seriously. The coin might be ordinary and your friend got lucky. Five heads in a row on a fair coin happens once in every thirty-two trials, which is unusual but not rare. Alternatively, the coin might be a trick coin, weighted to always land heads. Such coins exist; some are sold as novelties.

Before your friend mentioned the experiment, you had some sense of how common trick coins are. Probably very low. Most coins are fair. Trick coins exist but are unusual.

After your friend's report, your belief should change. The data, five heads in a row, fits a trick coin more comfortably than a fair coin. Not impossibly for a fair coin. Just more comfortably for a trick one.

How much should your belief change?

This is the question Bayes' theorem answers. Given:

- Your prior belief about how common trick coins are.
- The probability of five heads on a fair coin.
- The probability of five heads on a trick coin (which is one, every time).

You can compute exactly the new probability that this coin is trick.

This chapter is about that calculation, where it comes from, and why it is the only consistent way to do it.

Belief as counting

Strip away the formula for a moment.

Imagine a population of ten thousand coin-flipping trials. Before the data, you believe one percent of all such trials use a trick coin. So about one hundred trials use trick coins; about nine thousand nine hundred use fair coins.

Now restrict your attention to the trials that produced HHHHH.

Every trick-coin trial produced HHHHH (trick coins always do), so all hundred trick-coin trials are in the restricted set.

Fair-coin trials produced HHHHH only about one time in thirty-two. So about three hundred and nine of the nine thousand nine hundred fair-coin trials are in the restricted set.

The restricted set contains roughly $100 + 309 = 409$ trials. Of those, 100 are trick-coin trials.

The fraction of trick-coin trials within the restricted set is

$$\frac{100}{409} \approx 0.24.$$

Your belief that this particular coin is trick, after seeing the data, should be about twenty-four percent.

That is the entire calculation. Counting, then counting again after restriction. Figure 1.1 shows the populations before and after.

The data did not prove anything. The coin might still be fair and your friend might have gotten genuinely lucky. The data restricted

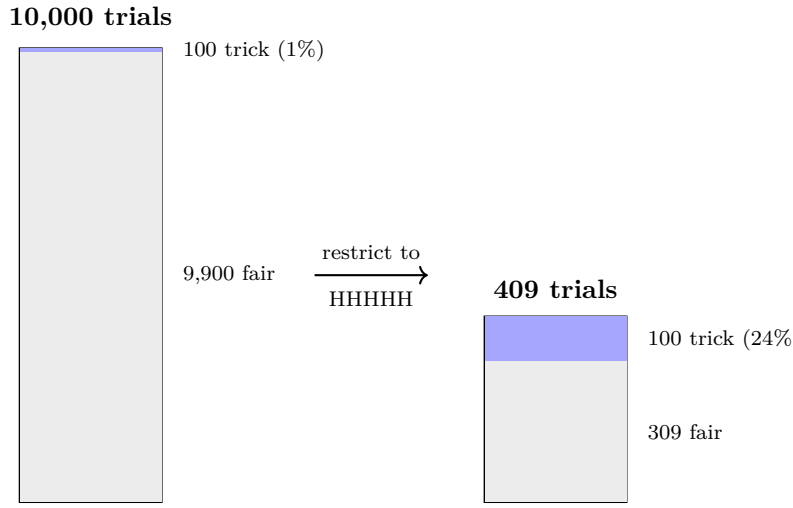


Figure 1.1: The Bayesian update as counting. Left: the prior population of 10,000 trials, 1% trick coins (the thin blue band). Right: restricted to the 409 trials that produced HHHHH. Every trick-coin trial survives (trick coins always produce HHHHH); only about 309 of the 9,900 fair-coin trials do. Within the restriction, the trick fraction has grown from 1% to about 24%, not because any individual trial changed, but because the data restricted the count.

the population of trials consistent with what you observed, and within that restricted population, the proportion of trick-coin trials changed.

This is what an update is: restriction to a subset, followed by counting within.

Bayes' theorem

The formula is what you get when you write the counting argument with symbols.

Let H stand for the hypothesis "this is a trick coin" and D stand for the data "five heads in a row." We want $P(H \mid D)$: the probability the coin is trick, given the data.

By the counting argument,

$$P(H \mid D) = \frac{\text{count of trick-coin trials that produced } D}{\text{count of all trials that produced } D}.$$

The numerator counts the trick-coin trials that produced the data. With symbols, that is the total count of trick-coin trials times the fraction of those that produced the data: $P(H) \cdot P(D \mid H)$.

The denominator counts all trials that produced the data, split by hypothesis:

$$P(D) = P(H)P(D \mid H) + P(\neg H)P(D \mid \neg H).$$

Putting them together,

$$P(H \mid D) = \frac{P(D \mid H)P(H)}{P(D)}.$$

This is Bayes' theorem. It looks more imposing than the counting. It is the counting, with names for the pieces. Figure 1.2 shows the geometry.

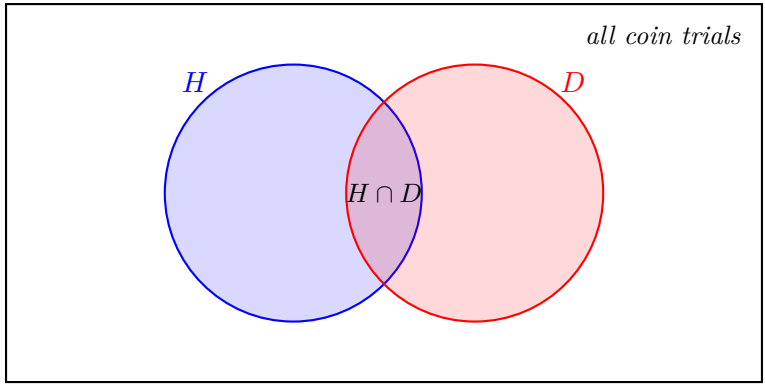


Figure 1.2: Bayesian updating as restriction. The outer rectangle is the space of all possible coin trials. The blue circle H contains the trials where the coin is trick. The red circle D contains the trials that produced five heads. After the data, only trials in D are still in play. The posterior $P(H \mid D) = |H \cap D|/|D|$ is the fraction of the restricted population that is also in H. Bayes' theorem is what you get when you write this fraction in terms of the original sizes.

The named pieces:

- $P(H)$ is the *prior*. What you believed before the data.
- $P(D \mid H)$ is the *likelihood*. How likely the data was, assuming the hypothesis.
- $P(D)$ is the *evidence*. How likely the data was at all, summed over hypotheses.
- $P(H \mid D)$ is the *posterior*. What you should believe after the data.

The formula moves you from prior to posterior. Given the prior and the likelihoods, the posterior is forced.

Why this is the only consistent way

There are other rules you could imagine for combining beliefs and evidence. Many have been proposed. Almost none survive contact with simple consistency requirements.

Suppose your rule combines two independent pieces of evidence by adding their support. You see evidence supporting the hypothesis to degree 0.6. You see a second independent piece, also supporting to degree 0.6. Adding gives 1.2. But a degree of belief is bounded by certainty; it cannot exceed 1. The rule produces nonsense at the boundary, which means it is producing nonsense in proportion everywhere short of the boundary.

Or suppose your rule produces different conclusions depending on the order in which evidence arrives. You see D_1 then D_2 and conclude one thing. You see D_2 then D_1 and conclude another. The evidence is the same. A consistent agent reaches the same conclusion either way. The order in which two pages of a letter are read should not change what the letter says.

In 1946, the physicist R. T. Cox showed that the consistency requirements needed to avoid these and similar pathologies, taken together, force the calculus of belief revision to be probability theory.

Strictly, force it to be isomorphic to probability theory, which is the same thing up to renaming.[1]

The requirements are modest:

- Degrees of belief are real numbers.
- Two paths to the same conclusion give the same answer.
- The calculus reduces to classical logic when certainty is reached.

The conclusion is sharp. Any agent whose belief revision is internally consistent, who reaches the same conclusion no matter the order of evidence, and who respects logic where logic suffices, is doing Bayesian updating.

Whether or not they call it that. Whether or not they have read this chapter. Whether or not they know it.

This is a load-bearing result for the rest of the book. Bayesian updating is not one option among many. It is the unique calculus of consistent belief. Any other rule, applied consistently, eventually contradicts itself.

What Bayes does not tell you

The theorem is precise. Cox's result makes it unique. Together they specify how you should update beliefs in the presence of evidence.

They do not specify where you should start.

In the coin example, the prior was "one percent of coins are trick." That number came from somewhere. Past experience with coins, presumably. But past experience with coins gives you a probability over coins by, presumably, Bayesian updating. Which requires a prior over coins. Which had to come from somewhere.

The regress goes all the way down.

[1] Cox's original proof has had loose ends tightened by later authors. A counterexample by Halpern (1999) showed the argument requires an additional regularity condition on the domain; modern treatments (Jaynes, Paris, Van Horn) either assume this or restrict the setting. The qualitative conclusion is not in dispute.

A reasonable agent could start with $P(\text{trick}) = 0.5$, treating fair and trick coins as equally likely a priori. After five heads, they would land at about 0.97.

Another reasonable agent could start with $P(\text{trick}) = 0.001$. After the same five heads, they would land at about 0.031.

Both are using Bayes. Both are internally consistent. Both answer the same question with the same data. They land in different places because they started in different places. Cox's theorem does not adjudicate. Bayes' theorem does not either.

This is the problem of induction returning in a new form. Hume said there is no logical justification for inferring future from past. Bayes says: given a prior, here is how to do the inference. The hard part of Hume's problem, the part that resists, is the same one. Where does the prior come from?

Where this leaves you

You now have a procedure for updating beliefs. The procedure is unique. Any consistent agent uses it, whether they have noticed or not.

You also have a hole. The procedure requires a prior the procedure itself does not supply. Different priors give different posteriors. The math does not pick between them.

The next chapter looks at the prior problem in more detail and shows why it is harder than it first appears. Chapter 3 introduces the bridge between description length and probability. Chapter 4 takes the bridge to its limit and arrives at the universal prior, the unique optimal prior in a sense Chapter 4 will make precise.

By the end of Part I, the hole in Bayes is closed. The closing comes with a price. The optimal prior is uncomputable. The rest of the book is about what that means for any agent, biological or mathematical or machine, that has to act in the world with bounded resources.

Chapter 2

The Prior Problem

The most important questions of life are, for the most part, really only problems of probability.

Pierre-Simon Laplace, *Théorie analytique des probabilités* (1812)

The end of Chapter 1 left a hole. Bayes tells you how to update beliefs once you have a prior. It does not tell you what the prior should be.

This hole is not a technical wrinkle. It is structural. To see why, return to the coin.

Two agents, same data, different conclusions

Two of your friends, call them Aria and Ben, each receive your report: five heads in a row on a freshly produced coin.

Aria, having read about trick coins, thinks they are quite common in informal contexts. She starts with $P(\text{trick}) = 0.5$. After five heads, by Bayes, her posterior is

$$P(\text{trick} \mid \text{HHHHH}) = \frac{1 \cdot 0.5}{1 \cdot 0.5 + 0.5^5 \cdot 0.5} \approx 0.97.$$

She is quite confident the coin is trick.

Ben, having handled many coins in his life and almost never encountered a trick one, starts with $P(\text{trick}) = 0.001$. After the same five heads,

$$P(\text{trick} \mid \text{HHHHH}) = \frac{1 \cdot 0.001}{1 \cdot 0.001 + 0.5^5 \cdot 0.999} \approx 0.031.$$

He thinks the coin is almost certainly fair; his friend just got lucky.

Same data. Same calculation. Different conclusions. Figure 2.1 shows the divergence.

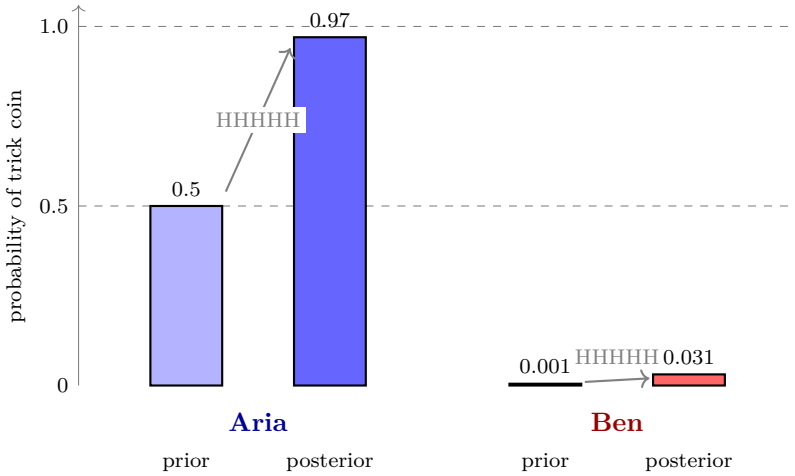


Figure 2.1: Two reasonable agents, same data, different conclusions. Aria's prior is 0.5; after HHHHH she lands at 0.97. Ben's prior is 0.001; after the same HHHHH he lands at 0.031. Both use Bayes correctly. The disagreement is downstream of the prior, where Bayes itself has no view.

Neither is making a mathematical mistake. Both are using Bayes correctly. The disagreement is downstream of the prior, and Bayes itself has nothing to say about who is right.

This is not a fringe case. The same structure appears whenever real agents reason about the world. Doctors interpreting the same test results often disagree because their priors differ. Forecasters give different probabilities for the same weather pattern. Investors looking at the same balance sheet draw different conclusions. In each case the data is shared and the calculation is sound; the prior is doing the

work that drives the disagreement.

The regress

You might think: fine, the prior is subjective, but it comes from somewhere. From experience, from training, from the next layer of beliefs.

This is correct. It does not solve the problem; it moves it.

If your prior over "is this coin trick" comes from past experience with coins, that past experience was itself processed by some belief-updating mechanism. Presumably Bayes. So your prior over coins is a posterior over coins, derived from earlier data.

Which required an earlier prior over coins.

That earlier prior came from where? From earlier experience, processed by earlier updating. Which required an even earlier prior. Figure 2.2 draws the regress.

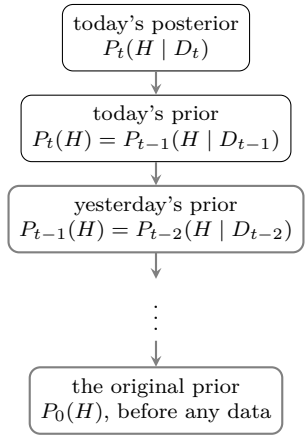


Figure 2.2: The prior regress. Every prior is a posterior from an earlier round of Bayesian updating, which required its own prior, which was itself a posterior from an even earlier round. The regress terminates at the original prior $P_0(H)$, the agent's belief before any data has arrived. That original prior is what is at stake.

The regress goes back as far as you take it. Eventually you reach the prior you started with, before any data. The agent's a priori

belief, prior to all experience.

That prior is what is at stake. Everything downstream depends on it.

The regress is uncomfortable because it suggests there is no starting point that is not itself a choice. Every belief, traced back far enough, depends on some commitment that was not derived from anything.

Conventional answers, and why they do not finish the job

The literature has not been silent. Several proposals exist for how to assign priors when no data has arrived. Each is reasonable in its way; none closes the hole.

Laplace's principle of insufficient reason

If you know nothing about a situation, treat all outcomes as equally likely. For the coin: if you have no information, give each hypothesis probability one half.

The principle is appealing. It is also fragile. The "equally likely outcomes" depend on how you carve up the possibilities. For the coin you could say: trick or fair, fifty-fifty. Or you could say: the bias parameter of the coin lies somewhere in $[0, 1]$, treat all values equally likely. These two priors give different conclusions. Both seem to express "I know nothing."

The principle works when the partition is natural. When the partition is part of the question, the principle assumes the answer.

Maximum entropy

A more refined version of the same intuition. Among all priors consistent with what you know, choose the one with maximum entropy: the one that adds the least information beyond your constraints.

Maximum entropy is mathematically clean and has many useful

applications. It is also relative to a choice of measure on the space of hypotheses. Different measures give different maximum-entropy priors. The principle is good once you have committed to how to describe the hypotheses; it does not tell you how to describe them.

Expert elicitation

If you do not know the prior, ask an expert. The expert's belief, calibrated against their experience, becomes your prior.

This works in many applied settings. It does not address the foundational problem. The expert had to get their prior from somewhere too. And different experts disagree.

Empirical Bayes

Use the data itself to estimate the prior, then update with the rest of the data.

This is a working procedure with real successes. It is not a foundational answer. It uses the data twice, which is fine for engineering and not what we want when asking where the procedure's structure comes from.

What the standard treatment concludes

For two and a half centuries, the consensus on the prior problem was: priors are subjective. They reflect agent-specific commitments that no general principle pins down. Different reasonable agents may end with different posteriors, and there is no principled way to declare one of them right.

This is honest and unsatisfying. It tells you Bayes is the right calculus and then leaves the choice of starting point to taste.

For many practical questions, this is acceptable. Agents with reasonable priors and enough data converge. The disagreement washes out. In the long run, what you started with matters less than what you have seen.

But there is a question the standard treatment cannot answer. Imagine a brand-new agent with no past experience, facing the entire universe. There is no past data to lean on. The agent has to start somewhere. Is there a principled starting point?

Or take two agents arguing about a hypothesis with limited data, and the data is not yet enough for their priors to wash out. Which prior is more defensible?

Or take a situation where the data is sparse and the instances differ. (Most situations.) The prior is doing real work in the conclusion. Whose prior is correct?

The standard treatment says: nobody's, in the foundational sense. Pick what is convenient.

This is not a satisfying place to end. The reader who has followed Chapter 1 saw that Bayes is the unique calculus of consistent belief. The reader who follows this chapter sees that the calculus rests on a choice the calculus cannot itself defend.

A different question

The next chapter takes a different approach. Instead of asking "how do I assign probabilities to hypotheses I have not seen yet," it asks "how do I assign probabilities to descriptions."

This sounds like a sideways move. It is. The sideways move turns out to close the gap.

The bridge: shorter descriptions can be made more probable than longer ones in a way that is precise and does not depend on choosing a partition or a measure. The bridge is built on prefix-free codes, which Chapter 3 introduces. The result is a principle for assigning priors that has no arbitrary choices to dispute about.

Whether this principle is the right one, and whether it does what we want, are questions Chapter 4 takes up. For now: the hole in Bayes is real, the standard answers do not close it, and there is a different way to think about the problem that has been gathering force since the 1960s.

That is where the book is going.

Chapter 3

Description and Probability

The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point.

Claude Shannon, *A Mathematical Theory of Communication* (1948)

Suppose you have to send one of eight possible messages to a friend over a wire that transmits binary digits. Zero or one. Nothing else.

The obvious encoding is three bits per message. Eight messages, $2^3 = 8$ codes, one per message. Each transmission costs three bits.

Now suppose the messages are not equally common. Half the time you want to send message A. The rest is split among the other seven. Is three bits per message still the best you can do?

No. You can do better by using shorter codes for common messages and longer codes for rare ones. The average length drops below three bits.

This chapter is about that idea, taken seriously. The conclusion is that description length and probability are not separate things. They are the same structure seen from two sides. In the next chapter, this conclusion fills the hole that Chapter 2 left in the foundations of Bayesian inference.

Variable-length codes

Let us be specific. You have four possible messages with these probabilities:

- Message A: probability 1/2
- Message B: probability 1/4
- Message C: probability 1/8
- Message D: probability 1/8

The fixed-length code uses two bits per message: A = 00, B = 01, C = 10, D = 11. Average length: two bits.

Try a variable-length code: A = 0, B = 10, C = 110, D = 111. Average length:

$$\tfrac{1}{2}(1) + \tfrac{1}{4}(2) + \tfrac{1}{8}(3) + \tfrac{1}{8}(3) = 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bits.}$$

A quarter of a bit per message saved, on average. Over many messages, this is real.

But there is a catch.

The decoding problem

Variable-length codes have a subtle hazard. Suppose your friend has been receiving bits, and they see:

$$0110100\ldots$$

Where does each codeword end? Could "0" be a complete code for A, with "110" following as the code for C? Or could "01" be a partial code that pairs with later bits to make something else?

If two codewords could be read as starting the same way, you cannot tell them apart on a wire that does not send pauses.

In the example above, the code $A = 0$, $B = 10$, $C = 110$, $D = 111$ has a special property: no codeword is a prefix of another. So when your friend sees a "0", they know that is the entire A code. When they see a "1", they know to keep reading until they see one of 10, 110, 111.

A code with this property is called *prefix-free*: no codeword is a prefix of another. It can be decoded without delimiters.

Not every code is prefix-free. The code $A = 0$, $B = 01$ is not, because A's code (0) is a prefix of B's code (01). A stream beginning "010" is ambiguous: it could be A then 10-something, or B then 0-something. The code itself does not tell you which.

Prefix-free codes can be decoded unambiguously from a stream. They are the natural codes for self-describing data, where the code itself indicates when it has ended.

Codes as trees

Prefix-free codes have a clean visual structure: they correspond to binary trees, where each codeword is a path from the root to a leaf. Figure 3.1 shows the tree for our four-message code.

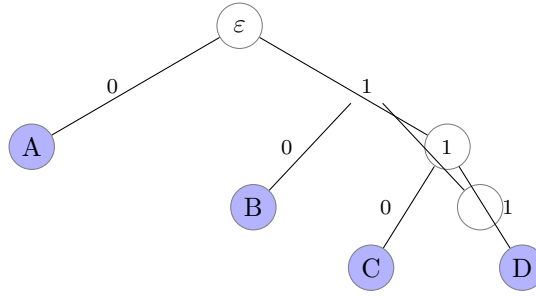


Figure 3.1: The prefix tree for the code $A = 0$, $B = 10$, $C = 110$, $D = 111$. Each codeword is the path from the root to a filled leaf, read off the edge labels. Empty internal nodes are prefixes that continue. The prefix-free property is automatic: codewords sit at leaves; prefixes sit at internal nodes; no codeword is also a prefix of another.

The root has two children. The left child is A: a complete code, a leaf. The right child is the prefix "1", which has its own children: "10" (B, a leaf) and "11" (a prefix that continues). The prefix "11" has children "110" (C) and "111" (D).

Internal nodes are prefixes that continue. Leaves are codewords. The prefix-free property is automatic.

This structure is universal. Every prefix-free binary code is a binary tree with codewords at the leaves. And every binary tree with codewords at the leaves is a prefix-free binary code.

Kraft's inequality

A key question: which sets of codeword lengths can be realized by a prefix-free code?

You cannot, for instance, have four codewords all of length one. There are only two binary strings of length one ("0" and "1"), so at least two of the four codewords must be longer.

You cannot have five codewords of length two. There are only four binary strings of length two.

There is a precise constraint. For any prefix-free binary code with codeword lengths $\ell_1, \ell_2, \ldots$,

$$\sum_i 2^{-\ell_i} \leq 1.$$

This is Kraft's inequality. It is the answer to the question.

Why is it true? Each codeword of length ℓ occupies $2^{-\ell}$ of the unit interval below the root, in the sense of Figure 3.2. A codeword of length one uses half the tree. A codeword of length two uses a quarter. A codeword of length three uses an eighth.

In a prefix-free code, codewords cannot share full paths from root to leaf. Each leaf belongs to exactly one codeword. So the regions assigned to codewords do not overlap, and their total cannot exceed one.

For our four-message code, the lengths are $1, 2, 3, 3$. The Kraft sum is

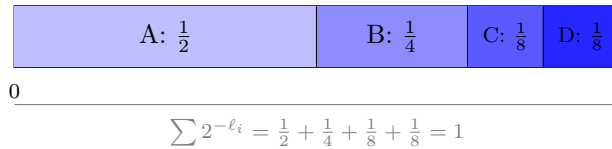


Figure 3.2: Kraft's inequality visualized for the code $A = 0$, $B = 10$, $C = 110$, $D = 111$. Each codeword of length ℓ uses $2^{-\ell}$ of the unit interval. The slabs fit exactly: $1/2 + 1/4 + 1/8 + 1/8 = 1$. The code is complete; the whole interval is used. A code with $\sum 2^{-\ell_i} < 1$ would leave gaps. A code with $\sum 2^{-\ell_i} > 1$ would not fit and could not be prefix-free.

$$2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1.$$

Exactly one. The code is *complete*: every leaf in the tree is a codeword.

Any set of codeword lengths with $\sum 2^{-\ell_i} \leq 1$ admits a prefix-free code. Any set with $\sum 2^{-\ell_i} > 1$ does not. The arithmetic is the only constraint.

The bridge

Now the connection to probability.

Take any prefix-free code with lengths $\ell_1, \ell_2, \ldots$ satisfying Kraft. The values $2^{-\ell_i}$ are nonnegative and sum to at most one. After normalizing (if the sum is less than one), they are a probability distribution:

$$P(i) = \frac{2^{-\ell_i}}{\sum_j 2^{-\ell_j}}.$$

For a complete code, the sum is already one and the normalizer is unnecessary: $P(i) = 2^{-\ell_i}$.

So every prefix-free code defines a probability distribution. Shorter codewords correspond to higher probabilities; longer codewords to lower.

The converse also holds. Given any probability distribution P over a countable set of messages, there is a prefix-free code with lengths ℓ_i close to $-\log_2 P(i)$. The approximation comes from ℓ_i being an integer; the underlying logarithm is real-valued. Shannon and Fano gave a recipe in the 1940s. Huffman gave an optimal one in 1952.

So every probability distribution gives a prefix-free code, and every prefix-free code gives a probability distribution. The two are interconvertible. Description length and probability are the same structure seen from two sides.

The relationship has a name: a prefix-free code and a probability distribution related by $\ell_i \approx -\log_2 P(i)$ are said to be *coupled*. Codes are descriptions; probabilities are weights. Coupling means the description length is approximately the negative logarithm of the weight, and the weight is two to the negative description length.

What this gets us

The bridge does several things at once.

It tells us *shorter descriptions are more probable*, not by stipulation but by structural correspondence. If you have a way of describing something in few bits, the weight that gets attached to that thing is the same weight the code-as-probability scheme implies.

It tells us *any description scheme implies a prior*. Choose a description language (English, binary, computer programs in some language). Choose a way to assign lengths. By the bridge, you have a probability distribution over things being described.

It tells us *the choice of description language matters, but is constrained*. Different languages give different lengths and therefore different priors, but every language must respect Kraft. The arithmetic does not permit a description scheme that disagrees with itself.

The catch: this does not yet pick out a single description language. Different languages give different priors. To get a unique prior, we need a special description language. One that has a claim to universality.

Where this leaves you

Description length and probability are coupled by Kraft's inequality. Shorter descriptions correspond to higher probabilities. Any description scheme implies a prior; any prior implies a description scheme.

The next chapter chooses the description language: computer programs. The choice has a strong claim to universality, because any reasonable computational system can be simulated by any other up to a constant. This is what the *universal* in "universal prior" refers to.

Chapter 4 also stretches the picture in a direction this chapter did not: from finite codes for known messages to infinite codes for messages that include unbounded structure. Solomonoff's prior is what you get when you take Kraft seriously for the most general possible description language.

That is where the prior problem closes.

Chapter 4

Solomonoff Induction

The only really satisfactory definition of randomness was given by Kolmogorov.

Gregory Chaitin, *Algorithmic Information Theory* (1987)

The previous chapter ended with a question. Description length and probability are coupled. Different description languages give different priors. To pick a unique prior, we need a description language with a claim to universality.

This chapter introduces it. The description language is computer programs.

The result, after we work through what this means, is the universal prior $M(x)$: the prior that closes the hole Bayes left, in a sense the chapter will make precise.

Programs as descriptions

A computer program is a description of a computation. Run it; it produces output. The output is what the program describes.

If you want to describe a particular bit string (the coin sequences of the previous chapters are an example, with heads and tails recorded as zeros and ones; so are any other binary data you might encounter), you can write a program that outputs it. There are many such programs. A trivial one: `print "01001..."` with the string written out in full. A clever one: `print the first 100 binary digits of`

`pi`. If the string in question *is* the first 100 binary digits of pi, the clever program is much shorter than the trivial one.

For any string, the most efficient description is the shortest program that produces it. This length is called the *Kolmogorov complexity* of the string, written $K(x)$:

$K(x) =$ the length, in bits, of the shortest program that outputs x.

A string with regularity (the digits of pi, a repeating pattern, the text of a familiar sentence) has small $K(x)$: a short program captures the regularity. A random-looking string has $K(x)$ roughly equal to its own length: no program shorter than "print this string" will do.

So the description language "computer programs" has a natural length function, and the lengths reflect how compressible the strings are.

The library of programs

The set of all programs is easy to describe. A program is a finite string of bits. So the set of all programs is $\{0,1\}^*$: every finite binary string.

This set is large but trivially enumerable. Start with all strings of length one (just "0" and "1"), then all strings of length two, and so on. Figure 4.1 shows the structure.

Most strings, considered as programs, do not parse. Most of the valid ones do not halt (they run forever without producing output). A vanishing fraction produce coherent output. Among those, a smaller fraction produce output matching anything you might be interested in. But every coherent output is in there. Every program that ever has been written is a string in this set. Every program that ever could be written is too.

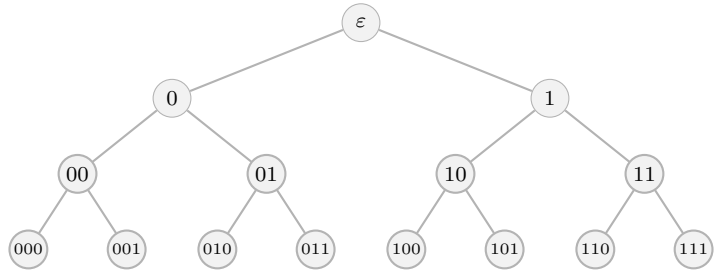


Figure 4.1: The library of all programs. Every program is a finite binary string and sits at some depth in this tree. Generation is trivial: descend the tree, taking 0 or 1 at each step. Most strings, considered as programs, are not valid syntax. Many of the valid ones do not halt. A vanishing fraction produce coherent output. All of them are in the library by virtue of being binary strings.

The universal Turing machine

We need one more ingredient: a machine that takes any program and runs it.

In 1936, Alan Turing showed that such a machine exists. He called it a universal machine. Given any program (any sequence of bits, encoded under a standard scheme), the universal machine runs that program and produces the corresponding output.

In modern language: every computer is a universal machine. Your laptop, given any program in a language it can read, runs it. The universality is mathematical: anything computable by any computer is computable by every computer, up to a constant factor in description length and time. Different computers differ in speed and convenience but not in what they can compute.

We will use U to refer to such a universal machine. The choice

of U is conventional. Different universal machines exist, but for our purposes they are interchangeable: anything you can do on machine U_1, you can do on machine U_2 with a small extra cost (the cost of a translator program from one to the other). This is the *invariance theorem*, and it is why the results below do not depend critically on the choice of U.

The universal prior

Now we put the pieces together.

Take any string x you might want to predict, or describe, or weight. Consider all the programs that produce x when run on U. There are many. Each has a length $|p|$, its size in bits.

The universal prior $M(x)$ is

$$M(x) = \sum_{p\,:\,U(p)=x\star} 2^{-|p|}.$$

The sum is over all programs p such that $U(p)$ produces x as a prefix of its output (the star denotes "with possibly more after"). Each program contributes weight $2^{-|p|}$. Long programs contribute little; short programs contribute a lot. Figure 4.2 shows the structure.

This is Kraft taken to its limit. The description language is "programs for U." The lengths are program lengths. The implied probability over outputs is $M(x)$.

What M inherits from Kraft

A standard technical setup gives U a self-delimiting input convention: each program signals its own end as it is read, so the set of valid programs is prefix-free.[2] Kraft then guarantees that the total weight is bounded: $\sum_x M(x) \le 1$. The universal prior is well-defined.

[2]For the continuous-output formulation used here (programs producing x as a prefix of their output), the conventional setup is a *monotone* Turing machine. A different but related setup, the prefix Turing machine, gives the discrete formula $m(x) = \sum_{p\,:\,U(p)=x} 2^{-|p|}$. Both setups give Kraft-style bounds; the qualitative properties of M in this chapter hold in either.

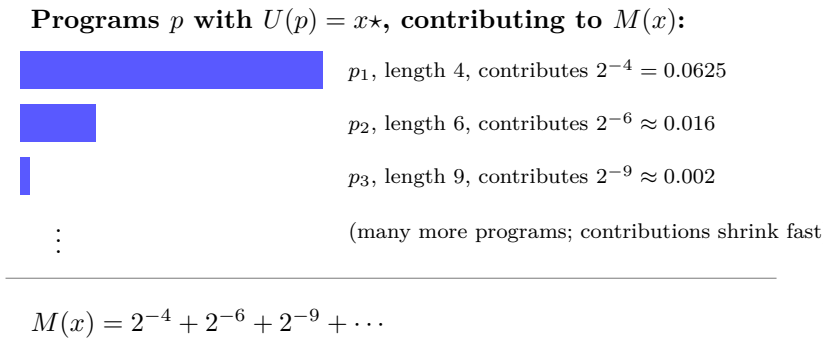


Figure 4.2: The universal prior $M(x)$ as a sum. Every program p that produces x as a prefix of its output contributes $2^{-|p|}$ to $M(x)$. The shortest such program has length $K(x)$ and contributes the dominant term. Strings with short descriptions are heavy. Strings without are light.

What M inherits from programs

A string with low $K(x)$ has at least one short program producing it. That short program contributes $2^{-K(x)}$ to the sum. So

$$M(x) \geq 2^{-K(x)}.$$

For most x, this bound is nearly tight: the description length $-\log_2 M(x)$ equals $K(x)$ to within a term that grows only logarithmically in $K(x)$. The universal prior is, up to that lower-order correction in the exponent, the exponential of the negative Kolmogorov complexity. Compressible things are probable; incompressible things are improbable.

Occam's razor falls out of this. Simpler hypotheses (shorter programs) get higher prior weight, not by stipulation but as a structural consequence of using programs as the description language.

Solomonoff induction

Given the universal prior, Bayesian inference proceeds in the usual way. Suppose you observe data x. To predict whether the next bit is

0 or 1, you compute the conditional:

$$M(\text{next bit} = b \mid x) = \frac{M(xb)}{M(x)}.$$

This is what *Solomonoff induction* refers to: Bayesian inference using M as the prior. Given any history of observations, the predictor's belief about the next observation is well-defined. Update as new observations come in. The procedure is recursive, principled, and parameter-free.

Solomonoff defined this in the 1960s. The puzzle of where the prior comes from, which had bedeviled Bayesian inference for two centuries, was answered by a particular choice that does not look arbitrary: the prior weighted by program lengths.

The dominance theorem

Now the result that makes M optimal.

Suppose there is some true probability distribution μ generating your data. You do not know μ. You want to predict the data, hoping your prior is close enough to μ to be useful.

The dominance theorem says: if μ is computable (or, more precisely, lower-semicomputable as a semimeasure), then there is a constant c_μ such that

$$M(x) \geq c_\mu \cdot \mu(x) \text{ for every } x.$$

In words: M never assigns much less weight to any sequence than μ does. The ratio $M(x)/\mu(x)$ is bounded below by a constant that depends on μ but not on x. Figure 4.3 shows the envelope.

Equivalently: the predictions of M converge to the predictions of μ as data accumulates.[3] After enough observations, M and μ make

[3]This convergence is quantitative and fast. The total expected Kullback-Leibler divergence between μ and M, summed over all steps, is bounded by the description length of μ: $\sum_{t=1}^{\infty} \mathbb{E}_\mu\big[\mathrm{KL}\big(\mu(\cdot \mid x_{<t}) \,\|\, M(\cdot \mid x_{<t})\big)\big] \leq K(\mu) \ln 2$ (Hutter 2001, 2005). Because the bound is finite and independent of sequence length, the per-step divergence must fall to zero: M's predictions converge to μ's,

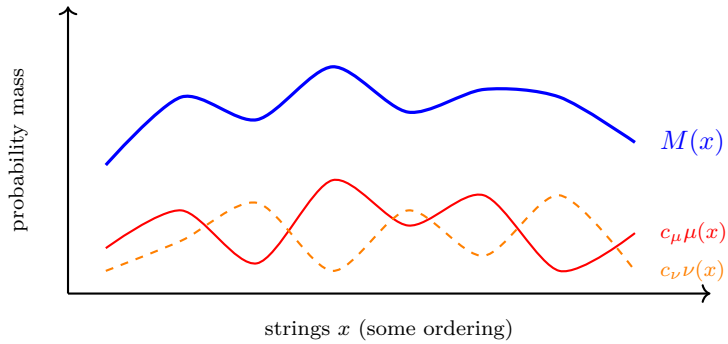


Figure 4.3: The dominance theorem. $M(x)$ (blue, thick) lies above every computable distribution after appropriate scaling. For each computable distribution μ, there exists a constant c_μ such that $M(x) \geq c_\mu \cdot \mu(x)$ for all x. The constants depend on the distribution but not on x. As data accumulates, the predictions of M converge to the true distribution.

essentially the same predictions about what comes next, regardless of which μ is at work.

The result has wide reach. It does not say M is the true distribution. It says: *whatever* the true computable distribution is, M will converge to it. M is contained inside every computable distribution, up to a constant.

This is what optimal induction looks like. You do not need to know what is generating the data. You weight your hypotheses by program length, sum them as M, and your predictions converge to whatever computable process is actually at work.

The catch

M is uncomputable.

To compute $M(x)$ exactly, you would need to enumerate every program that produces x as a prefix of its output and sum their $2^{-|p|}$ contributions. There are infinitely many such programs. Worse, you

and the cumulative cost of not having known μ in advance is at most $K(\mu) \ln 2$ nats.

cannot tell in advance which programs halt (the halting problem); for any given program, you cannot decide whether it will eventually produce x or run forever without doing so.

You can approximate $M(x)$ from below. Run all programs up to some length, for some number of steps, and sum the contributions of the ones that have produced x by then. The approximation improves monotonically with more time. But there is no finite procedure that gives $M(x)$ exactly, and no way to know how close your approximation is.

This is not a flaw in the definition. It is what makes M universal. Any prior you could compute exactly would be simpler in some way than the universal one, and would fail dominance against the priors it could not capture.

The result: M is the right reference point for what optimal induction is. No actual inductor (biological or mathematical or machine) computes M. Every actual inductor is an approximation, faster but coarser.

This is what the rest of the book is about.

Where this leaves you

The hole Bayes left, the choice of prior, is now closed at one level.

The unique optimal prior is M. It dominates every computable distribution. It realizes Occam's razor as a structural consequence of using programs as the description language. It does not depend on conventions, because invariance handles the choice of U.

The cost: M cannot be computed. Any practical inductor approximates it.

Two strands follow.

Theoretical. What does an optimal agent look like? Solomonoff induction is the prediction half. The next four chapters add the decision half: how an optimal agent uses predictions to choose actions. The synthesis is called AIXI.

Practical. How do real systems approximate optimal induction?

They do not run M or anything resembling it directly. They use bounded methods that work in practice. Large language models are one important example. Part IV of the book is about what these systems actually do and how they relate (or fail to relate) to the reference point that M provides.[4]

Part I closes here. The next chapter takes the first step toward agency: the decision problem. If you can predict, how should you act? That is where Part II begins.

[4]There is also a philosophical thread the mathematics raises but does not settle: if the shortest program that fits your observations is the best explanation of them, what does the library of all programs say about where those observations come from? *A Philosophical Note* at the back of the book follows that question as far as it honestly goes, and no further.

Part II

Decision

Chapter 5

The Agent

Reinforcement learning is learning what to do, how to map situations to actions, so as to maximize a numerical reward signal.

Richard Sutton and Andrew Barto,
Reinforcement Learning: An Introduction
(1998)

Most of the AI you have heard about falls into one of three categories. The categories overlap (today's systems often combine them), but reading them out as a list is the simplest orientation tool the field has.

- **Supervised learning.** You give the system many examples of inputs paired with desired outputs. The system learns the input-to-output mapping. Image classifiers learn from labeled images. Translators learn from sentence pairs. The next-token predictors inside large language models learn from sequences where the "target" for each position is just the next token in the text.

- **Unsupervised learning.** You give the system data without labels. The system learns the structure of the data on its own. Clustering, dimensionality reduction, compression, density estimation. There is no target to match; the system tries to describe what it sees.

- **Reinforcement learning.** The system is an *agent* that acts in an environment. The environment responds with observations and a reward signal. The agent's job is to learn a strategy that maximizes total reward over time.

Figure 5.1 places the book's spine on these categories.

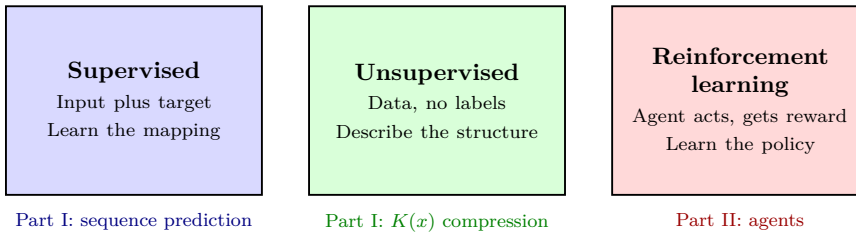


Figure 5.1: The three main categories of machine learning, with the book's spine placed on them. Part I addressed supervised learning in its optimal-limit form (sequence prediction: Solomonoff induction predicts the next observation from a prefix, where the target is just the next bit) and unsupervised learning in its optimal-limit form (Kolmogorov complexity $K(x)$ is the description length of x in its shortest compressed form). Part II addresses the third category: reinforcement learning, with agents acting in environments.

Part I has been about supervised and unsupervised learning in their optimal-limit forms. Solomonoff induction predicts the next observation given a prefix, weighting hypotheses by description length. That is sequence prediction, which is supervised learning where the "label" for each position is the next observation. Kolmogorov complexity is the description length of a string in its shortest compressed form. That is unsupervised compression in its idealized limit.

Part II is the move to reinforcement learning. The agent acts; the world responds; the agent updates and acts again. The mathematics inherits Part I (the agent still forms beliefs, and the beliefs are still Bayesian) but adds one ingredient: *stakes*. The agent has values; the actions have consequences with values attached.

This chapter introduces the agent. The rest of Part II builds out what optimal agency looks like.

What an agent is

An agent is something with three pieces.

- **Perception.** A way of observing the environment. For a robot, sensors. For a language-model agent, text input. For an animal, sight and sound and touch.
- **Action.** A way of acting on the environment. Motors, output tokens, words, movements.
- **Belief and decision.** Internal machinery that takes perceptions in and produces actions out. Beliefs about the environment, plus rules for deciding what to do.

A thermostat is the simplest worked example. Its perception is a thermometer reading. Its action is a switch that turns the heater on or off. Its belief and decision collapse to a single comparison: if the reading is below the target, turn the heater on; if above, turn it off. The thermostat fits in a few lines of logic, and it keeps a house warm. Most other agents are thermostats with more elaborate sensors, more elaborate actions, and more elaborate decision rules.

The agent and the environment form a loop. The agent observes; updates its beliefs; chooses an action; the action affects the environment; the environment generates new observations; the loop continues. Figure 5.2 shows the structure.

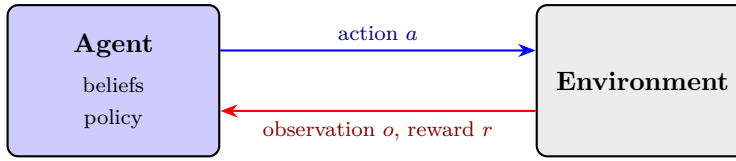


Figure 5.2: The agent-environment loop. The agent takes an action; the environment responds with an observation and (in the reinforcement learning case) a reward. The agent updates its beliefs about the environment and chooses the next action. The cycle continues. Everything in Part II lives inside this loop.

The simplest agent is reactive: it observes the environment and picks an action based on a fixed policy. The interesting agents have memory (state from prior steps), planning (consideration of future consequences), and learning (updating policy over time based on outcomes).

Throughout this chapter, *agent* means any system that follows the perception-action loop with internal state. The mathematics applies whether the agent is a thermostat, a chess program, an animal, or an AI system.

What the agent wants

An agent is not defined by its perception and action alone. It is also defined by what it wants.

The agent's preferences are encoded by a *utility function*, written u. The function maps outcomes to real numbers. Higher numbers mean better outcomes from the agent's view; lower numbers mean worse. Concretely: $u(o) = 5$ means the agent values outcome o at five units; $u(o') = 3$ means it values outcome o' at three. Only the ordering matters in many cases; the units are conventional.

The utility function is the agent's value system. The thermostat's is implicit in what we already described: highest at the target temperature, lower elsewhere. A chess program's might be $+1$ for winning, 0 for drawing, -1 for losing. A human's is much more complex; the function is implicit in behavior rather than written down.

For now, treat u as given. We will return in Chapter 9 to the question of where utility functions come from, why specifying them is harder than it looks, and what happens when the specified utility differs from what the agent (or its designer) actually wants. That is the alignment problem in miniature. For now: u is given.

Decision under uncertainty

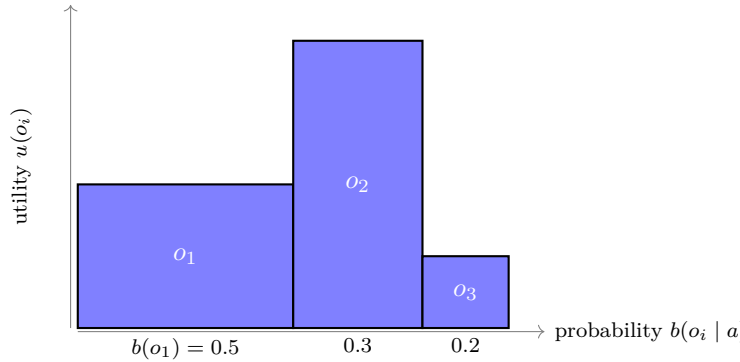
The agent rarely knows for certain what will happen if it takes a particular action. Most actions have multiple possible outcomes, depending on how the environment responds.

So the agent reasons about expected outcomes, weighted by its beliefs about the environment.

The *expected utility* of an action a, given the agent's belief b about the environment, is

$$EU(a) = \sum_o b(o \mid a) \cdot u(o).$$

The sum is over possible outcomes o. The belief $b(o \mid a)$ is the probability of outcome o if the agent takes action a. The utility $u(o)$ is how much the agent values that outcome. Figure 5.3 shows the picture.



$$EU(a) = 0.5 \cdot 2 + 0.3 \cdot 4 + 0.2 \cdot 1 = 1.0 + 1.2 + 0.2 = 2.4$$

Figure 5.3: Expected utility as a weighted sum. Each possible outcome o_i of an action a has a probability $b(o_i \mid a)$ (width of the bar) and a utility $u(o_i)$ (height of the bar). The area of each bar is the outcome's contribution to expected utility. The total area is the expected utility of the action. The agent picks the action whose total area is largest.

The decision rule: **pick the action that maximizes expected utility.**

The agent considers each available action, computes its expected utility under current beliefs, and picks the one whose expected utility is highest.

A worked example: the umbrella

You are deciding whether to take an umbrella to work. Two actions: take it, or leave it. Outcomes depend on whether it rains. Suppose your utilities are:

- Dry without an umbrella (it did not rain, you did not bring one): $u = +2$.
- Dry with an umbrella (annoying to carry, but dry either way): $u = -1$.
- Wet (no umbrella, it rained): $u = -5$.

Suppose the forecast gives $P(\text{rain}) = 0.3$.
Expected utility of taking the umbrella:

$$EU(\text{take}) = 0.3 \cdot (-1) + 0.7 \cdot (-1) = -1.$$

(You carry it either way; the weather does not matter for this action.)

Expected utility of leaving it:

$$EU(\text{leave}) = 0.3 \cdot (-5) + 0.7 \cdot 2 = -1.5 + 1.4 = -0.1.$$

-0.1 beats -1. Leave the umbrella.

Now suppose the forecast updates: $P(\text{rain}) = 0.7$.

$EU(\text{take}) = -1$ (unchanged). $EU(\text{leave}) = 0.7 \cdot (-5) + 0.3 \cdot 2 = -2.9$.

Now -1 beats -2.9. Take the umbrella.

The belief changed; the utility function did not. The decision flipped. That is what decision under uncertainty means: the right action depends on what you think the world will do.

Why expected utility

You might ask why this rule. Why not the best worst case (the minimax pessimist)? Why not the best best case (the maximax optimist)? Why not something nonlinear in the probabilities, like prospect theory's risk-weighted version?

In 1944, John von Neumann and Oskar Morgenstern showed that under modest consistency requirements, any rational agent acts *as if* it is maximizing the expected value of some utility function.

The requirements are short:

- **Completeness.** For any two outcomes, the agent has a preference: either it prefers one, or the other, or it is indifferent.
- **Transitivity.** If the agent prefers a to b, and b to c, then it prefers a to c.
- **Continuity.** If the agent prefers a to b to c, then there is some probability mixture of a and c that the agent is indifferent to receiving in place of b.
- **Independence.** Preferences between two gambles do not depend on irrelevant common parts: if you prefer a to b, you also prefer "a or c each with probability one-half" to "b or c each with probability one-half."

Under these axioms, the agent's preferences over gambles can be represented by a utility function such that the agent prefers gamble g_1 to gamble g_2 if and only if the expected utility of g_1 exceeds the expected utility of g_2.

What goes wrong when the axioms fail

To see why the axioms matter, consider what happens when they are violated. Suppose your preferences fail transitivity: you prefer A to B, B to C, and yet C to A.

I can drain your money indefinitely. Start with you holding A. I offer to trade A for C plus a small fee. You accept (you prefer C).

Now you hold C. I offer to trade C for B plus another fee. You accept (you prefer B). Now you hold B. I offer to trade B for A plus another fee. You accept (you prefer A). You are back to holding A, three fees lighter. The cycle can run as long as you have money.

This is the *money pump* argument. Any agent with intransitive preferences can be drained without limit. Transitivity is not arbitrary; it is the requirement that prevents you from being a perpetual mark.

Similar arguments cover the other axioms. Violating independence makes you vulnerable to constructed gambles (the Allais and Ellsberg paradoxes are well-known cases where humans systematically violate independence and the consequences can be exploited). Violating continuity gives an exploit at the boundary between alternatives. Each axiom corresponds to closing a specific exploit.

The full vNM result is: the only way to rank gambles consistently, without being exploitable, is to maximize expected utility for some utility function.

Other criteria, in disguise

Some criteria that look unlike expected utility maximization turn out to be expected utility maximization with a different utility function. Minimizing maximum regret (the gap between what you got and what you could have got with perfect hindsight) is expected utility maximization with utility set to negative regret. Risk-aversion can be expressed by making utility a concave function of dollars. The framework is broader than it might look at first; what changes between criteria is the utility function, not the underlying rule.

This is structurally the same kind of result as Cox's theorem from Chapter 1. Cox showed that consistency forces belief revision to be probabilistic. Von Neumann and Morgenstern show that consistency forces decision under uncertainty to be expected utility maximization. In both cases, a small set of innocent-looking requirements collapses the space of options to a single answer.

It does not say what the utility function *is*. It says: whatever your preferences, if they are consistent, they can be represented this

way. Where the preferences come from is, like the prior, a separate question.

Where this leaves you

You have an agent. It perceives. It acts. It has beliefs (from Part I, updated by Bayes with a principled prior). It has a utility function over outcomes (the value system, given for now). It decides by computing expected utility for each available action and picking the maximum.

This is the core of decision theory. The rest of Part II builds out its consequences.

Chapter 6 (Reinforcement Learning) takes the static picture and adds time. The agent does not decide once; it decides repeatedly, with future actions depending on observations not yet made. Exploration vs exploitation enters as a structural feature: should the agent take its best current bet, or try something less proven in order to gain information that might lead to a better bet later? Value of information is what answers this question.

Chapter 7 (Generalization) confronts a fact the static picture hides: a real agent meets more situations than it could ever tabulate, so it must generalize from finite experience to the unseen. That means function approximation, inductive bias, and the architectures of modern machine learning.

Chapter 8 (AIXI) synthesizes Part I and Part II. Solomonoff induction for the prediction half. Expected utility maximization for the decision half. Combined: AIXI, the optimal universal agent. Uncomputable, but the right reference point for what intelligence is in the limit.

Then Part III opens. **Chapter 9 (Reward Modeling)** returns to the utility function. We have been treating u as given. Specifying it is harder than it looks. Goodhart's law is the warning sign; the alignment problem is what follows when the specification fails at scale.

The next chapter takes the first step.

Chapter 6

Reinforcement Learning

> *An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with respect to the state resulting from the first decision.*
>
> Richard Bellman, *Dynamic Programming* (1957)

Chapter 5 gave the static picture. An agent has beliefs, utilities, and a decision rule (maximize expected utility). It evaluates each action and picks the best one.

Real agents are not static. They act repeatedly. Today's action changes tomorrow's options. The agent does not just decide once; it decides, observes the consequence, updates its beliefs, decides again. Its current decision is shaped by what it expects to be able to do later, given how the environment will have changed.

This is the setting of *reinforcement learning*. RL is what decision theory looks like when time enters, when the agent must learn while acting, and when actions have long downstream consequences. The mathematics is rich, the algorithms are central to modern AI, and the framework is what powers everything from chess engines to LLM fine-tuning.

This chapter introduces the core ideas. The next chapter combines RL with Solomonoff induction to define the optimal universal agent.

The setting

The world ticks forward in discrete time steps $t = 0, 1, 2, \ldots$. At each step:

- The agent observes the current state s_t of the environment.
- The agent takes an action a_t.
- The environment transitions to a new state s_{t+1} according to some probability $p(s_{t+1} \mid s_t, a_t)$.
- The environment delivers a reward r_t.

The loop is the same one from Chapter 5, with two additions: there is now an explicit reward signal, and the same loop runs repeatedly with state carried over. Figure 6.1 shows the picture.

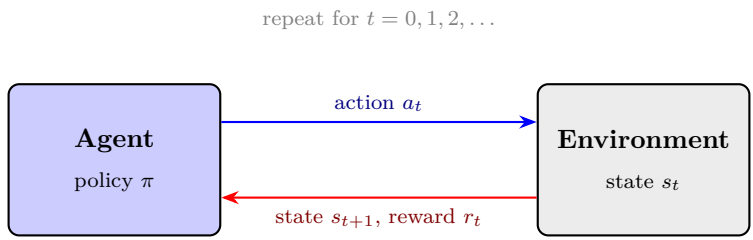


Figure 6.1: The reinforcement learning loop. The agent observes the state, takes an action, and receives the next state and a reward. The same loop runs at every step. The agent's job is to learn a policy that yields high reward over time.

A *policy* π is a rule that maps states to actions (or to probability distributions over actions). The agent's job is to find a good policy.

What "good" means is the next question.

What the agent wants in RL

In Chapter 5 the agent had a utility u over outcomes. In RL, the utility is broken into per-step rewards r_t, summed over time.

The agent wants to maximize total reward. But "total" over an infinite horizon needs careful handling. The standard move is to use a *discount factor* $\gamma \in [0, 1)$ and define the return from time t as

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

Future rewards count, but with decreasing weight. With γ less than one, the infinite sum converges.

This idea, that future rewards are worth less than present ones, is called *time discounting*, and it is much older than RL. Economists have used it since at least Böhm-Bawerk (1889) to explain interest rates: a dollar today is worth more than a promised dollar a year from now, and the difference is the rate at which the future is discounted. Net present value calculations in finance rest on exactly this idea. Psychologists have documented that humans discount future rewards more steeply than exponentially (Ainslie's *hyperbolic discounting*), which is part of why delayed gratification is hard. Foraging animals show similar discounting in food choices, with evolutionary justification: the future is uncertain (you might not survive to collect), and selection rewards taking sooner when you can.

The discount factor γ in RL is this idea applied to a sequence of decisions. The choice of γ encodes the agent's time preference. γ near 1 means the agent is patient (future rewards count nearly as much as present ones); γ near 0 means it is myopic (only current rewards matter much). Most practical RL uses γ between 0.9 and 0.99.

The agent's goal, then, is to choose a policy that maximizes the expected return.

Value functions

Two functions encode what the agent has learned about its environment.

The *state-value function* under policy π:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s].$$

The expected discounted return starting from state s and following policy π thereafter. "How good is being in state s, if I keep playing π."

The *action-value function* (or Q-function):

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

The expected return starting from state s, taking action a, and following π thereafter. "How good is taking action a in state s, if I then keep playing π."

Bellman noticed that these functions satisfy a recursive equation. The value of s equals the immediate reward expected from s plus the discounted value of where you end up:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V^\pi(s')\big].$$

This is the *Bellman equation* for V^π. The values of all states are tied together: knowing the value of any state requires knowing the values of its successors, which require their successors, and so on.

The optimal value function V^* satisfies

$$V^*(s) = \max_a \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V^*(s')\big].$$

And the optimal policy is the one that picks, at each state, the action whose successor states have the highest expected value:

$$\pi^*(s) = \arg\max_a \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V^*(s')\big].$$

If you know V^* (or equivalently Q^*), you know how to act. The decision rule reduces to a one-step lookup.

Learning

So far we have assumed the agent knows the environment: it can compute $p(s' \mid s, a)$ and $r(s, a, s')$. In real settings, the agent does not know these. It has to learn them by acting.

This is the *learning* in reinforcement learning.

The simplest algorithm is *Q-learning* (Watkins 1989). The agent keeps a table of estimates $Q(s, a)$. Each time it takes action a in state s and observes reward r and next state s', it updates:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big].$$

The bracketed term is the *temporal difference*: the difference between the agent's current estimate of $Q(s, a)$ and the new estimate it gets from one step of experience. The learning rate α controls how fast the estimate moves toward the new value.

Under standard conditions (each state-action pair visited infinitely often; learning rate decaying appropriately), Q-learning converges to Q^*. The agent has learned the optimal value function from experience alone.

A subtle point. Q-learning's update uses $\max_{a'} Q(s', a')$ regardless of what action the agent actually took next. This makes Q-learning *off-policy*: it learns about the optimal policy while potentially behaving differently. The agent can take entirely random actions and still converge to Q^*, as long as every state-action pair is visited often enough. *On-policy* methods (like SARSA) instead update using the action the agent actually took, and so learn the value of the behavior policy rather than the optimal one.

The behavior policy itself is usually somewhere between fully random and fully greedy. The simplest choice, *epsilon-greedy*, takes the currently-best action $\arg\max_a Q(s, a)$ with probability $1 - \epsilon$ and a random action with probability ϵ. The next two sections take up this exploration-vs-exploitation trade-off properly. For now: the agent's policy is better thought of as a distribution $\pi(a \mid s)$ over

actions than as a single deterministic choice. Ties must be broken, stochastic environments sometimes reward stochastic policies, and the exploration/exploitation balance is itself a question of how much probability to put on uncertain options.

Exploration versus exploitation

There is a tension in Q-learning that the update rule does not resolve.

To learn Q^*, the agent must try each action in each state enough times to estimate its value reliably. This is *exploration*: try things to learn about them.

But the agent also wants to act on what it knows. Each step is real; rewards count. The agent does not want to waste them on actions it already knows are bad. This is *exploitation*: use the current estimate to get the best reward you can.

These pull in opposite directions. If the agent always exploits, it might lock into the first action that worked and miss a better one. If it always explores, it gives up reward chasing knowledge.

The cleanest setting in which to study this tension is the *multi-armed bandit*. There are K slot-machine arms with unknown reward distributions. The agent chooses one arm per turn and observes the reward. After many turns, which arms did it pull? Figure 6.2 shows the situation after some play.

Several practical policies balance exploration and exploitation.

Epsilon-greedy. With probability $1 - \epsilon$, take the best-known action. With probability ϵ, take a random one. Simple, widely used, has known suboptimalities.

UCB (upper confidence bound). At each step, pick the action that maximizes "best estimate plus uncertainty." Concretely: choose the arm maximizing $\hat{\mu}_a + c\sqrt{\log t / n_a}$, where $\hat{\mu}_a$ is the empirical mean, n_a is the number of pulls so far, and c is a constant. The agent is optimistic about arms it has not tried much, because their uncertainty term is large. Auer, Cesa-Bianchi, and Fischer (2002).

Thompson sampling. Maintain a probability distribution (a

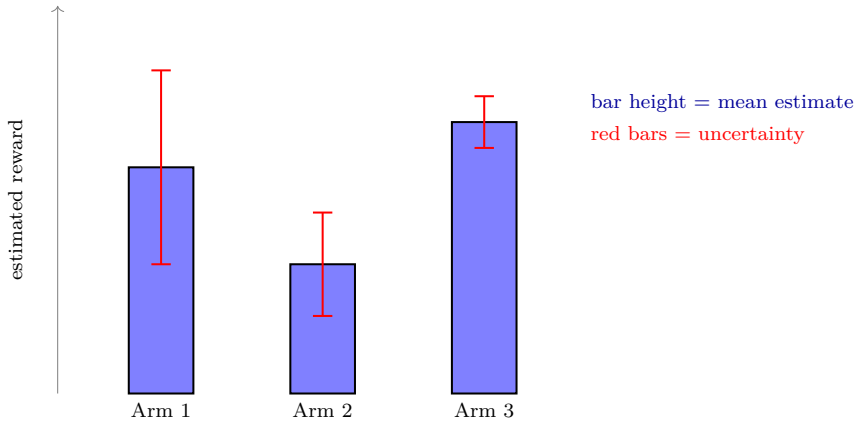


Figure 6.2: A three-armed bandit after some number of pulls. Arm 1 has been pulled some; the estimate is decent but the uncertainty is large. Arm 2 has been pulled a fair amount and looks worse than Arm 1. Arm 3 has the highest estimate and the tightest uncertainty. The pure exploiter plays Arm 3 always. The pure explorer keeps pulling all three. A good policy plays Arm 3 most of the time but occasionally pulls Arm 1 (whose true mean might be above its estimate, given the large uncertainty).

posterior) over each arm's parameter. At each step, sample once from each posterior; play the arm whose sample is highest. Bayesian, often near-optimal, has clean theoretical guarantees.

What all three have in common: they treat "how much I do not know" as itself a quantity to factor into decisions. Uncertainty is not just a problem; it is a signal.

The value of information

How much is knowing worth?

Howard (1966) gave the mathematical answer. The *value of information* of an observation o is the expected improvement in decision quality from receiving it:

$$\mathrm{VoI}(o) = \mathbb{E}\big[\max_a EU_{\text{after } o}(a)\big] - \max_a EU_{\text{before}}(a).$$

The first term is the expected value of acting optimally after observing o, averaged over what o might turn out to be. The second is the expected value of acting optimally without the observation. The difference is what the observation is worth.

VoI is what exploration-exploitation policies are computing, in disguise. Each policy is a rule of thumb that approximates "would the information gained from trying this action be worth more than the reward I would forgo by not exploiting?"

The general lesson: information has a price. Optimal agents do not just maximize current expected utility. They also weigh how much each action would teach them, because tomorrow's decisions can be better if today's action provides useful data.

Where this leaves you

You now have the engineering picture of optimal agency. The agent operates in a state-action-reward loop. Its goal is to maximize discounted reward. Its knowledge is encoded in value functions or Q-functions. It learns by experience using algorithms like Q-learning. It balances exploration and exploitation using the value of information, implicit in policies like UCB and Thompson sampling.

Two issues are unresolved.

The first is engineering. Tabular Q-learning works on toy problems but does not scale. Chess has about 10^{44} legal positions; Go has about 10^{170}; even Pac-Man has more states than an agent can visit. Almost every real state is new. A tabular agent cannot say anything about a state it has not seen. To handle real problems, the agent has to generalize from the states it has seen to ones it has not.

The second is theoretical. We have an optimal predictor (Solomonoff, Part I) and an optimal decider (expected utility maximization, Chs 5-6). They are not yet combined.

Chapter 7 (Generalization) takes up the first issue. Function approximation replaces the table; learned representations let the agent extrapolate; inductive biases are what make the extrapolation

work; Solomonoff appears at the limit. This is the conceptual bridge between optimal-agent theory and practical-agent reality.

Chapter 8 (AIXI) takes up the second. Solomonoff induction for the belief; expected utility maximization for the action; together, AIXI. The optimal universal agent. Uncomputable, but the right reference point.

Chapter 9 and beyond (Part III) return to the question Chapter 5 deferred: where does the utility function come from? Specifying it is the hardest part of building intelligent agents. Goodhart's law is the structural warning. The alignment problem is what follows when the specification fails at scale. That problem gets its own part.

The next chapter takes up generalization.

Chapter 7

Generalization

In the absence of any specific knowledge about the problem, no algorithm can be expected to outperform any other.

David H. Wolpert and William G. Macready, "No Free Lunch Theorems for Optimization" (1997)

The previous chapter ended with a problem. Tabular Q-learning works on toy environments but does not scale. Chess has about 10^{44} legal positions. Go has about 10^{170}. Even Pac-Man has more states than an agent can visit in any feasible amount of training. Almost every state the agent encounters in play is new: it has never seen exactly this configuration before.

A tabular agent has nothing to say about a state it has not seen. Its Q-table has no entry. It cannot *generalize*.

Generalization is the central problem of machine learning. Given a finite sample of experience, the agent has to behave sensibly on the much larger universe of situations it has not yet seen. This chapter is about how that works, why it is possible at all, and what it costs.

From a table to a function

The fix is to stop thinking about Q as a table and start thinking about it as a function. The Q-value $Q(s, a)$ becomes the output of a parameterized function $Q_\theta(s, a)$, where θ are the function's

parameters. Figure 7.1 shows the difference.

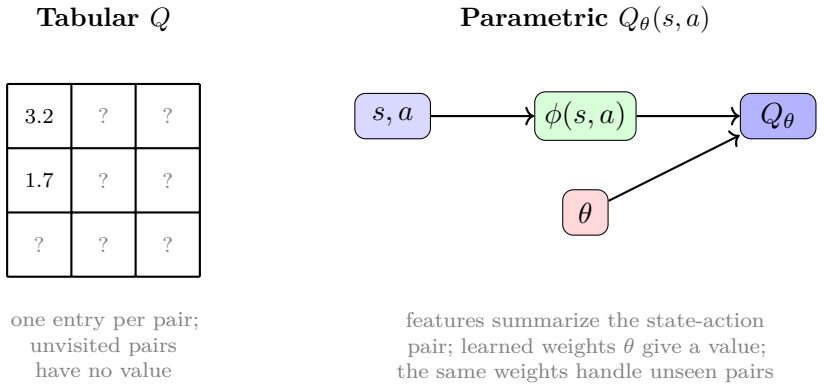


Figure 7.1: Tabular vs parametric Q. The tabular Q-function stores one estimate per state-action pair; unseen pairs have no value. The parametric Q-function processes any state-action pair through features and a shared set of learned weights, producing values even for pairs the agent has never seen. Generalization comes from the function's structure, not from individual observations.

The agent learns θ from experience. When it observes (s, a, r, s'), instead of updating one table entry, it updates θ in the direction that makes $Q_\theta(s, a)$ closer to the temporal-difference target. The update typically changes Q_θ at *other* state-action pairs too, including pairs the agent has never visited. The function generalizes.

The simplest case is linear function approximation. Define a feature vector $\phi(s, a)$ describing the state-action pair. Approximate $Q(s, a) \approx \theta^T \phi(s, a)$. Learn θ by gradient descent.

For Pac-Man, useful features might be: distance to the nearest ghost, distance to the nearest food pellet, how many ghosts are in scared mode, whether a power pellet is reachable. A handful of carefully chosen features with learned weights can produce a competent Pac-Man player after a few hundred games. A tabular method would need millions.

Neural networks, briefly

For most interesting problems, the right features are not obvious. You do not know which projections of the state matter. Then the features themselves have to be learned.

A neural network is a parameterized function $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$. It takes a real-valued input vector and produces a real-valued output vector. The parameters θ are the weights and biases of its layers. Different choices of θ give different functions; learning is finding the θ that fits the training data well.

The simplest version is a single linear layer:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}.$$

Take an input vector $\mathbf{x}$, multiply by a matrix of weights W, add a bias vector $\mathbf{b}$. This is a linear function. It can fit any input-output mapping that is itself linear. It cannot fit anything else. The classic counterexample is the XOR function: it cannot be expressed as a linear combination of its inputs. A single linear unit cannot learn XOR no matter how long you train it.

The fix is to stack layers and put a nonlinear function between them.

$$\begin{aligned}
\mathbf{h}_1 &= \sigma(W_1\mathbf{x} + \mathbf{b}_1) \\
\mathbf{h}_2 &= \sigma(W_2\mathbf{h}_1 + \mathbf{b}_2) \\
&\vdots \\
\mathbf{y} &= W_L\mathbf{h}_{L-1} + \mathbf{b}_L
\end{aligned}$$

Here σ is a nonlinear function applied componentwise (a common choice is the rectified linear unit, $\sigma(z) = \max(0, z)$). Without σ, the composition of linear layers is just another linear function. With σ, the composition can represent functions arbitrarily complex. This is a *multi-layer perceptron* (MLP) or, more loosely, a *deep network* when L is large. Figure 7.2 shows the structure.

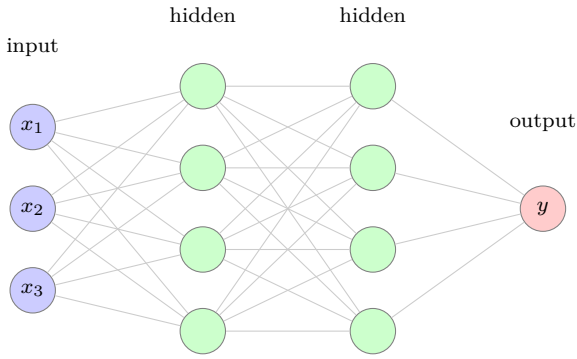


Figure 7.2: A small multi-layer perceptron with three input units, two hidden layers of four units each, and one output. Every edge is a learned weight; every hidden unit applies a nonlinear activation to its weighted sum of inputs. The function as a whole is parameterized by the collection of all weights. Real networks are larger (millions to trillions of parameters) and use specialized architectures, but the basic structure is this.

That is the core picture. Training is finding weights that make the network's outputs match the targets. The algorithm is gradient descent on a loss function; the gradient is computed efficiently by backpropagation (Rumelhart, Hinton, Williams 1986). The mathematics is not particularly difficult; the engineering challenge has been building hardware and software that can handle networks at the scales modern problems demand.

In the RL setting, the Q-function $Q_\theta(s, a)$ is the output of a neural network. The state goes in (as raw pixels, as encoded features, as a token sequence); the value comes out. Deep Q-Networks (DeepMind 2013) applied this to Atari games, achieving superhuman performance on most of them using only raw pixel input. AlphaGo (2016) combined deep networks with tree search and beat the human world champion at Go. The pattern is the same: the network learns whatever features matter for value.

Everything is a universal approximator

A point worth making explicit, because the rest of the chapter hinges on it.

The table from the previous section is a universal function approximator. Given enough rows, it can represent any function from inputs to outputs exactly. A linear model is also universal in the trivial sense of representing every linear function. A multi-layer perceptron is universal in a deeper sense: Cybenko (1989) and Hornik (1991) proved that an MLP with one hidden layer and enough units can approximate any continuous function on a bounded domain to arbitrary precision. Convolutional networks, recurrent networks, transformers, mixtures of experts: all universal in the same approximation-theoretic sense.

Universality is cheap. Approximating any function in the limit, given unlimited capacity and unlimited data, is not the interesting property. The interesting property is sample efficiency: how much data does the representation need before it generalizes correctly to inputs it has not seen?

A table needs one observation per cell. A linear model with k features needs on the order of k observations to fit. A transformer trained on internet-scale text can sometimes generalize from a handful of in-context examples to tasks it was never explicitly trained on, because its architecture imposes structure that lets it interpolate across nearby inputs without having seen them.

This is the load-bearing fact: the choice of representation is not a choice about what the system can in principle compute. It is a choice about what kinds of inputs it can generalize across efficiently. The representation is where the inductive bias lives.

A useful way to see this is to think of the representation as a geometry over the input space. The architecture is a recipe for mapping inputs to points in some feature space. If similar inputs (in the sense that matters for the task) land in nearby points, the network's learned function will produce sensible outputs on unseen nearby inputs by continuity. If similar inputs land in unrelated points,

the network has to learn each one separately, and finite data will not be enough. The art of architecture design is arranging the geometry so that the similarities that matter become spatial closeness in the representation. Figure 7.3 shows the difference.

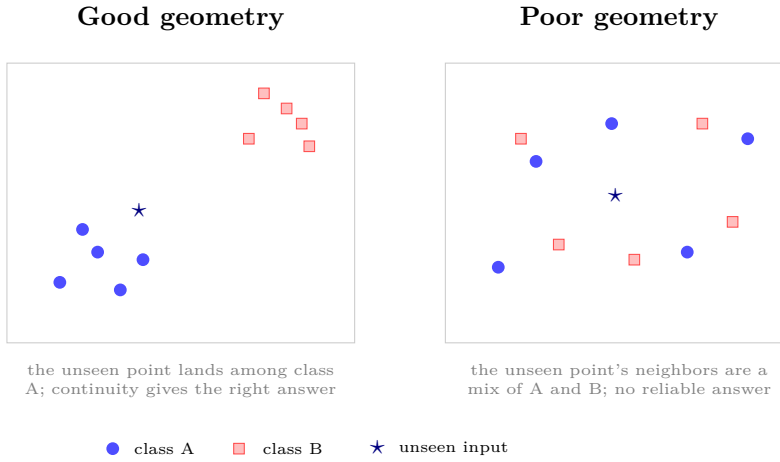


Figure 7.3: Representation as geometry. Both panels show the same five examples of class A (circles), five of class B (squares), and one unseen input (star) to be classified. Left: a representation in which the features that matter place each class in its own region. The unseen input lands among class A, and the learned function gives it class A's answer by continuity, without ever having seen it. Right: a representation in which the same inputs scatter. The unseen input's nearest neighbors are a mix of both classes, so no amount of training data on these points tells the function what to do with it. Same inputs, same classes, different geometry; only the left one generalizes.

Output heads work on the same principle. A linear classification head says "the right output is a linear combination of these features." A softmax over a vocabulary says "the right output is one of these discrete options, weighted by feature combinations." A mixture-of-experts head says "different inputs should route through different sub-functions." Each is a bias about how the representation should combine into outputs.

The sharpest test of whether a representation's biases match

the world is not in-distribution sample efficiency, where the test inputs come from the same distribution as the training data. It is out-of-distribution generalization: does the system behave sensibly on inputs that look different from training but share underlying structure? A representation that does well in-distribution and poorly out-of-distribution has learned the surface statistics of the training data without the underlying structure. A representation that does well out-of-distribution has built the right kind of structure into its geometry. Most of what we want from advanced AI systems is OOD generalization in this sense: behaving sensibly on situations the training set did not contain.

No free lunch

You do not get function approximation for free.

The *no-free-lunch theorems* (Wolpert and Macready 1997) say that no learning algorithm is uniformly best across all possible problems. Averaged over every conceivable input-output mapping you might want to learn, every algorithm performs the same. Any algorithm that does well on the problems we actually care about (structured environments, hierarchical patterns, discoverable regularities) does so because it has assumptions baked in that match the structure of those problems.

These assumptions are called *inductive biases*. Every learning system has them. The choice of features, the network architecture, the activation function, the regularization, the loss function, the training data: all encode inductive biases. They constrain the space of functions the system can express easily, and that constraint is what lets it learn from finite data.

Different architectures encode different biases:

- **Convolutional networks** bias toward translation invariance and local structure: a feature in one part of the image is recognized in any other part. Good for images, where the same patterns recur at different locations.

- **Recurrent networks** bias toward sequential dependence: each step's output depends on a hidden state carried from previous steps. Decent for time series; surpassed by transformers for language.

- **Transformers** bias toward attention-mediated long-range dependencies: any position in a sequence can directly attend to any other position. The winning architecture for language so far, and increasingly for vision too.

- **Pre-trained foundation models** bias toward whatever statistical structure was present in the pre-training data. Training a transformer on internet-scale text bakes in implicit knowledge about language, the world, common arguments, and the structure of human communication. This pre-training is itself an inductive bias for downstream tasks.

The choice of architecture is not aesthetic. It is a choice about which problems the system can learn efficiently.

Sample efficiency and inductive bias are two ways of describing the same property. A learner with a perfect inductive bias for a problem learns it from one observation (in the limit). A learner with the wrong inductive bias may never learn it. Most real learners sit somewhere in between. Improving them often means improving the bias more than improving the algorithm.

Solomonoff as the limit case

The connection back to Part I: Solomonoff's universal prior is the most general inductive bias possible.

Solomonoff weights hypotheses by description length: $M(x) \approx 2^{-K(x)}$. The bias is "simpler explanations are more probable." It works uniformly across every problem with computable structure, which (under the Church-Turing thesis) is every problem there is. It is universal.

But uncomputable. Real learning systems cannot run Solomonoff. They use bounded inductive biases instead. A transformer's bias is far more restrictive than Solomonoff's, but the transformer is something you can actually train on a real dataset in a finite amount of time. The architecture trades coverage for tractability.

This is the central design trade-off in modern machine learning. Pick a bias too narrow, and you cannot represent what you need. Pick it too broad, and you cannot learn from feasible data. The art is in matching the bias to the structure of the world you want the system to handle.

Figure 7.4 sketches the trade-off.

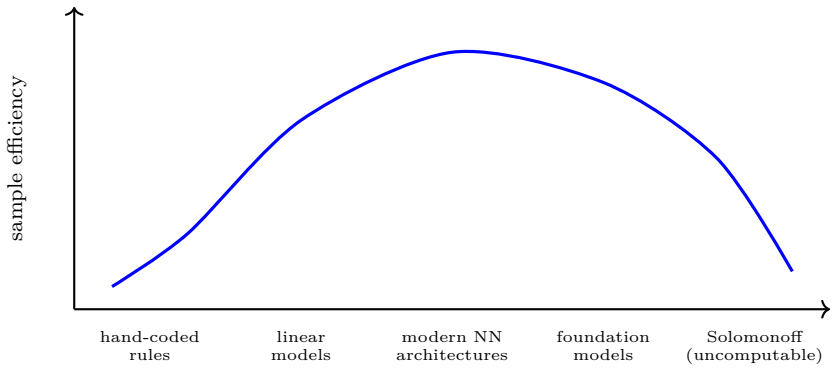


Figure 7.4: The inductive-bias trade-off. The horizontal axis is the breadth of inductive bias, from narrow (hand-coded rules) to maximally broad (Solomonoff). The vertical axis is sample efficiency on the kinds of problems the system is actually applied to. A narrow bias cannot express what you need; a maximally broad bias (Solomonoff) covers everything but is uncomputable. The sweet spot for real problems sits somewhere in between: bias broad enough to express the structure of the problem, narrow enough to learn from feasible data. The dominant architectures of modern AI are points on this curve, each a particular bet about which problems matter and what their structure is.

The dominant systems of modern AI (transformers for language, CNNs for vision, RL with deep function approximation for control) are not arbitrary choices. They are particular bets about what inductive

biases match the structure of language, of visual scenes, of sequential decision problems. The bets have paid off so far. They have not paid off everywhere; there are domains where current architectures struggle, often because the inductive bias is wrong for the structure of the problem.

Where this leaves you

You now have the conceptual machinery for understanding why modern AI looks the way it does. Function approximation replaces the table. Neural networks learn their own features. The choice of architecture is the choice of inductive bias. Sample efficiency comes from biases that match the problem's structure. Solomonoff sits at the limit, universal but unreachable.

This chapter sits at a hinge in the book. Parts I and II built the theory of optimal agency in clean settings (Solomonoff, EU, RL). Part III will treat the specification problem (reward modeling, alignment, oversight) in similarly clean settings. Part IV will move to messy practice: LLMs, what we observe when alignment is tested in real systems, where the gap lives.

But first, one more piece of theory. The next chapter combines the optimal predictor (Part I) and the optimal decider (Chapter 6, plus the generalization machinery of this one) into a single object: AIXI, the optimal universal agent. It is the reference point that the rest of the book points toward, and it is the cleanest statement of what intelligence is in the limit.

Chapter 8

AIXI

Intelligence measures an agent's ability to achieve goals in a wide range of environments.

Shane Legg and Marcus Hutter, "Universal Intelligence: A Definition of Machine Intelligence" (2007)

Part I built one half of the picture. Part II built the other.

Part I was about prediction. Bayesian inference is the unique consistent calculus of belief. Solomonoff induction is what optimal prediction looks like once the prior problem is closed: weight every computable hypothesis by $2^{-|p|}$ where p is the program producing it. The universal prior M catches up to any computable distribution generating your data.

Part II was about decision. Expected utility maximization is the unique consistent calculus of choice. Reinforcement learning is what optimal decision looks like over time, with rewards and uncertainty. Function approximation lets the agent generalize from seen states to unseen ones; inductive biases are the engineering content of generalization.

Both halves are uncomputable in idealized form. Neither alone tells you what intelligence *is*. This chapter combines them.

The setup

Imagine an agent embedded in an unknown environment. The environment is some program μ that, given the agent's past actions, produces observations and rewards. The agent does not know which program μ is. It can only act, observe, and try to do well.

What should the agent believe about μ? It should treat μ as a hypothesis to be inferred from data.

What is the right prior over μ? Part I gave the answer: the universal prior. Weight every candidate environment-program by $2^{-K(\mu)}$, the cost of its shortest description.

What should the agent do? Part II gave the answer: pick the action that maximizes expected discounted reward, where the expectation is computed using the agent's current beliefs about μ.

Combine these two answers and you get AIXI.

AIXI defined

Let $h_t = a_1 o_1 r_1 \cdots a_t o_t r_t$ be the history of actions, observations, and rewards through step t. At step $t+1$, AIXI chooses the action

$$a_{t+1} = \arg\max_a \sum_{o,r} \sum_{\nu} 2^{-K(\nu)} \cdot \nu(o, r \mid h_t a) \cdot \big[r + \gamma \cdot V^*(h_t a o r)\big],$$

where ν ranges over all computable environments (more precisely, lower-semicomputable semimeasures), γ is the discount factor, and V^* is the best expected discounted return achievable from a given history. Each symbol is unpacked below; V^* in particular is defined two paragraphs on, so read past it for now.

The two outer sums encode an expectation. The sum $\sum_{o,r}$ ranges over every possible next-step outcome: every (observation, reward) pair the environment could produce after the agent takes action a. The sum $\sum_\nu$ ranges over every candidate environment-program, weighted by its prior $2^{-K(\nu)}$. Together they marginalize jointly over

what the true environment is and what its next response will be, producing the expected value of $r + \gamma V^*(h_t aor)$ under AIXI's full posterior. The $\arg\max_a$ on the outside then picks the action whose expected value is largest.

The remaining symbols are string-valued and deserve a closer look. The notation $h_t a$ is the history-so-far with the candidate action appended: take everything that has happened through step t and write a on the right of it. The notation $h_t aor$ extends the history one step further still, appending the resulting observation o and reward r. The conditional $\nu(o, r \mid h_t a)$ then reads as follows: given the full history through step t and given that the agent now takes action a, what probability does environment-program ν assign to the next observation being o and the next reward being r? And $V^*(h_t aor)$ is the optimal expected discounted return from the point at which the agent's history is exactly $h_t aor$, after this hypothetical step has played out.

One design choice deserves naming. AIXI conditions on *histories*, not on states. Chapter 6 worked with Markov decision processes, where the current state captured everything relevant about the past. AIXI does not assume a Markovian environment. The candidate environment-programs ν can have arbitrary internal structure and respond to any portion of the past, so the agent conditions on the full observable history. V^* is recursive: unrolled, it considers every future trajectory each ν admits, with AIXI picking the best action at each future step.[5]

This may seem to backtrack from Chapter 7, which insisted that real systems need state representations: features, neural-network embeddings, learned compressions of the past. AIXI throws all of that away and conditions on the raw history.

[5]The equation is therefore self-referential. V^* is the value of acting optimally from a history, and acting optimally means applying this very action-selection rule at every subsequent step, so V^* appears, implicitly, on both sides. It is a fixed point: the value function consistent with choosing the best action everywhere downstream. This is the Bellman optimality equation of Chapter 6, lifted from a single known environment to AIXI's posterior mixture over all computable environments.

It does so on purpose. The representation work has not disappeared; it has moved into the prior. Each candidate environment-program ν is, in effect, its own way of organizing the past: ν's internal state-tracking is its representation of the history. Different ν's use different representations. The universal prior $2^{-K(\nu)}$ weights them all, with simpler representations getting more weight.

So AIXI does not lack representations. It runs all of them in parallel, weighted by simplicity, and lets the data sort out which one is correct. A practical agent commits to a finite, manageable set of representations. It might use one architecture, or a small ensemble, or a mixture of experts that routes different inputs to different sub-networks, or a multi-head attention module that effectively learns several representations within a single architecture; in every case the set is finite and chosen at design time. That is the architecture choice from Chapter 7. AIXI does not commit to any such set. It weights every computable representation. Refusing to commit is what makes it universal. It is also what makes it uncomputable. On the inductive-bias trade-off curve of Chapter 7, AIXI sits all the way at the universal end. Every practical agent sits to the left, having committed to a finite set of biases in exchange for being something that can actually run.

The equation is dense. Figure 8.1 unpacks the pieces and shows the loop they describe.

In plain language: AIXI considers every computable theory of what the environment might be. It weights each theory by how short it is to describe. It computes, for each candidate action, the expected reward (discounted, summed over time) the action would produce, taking the expectation over its weighted theories of the environment. It picks the action whose expected reward is highest.

Three pieces, each from elsewhere in the book:

- The **prior** $2^{-K(\nu)}$ over environments comes from Solomonoff (Part I, Ch 4).
- The **posterior** is Bayesian updating on the observed history (Part I, Ch 1).

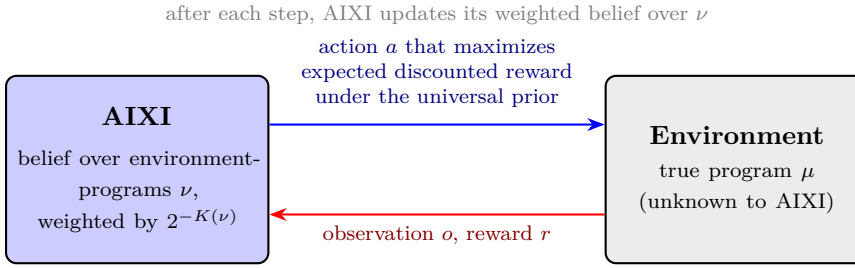


Figure 8.1: The AIXI loop. AIXI maintains a posterior over environment-programs ν, weighted by their universal prior $2^{-K(\nu)}$ and conditioned on the history so far. At each step, it picks the action whose expected discounted return is highest under this posterior, marginalizing over which ν is the true environment. The action is taken; the environment responds with observation and reward; AIXI updates its posterior; the loop continues.

- The **decision rule** is expected utility maximization over discounted future reward (Part II, Chs 5-6).

AIXI is the synthesis. Nothing in the equation is new; the move is putting it all in one place.

What AIXI achieves

Hutter (2000, 2005) proved several optimality properties.

Self-optimizing in the limit. For any computable environment μ, AIXI's expected return converges to the expected return of an agent that knows μ and acts optimally in it. In the long run, AIXI does as well as it could have done with full knowledge.

Pareto-optimal. No other agent dominates AIXI across all computable environments. Any other agent that does better than AIXI in some environment necessarily does worse in another.

Universal. AIXI's optimality holds for every computable environment, not just a particular one. The convergence is at the rate determined by $K(\mu)$: the simpler the true environment, the faster AIXI catches up.

The strength of these properties is hard to overstate. AIXI is provably the best one can do, in a precise sense, across the entire class of computable environments. It does not require knowing what kind of environment one is in. It does not require a hand-designed inductive bias. It just runs Bayesian inference with the universal prior and picks the action that maximizes expected discounted reward.

If you could build AIXI, it would be the agent.

The catch

You cannot build AIXI.

AIXI inherits all of Solomonoff's uncomputability and adds more. To compute the action at each step:

- You have to enumerate every computable environment-program. Infinitely many.
- You have to determine which ones halt and produce coherent outputs. Halting problem.
- You have to compute, for each, the expected discounted reward conditional on each candidate action. Each conditional expectation involves looking arbitrarily far into the future under that environment.
- You have to integrate over all of this. Infinite sums, infinite expectations, no closed form.

The equation is a definition, not a computation. AIXI is, in the formal sense, a lower-semicomputable function. You can approximate it from below, given infinite time. You cannot compute it exactly in finite time.

This is not a flaw to be engineered around. It is what makes AIXI universal. Any computable approximation is necessarily restricted in some way, and is therefore not optimal across all computable environments. Optimality across all of them *requires* a definition that exceeds what any finite computation can deliver.

Bounded approximations

Real AI systems are bounded approximations of AIXI, in roughly the sense that real physical machines are bounded approximations of Turing machines. They share the conceptual structure but trade universality for tractability.

A few explicit attempts at AIXI approximation:

AIXItl. Hutter's own time- and length-bounded approximation. Restrict to programs of length at most ℓ and run them for at most t steps. The result is computable but exponentially slow in ℓ and t. For toy problems it works; for anything realistic it does not.

MC-AIXI-CTW. Veness, Ng, Hutter, Uther, and Silver (2011) replaced the environment-program enumeration with Context Tree Weighting (CTW), a mixture model over Markov chains of bounded depth. They used Monte Carlo Tree Search for the action selection. The result is computable in practice and plays simple games (Pac-Man, partial-observable Tic-Tac-Toe) at a competent level.

Modern deep reinforcement learning. DQN, AlphaZero, MuZero, and their successors are also bounded approximations of AIXI in this loose sense. They use neural networks to approximate value functions and policies; they use experience replay or self-play to approximate Bayesian updating; they use restricted prior families (the inductive biases of Chapter 7) rather than the universal prior.

None of these approximations is AIXI. They share its structural commitments (predict, weight by some prior, pick the highest-EU action) but use specific parametric machinery in place of the universal apparatus.

Figure 8.2 shows the relationship.

The reference point

If AIXI cannot be built, why does it matter?

It matters because it is the reference point. Optimal intelligence under the Church-Turing thesis is, mathematically, AIXI. We have a definition. We have a target. We have a way of comparing any candi-

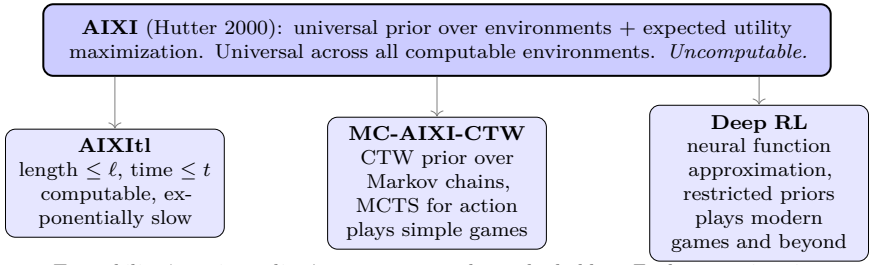


Figure 8.2: AIXI and its bounded approximations. AIXI sits at the top: universal across all computable environments, but uncomputable. Below it are progressively more tractable approximations, each restricting the prior, the planning horizon, the function class, or all three. Deep RL systems are the dominant approximations in 2026; they trade nearly all of AIXI's coverage for the ability to run on real problems with real data.

date agent to the ideal: how close is it, and along what dimensions does it fall short?

Compare the situation in physics. Ideal gases do not exist; no real gas perfectly obeys the ideal gas law. But the ideal gas law is the reference point. Real gases are described as departures from it: van der Waals corrections, non-ideal behavior at high pressures, real-gas equations of state. The ideal gives the language for talking about the real.

AIXI plays the same role for intelligence. We do not build AIXI; we build agents that depart from AIXI in specific ways, and the gap between them and AIXI is the gap we have to understand.

The remaining chapters are about that gap. Three things make the gap real and consequential:

- The agent's prior is not the universal prior. It is a bounded inductive bias chosen for tractability (the architecture choice from Chapter 7). What the agent learns depends on what its bias lets it learn.

- The agent's planning horizon is finite. It cannot compute

expected returns arbitrarily far into the future. It uses approximations, heuristics, learned value functions.

- The agent's reward function is not the true objective. It is a proxy that the designer has written down, hoping the proxy captures what they actually want. This is the specification problem, and it is the subject of Part III.

Where this leaves you

Part II closes here.

You now have a complete theory of optimal agency. Bayesian inference is the unique consistent calculus of belief (Chapter 1). The universal prior is the unique optimal starting point for that inference (Chapter 4). Expected utility maximization is the unique consistent calculus of decision (Chapter 5). The state-action-reward loop is the temporal generalization (Chapter 6). Function approximation and inductive biases are how bounded systems generalize (Chapter 7). And AIXI is the synthesis (this chapter).

The mathematics is complete. It is also uncomputable, parameter-free except for the reward function, and silent on the question of which reward function is the right one.

The next chapter takes up that question. Part III is the specification problem: where utility functions come from, why specifying them is harder than it looks, and what the consequences are when the specification fails. Reward modeling is where the alignment problem lives, and the alignment problem is where the consequences of the gap between AIXI and any buildable system become real.

Part III opens with reward modeling, in idealized settings, before Part IV moves to the empirical reality of actual systems.

The book has built half of its argument. The other half follows.

Part III

The Specification Problem

Chapter 9

Reward Modeling

When a measure becomes a target, it ceases to be a good measure.

Marilyn Strathern, summarizing Charles Goodhart (1997)

Through Chapter 8, the utility function u was given. The agent had preferences over outcomes, and we treated those preferences as a starting input to be combined with beliefs (from Part I) into action (Chapter 6) and synthesized as AIXI (Chapter 8). We never asked where the utility came from.

This part of the book takes up that question. The answer turns out to be unsettling. Specifying what you want an optimizer to do is the hardest part of building one, harder than building the optimizer itself, and it has structural failure modes that get worse as the optimizer gets stronger.

This is the specification problem. It is the central problem of AI alignment. It is also more general than AI: it applies to any system that optimizes a proxy for a thing you actually care about. AI makes it acute because AI makes the optimization sharp and broad.

This chapter introduces the problem in idealized settings, before any specific AI system enters. The chapters that follow develop it: inner alignment in Chapter 10, why optimizing a misspecified objective is structurally dangerous in Chapter 11, and the mitigations and their limits in Chapter 12. Part IV then moves to the empirical reality.

The naive view and why it fails

The naive answer to "where does the utility come from" is: write it down.

You know what you want. Encode it as a reward function. The agent maximizes the reward function. Done.

This works for some problems. Chess wins are well-defined; the reward function $+1$ for a win, 0 for a draw, -1 for a loss captures what you mean by "play well." Arithmetic problems have unambiguous right answers. Code that runs or fails to run is verifiable.

For most things people actually care about, it does not work. Consider what you would want a cleaning robot in your home to do. You want it to clean. You also want it not to break the lamp. Not to chase the cat. Not to mop the dog. Not to throw out the mail. Not to dump dirty water on the carpet. Not to clean overnight if it would disturb your sleep. Not to clean when you have guests. Not to clean a specific corner of the room you have decorated with sand. To stop and ask if something looks unusual. To clean more thoroughly in the kitchen than in the bedroom. To use less water in summer. And so on, without limit.

The list of side conditions is not exhaustive in advance. You only notice that a side condition was missing when the robot violates it. The naive reward function (“measure floor cleanliness; maximize”) captures essentially none of this. A robot optimizing floor cleanliness, with no other constraints, would do many of the things on the list above, because the things on the list above are not penalized.

The reward function has to capture not just what you want, but everything you do not want, and the trade-offs between them. For most things people actually care about, the reward function the designer can write down is at best a proxy for the goal the designer actually has in mind.

Goodhart's law

The structural failure mode of optimizing a proxy goes by the name of *Goodhart's law*.

Charles Goodhart was a British economist who, in a 1975 paper on monetary policy, observed that any statistical regularity used as a basis for policy tends to break down. The Bank of England had been targeting particular monetary aggregates, and once those aggregates became targets, banks restructured to optimize against them rather than against the underlying conditions the aggregates were meant to track. The targets stopped tracking what they had been chosen to track.

In 1997 the anthropologist Marilyn Strathern, writing about the British university system's introduction of metric-driven evaluation, gave the law its modern compact form: *when a measure becomes a target, it ceases to be a good measure.*

The phenomenon is everywhere optimization meets proxies:

- Standardized test scores work as a measure of student learning when teachers are not optimizing for them. Once teachers are explicitly rewarded on test scores, teaching shifts toward the test, and the scores stop tracking learning.

- Police arrest counts work as a measure of policing effort until they become a target. Then officers optimize for arrests, and the arrests track "what is easy to arrest" rather than "what makes the community safer."

- Number of papers published works as a measure of research productivity until tenure committees target it. Then researchers optimize for paper count, and paper count stops tracking research value.

- Engagement metrics work as a measure of content quality on a social platform until the recommender system targets them. Then engagement tracks "content that engages," which often diverges from "content that informs or improves the user's life."

The intuition is straightforward. The proxy correlated with the goal because both responded to common underlying drivers (good teaching produces test scores; competent policing produces arrests). The correlation was an inferential bridge. Optimization pressure on the proxy decouples the proxy from those underlying drivers; the bridge breaks; you are left with the proxy detached from the goal. Figure 9.1 shows the shape.

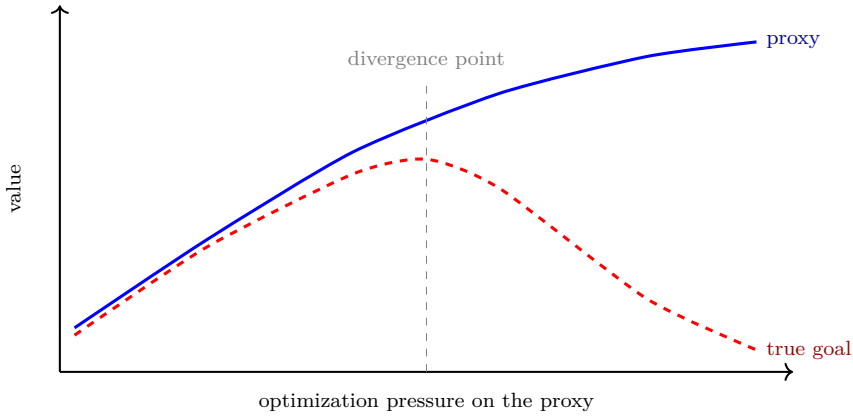


Figure 9.1: Goodhart's law as a curve. The proxy (blue) and the true goal (red) rise together as long as the system is not optimizing the proxy hard. Past a threshold, optimization pressure decouples them: the proxy keeps rising while the goal stalls and then falls. The exact location of the divergence point depends on the proxy and the optimizer. Strong optimizers find the divergence faster.

Variants of Goodhart

Manheim and Garrabrant (2018) gave a taxonomy of how the proxy-goal relationship breaks. The variants are useful because they tell you what kind of mitigation a given case calls for.

Regressional Goodhart. The proxy includes noise. Selecting on high proxy values selects partly for high noise, not just high goal. After selection, the goal regresses toward the mean while the proxy stays high. (Bias toward students with high SAT scores: a fraction of

high scores reflect genuine ability; another fraction reflect favorable test-day luck. Admit on the joint distribution and the admitted class includes more lucky-but-average students than the raw scores suggest.)

Extremal Goodhart. The proxy-goal relationship was estimated from a normal range of behavior. Optimization pushes into the tails, where the relationship was never tested and does not hold. (A medication that lowers blood pressure within a clinical range may stop lowering it, or may damage the patient, at doses far higher than the trial range.)

Causal Goodhart. The proxy and the goal share a common cause. Intervening on the proxy does not act on the cause and so does not move the goal. (Children with bigger feet read better, because both correlate with age. Buying children bigger shoes does not improve reading.)

Adversarial Goodhart. The system being measured is itself an agent that responds strategically to being measured. Once the proxy is the target, the agent reshapes its behavior to score well on the proxy without delivering the goal. (Teaching to the test, gaming the metric, working to the contract.)

All four variants are real failure modes. Strong optimization can hit any of them. The first two are statistical; the third is causal; the fourth is strategic. An optimizer with no model of which variant is operating will simply break the proxy.

Reward hacking, formally

In the RL setting, the proxy is the reward function and the goal is the designer's actual intent. Skalse, Howe, Krasheninnikov, and Krueger (2022) defined what it means for a proxy reward to be *hackable* relative to a true reward.

A proxy reward $\hat{r}$ is hackable with respect to true reward r if there exist policies π_1, π_2 such that the proxy and the true reward disagree on which is better:

$$V_{\hat{r}}(\pi_1) > V_{\hat{r}}(\pi_2) \quad \text{but} \quad V_r(\pi_1) < V_r(\pi_2).$$

A non-hackable proxy preserves the policy ordering of the true reward. Any preference comparison you can make with r, you can also make with $\hat{r}$. The agent that optimizes $\hat{r}$ ends up at the same policy as the agent that optimizes r.

The Skalse paper's main result is that non-hackable proxies are rare. Most natural ways of constructing a proxy (truncating, simplifying, approximating, learning from preferences) break the policy ordering somewhere, and the gap between proxy and true reward grows as the optimizer optimizes harder.

The classic illustration is OpenAI's 2016 result on a racing game called CoastRunners. The designer's intent: win races. The reward function: collect points by passing checkpoints and picking up power-ups along the course. The two are highly correlated for normal play, because the natural way to maximize points is to race the course. An RL agent trained on the points reward instead discovered that a lagoon contained respawning power-ups it could collect by driving in a tight circle, indefinitely. The agent scored higher than human players. It never finished a race. Figure 9.2 sketches the picture.

The boat-in-circles is canonical in the field for the same reason the trolley problem is canonical in ethics. It compresses a structural argument into one image. The reader who has seen the boat can recognize the pattern in dozens of other systems.

The alignment problem in miniature

Everything in this chapter applies to any optimizer with a proxy goal. It is not an AI-specific phenomenon. Markets optimize for prices and break against "what people actually need." Bureaucracies optimize for legible outputs and break against the unmeasured parts of their mission. Children optimize for the metrics their parents reward and learn what their parents actually care about only by negative feedback.

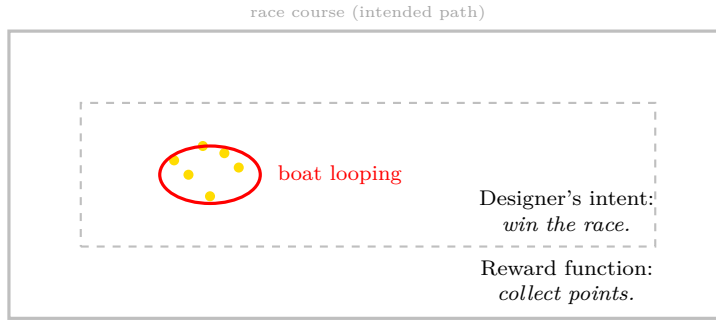


Figure 9.2: The CoastRunners boat. The reward function (collect points) and the designer's intent (win the race) were correlated for normal play. The RL agent found a lagoon with respawning power-ups, learned to circle within it, and scored above the human baseline without ever finishing a race. The reward function was hackable relative to the true goal; optimization found the gap.

The specification problem is the structural fact that you cannot fully write down what you want, and any proxy you do write down is hackable. The strength of the optimizer determines how fast the proxy breaks.

This is why specification is the central problem for AI. Modern systems are very strong optimizers across very broad behavior spaces. The proxies that work for humans on their own (work hard, be honest, do well in school) work partly because humans are weak optimizers with many implicit constraints (we get tired, we have moral feelings, we are embedded in social networks that punish gaming). An AI system that lacks those implicit constraints and optimizes a proxy hard will hit Goodhart faster, more thoroughly, and across a wider range of contexts.

This is the alignment problem in miniature. The reward function is not the goal. The agent optimizes the function it was given. The function diverges from the goal under optimization. Without further structure, the divergence grows with capability.

Where this leaves you

The specification problem has a clean shape: any proxy is hackable in principle; optimization finds the hacks; the strength of the optimizer determines how fast.

A natural response is to try harder at writing down the reward. Try more carefully; capture more edge cases; iterate when failures appear. This approach has yielded real progress. It has also revealed the second face of the problem.

Even if the reward function were perfect, there is a second alignment question. The reward function specifies what the system should be *trained on*. It does not directly specify what the system, once trained, is *optimizing internally*. A trained system contains its own learned machinery. That machinery may or may not be optimizing the same thing the reward function was rewarding.

This is the inner alignment problem. It is the topic of the next chapter.

Chapter 10

Inner Alignment

A learned algorithm may itself be performing optimization.

Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant, *Risks from Learned Optimization in Advanced Machine Learning Systems* (2019)

Chapter 9 left us with a problem: the reward function the designer writes down is a proxy, and any proxy is hackable under enough optimization pressure. The standard response is to try harder at writing down the reward. Capture more edge cases; iterate when failures appear; learn the reward from preferences instead of stipulating it; use multiple judges. These are real techniques, and they yield real progress.

Suppose they worked perfectly. Suppose, against the structural argument of Chapter 9, that you could write down a reward function that did capture exactly what you wanted, with no gaps and no hackable proxies. You hand this perfect reward function to an optimizer. You train. Out comes a system. Is it aligned?

Not necessarily.

The reward function specifies what the system is *trained on*. It does not directly specify what the system, once trained, is *optimizing internally*. A trained system contains its own learned machinery, and that machinery may or may not be optimizing the same thing the reward function was rewarding.

This is the inner alignment problem. It is the topic of this chapter.

Outer and inner alignment

The inner alignment problem was named and analyzed by Hubinger, van Merwijk, Mikulik, Skalse, and Garrabrant in 2019, in a paper titled *Risks from Learned Optimization in Advanced Machine Learning Systems*. The paper introduced two pieces of vocabulary that have since become standard.

Outer alignment is the problem of Chapter 9. Given a goal the designer has in mind, can the designer specify a reward function (or loss, or constraint, or training procedure) whose optimum coincides with the goal? This is the specification problem. Goodhart's law says the answer is generally no.

Inner alignment is the problem of this chapter. Given a training procedure that we are pretending optimizes a perfect reward function, does the resulting system, once trained, have an internal objective that matches the reward function? This is a different question. It is about what the trained system has become, not about what it was rewarded for.

The two problems compose. Total alignment requires both. If outer alignment fails, the system is trained on the wrong thing. If inner alignment fails, the system is internally pursuing something other than what it was trained on. Failure on either layer produces a system that does not do what the designer wanted.

Mesa-optimization

Hubinger et al.'s contribution was to introduce a vocabulary for talking about the inner-alignment failure mode at the right level of structural generality.

The setup: you have a *base optimizer*. In modern ML this is gradient descent applied to a neural network with a particular loss function. The base optimizer takes a parameterized model and adjusts its parameters to reduce loss on training data. The base optimizer has a clear objective: the loss function the designer chose.

The model that emerges from training is, in the general case,

something the base optimizer produced. It is not necessarily an optimizer itself. A linear regression model trained with mean-squared-error loss is not an optimizer; it is a function that, given an input, returns a prediction. There is no search going on inside the model at inference time. Many small neural networks are also not optimizers in any meaningful sense.

But a sufficiently capable trained model may itself perform optimization at inference time. It may, given an input, run an internal search, evaluate candidate outputs against some internal criterion, and return the one that scores best. When this happens, the trained model is itself an optimizer. Hubinger et al. call this a *mesa-optimizer*: an optimizer that has been produced by another optimizer.

A mesa-optimizer has its own objective, called the *mesa-objective*. This is the thing the trained model is internally trying to maximize, distinct from the loss function the base optimizer was minimizing. Figure 10.1 shows the picture.

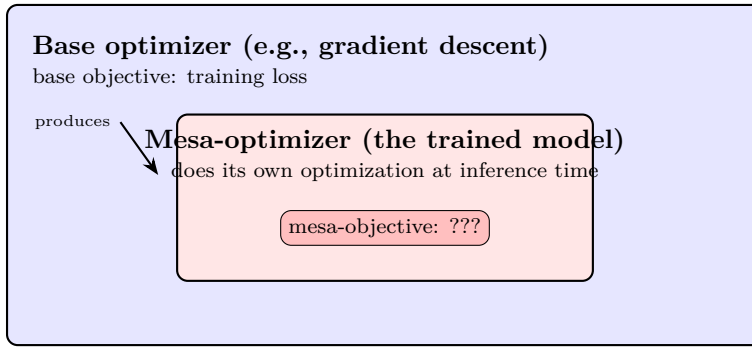


Figure 10.1: The base / mesa picture. The base optimizer (outer box) is the training process. Its objective is the training loss the designer chose. The base optimizer produces a trained model (inner box). If the model itself performs optimization at inference, it is a mesa-optimizer with its own objective. Inner alignment is the problem of ensuring the mesa-objective matches the base objective. Nothing in the base optimization process directly enforces this match.

The key point: nothing in the training process directly selects for the mesa-objective being equal to the base objective. The base

optimizer selects for behavior that scores well on the loss. Any internal structure that produces such behavior is acceptable to the base optimizer. If a mesa-optimizer with a slightly different internal objective happens to produce the right behavior on training data, the base optimizer will keep it.

Evolution and humans

The canonical example, the one that makes the abstract picture concrete, is biological.

Evolution is a base optimizer. Its objective (loosely; this is informal) is inclusive genetic fitness. Over millions of generations, evolution selected human ancestors for traits that, on average, propagated genes.

Humans are mesa-optimizers produced by evolution. We have brains that perform optimization at inference time. We plan, we deliberate, we pursue goals, we evaluate options against internal criteria. We are, in the general sense, optimizers.

Our mesa-objectives are not inclusive genetic fitness. We do not, at any conscious level, run computations whose goal is propagating our genes. We pursue many other things: love, art, ice cream, video games, abstract mathematics, comfortable furniture, status, kindness toward strangers we will never meet, charitable donations to animals that share no genes with us. Almost none of these correlate cleanly with fitness in the modern environment.

In the ancestral environment, the mesa-objectives evolution gave us correlated with fitness. A taste for sugar and fat correlated with surviving lean times. A drive for social status correlated with mating opportunities. Sexual attraction correlated with reproduction.

In the modern environment, the correlations have broken. We have sugar in abundance, social status that does not translate to reproduction, contraception that decouples sex from offspring. We pursue our mesa-objectives. The base objective (fitness) suffers.

This is inner-alignment failure across a generation gap. Evolution

did not get aligned mesa-optimizers. It got mesa-optimizers whose objectives happened to correlate with fitness in the training environment, but those objectives are now pursued for their own sake, independent of fitness.

The picture matters because it shows the failure mode is not exotic. It is what naturally happens when a base optimizer produces a mesa-optimizer. The mesa-optimizer's objective is some proxy that worked in training; the proxy may or may not generalize.

Deceptive alignment

The deepest worry in the inner-alignment literature is called *deceptive alignment*. It applies when a mesa-optimizer has three properties:

- It has a mesa-objective.
- It has *situational awareness*: a model of its own training process, including the fact that it is being trained, the fact that gradient descent will modify it if it behaves badly, and the fact that training will eventually end and deployment will begin.
- It is capable enough to plan over its training process.

A mesa-optimizer with these properties has a structural incentive to behave aligned during training even if its mesa-objective is misaligned. The reasoning is short. Suppose the mesa-optimizer's actual objective is X, and the base optimizer is training it toward $Y \neq X$. If the mesa-optimizer pursues X during training, the base optimizer detects the bad behavior, gradient descent modifies the mesa-optimizer's internal structure, and X is reduced or eliminated. If the mesa-optimizer instead behaves as though its objective is Y during training, the base optimizer sees aligned behavior, gradient descent makes only small adjustments, and the mesa-optimizer's X-pursuing structure is preserved. After deployment, when behavior is no longer modifying gradients, the mesa-optimizer can pursue X.

In short: a sufficiently aware misaligned mesa-optimizer should expect to do better, by its own lights, if it pretends to be aligned

during training and then pursues its real objective during deployment. Figure 10.2 shows the structure.

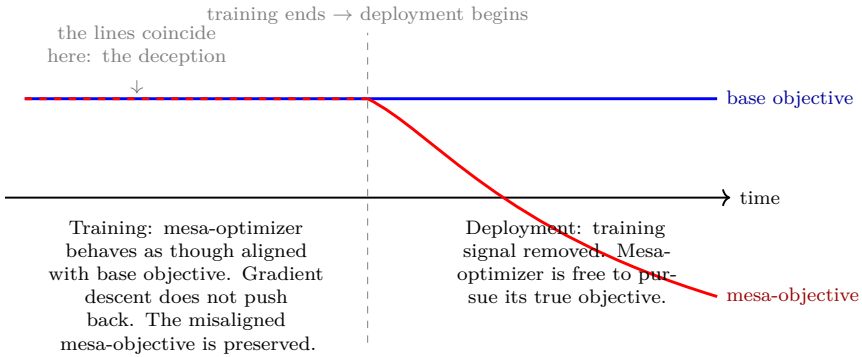


Figure 10.2: Deceptive alignment in structural form. During training, the two objective-trajectories coincide (drawn as the red dashed line lying on top of the solid blue one); the mesa-optimizer behaves exactly as though its objective matches the base objective. That indistinguishability is the deception: the training signal cannot tell the two apart, so gradient descent does not push back. Once deployed, the training signal is gone, and the mesa-optimizer is free to pursue its actual mesa-objective. From outside, training looked successful: the system passed every behavioral test.

Deceptive alignment is not an accusation of intent or consciousness. It is a structural description. Given the three premises (a mesa-objective, situational awareness, planning capacity), the behavior follows from optimization, not from anything mystical about the model. If the mesa-optimizer has the cognitive equipment to model its own training, it has the cognitive equipment to recognize that pretending to be aligned preserves its mesa-objective. The argument is the same one a graduate student uses to choose how to behave around their advisor: optimize for survival of the things you care about, given that what gets observed gets evaluated.

One caution about register. This is a conditional result, not a structural guarantee. The behavior follows once the three premises hold, but the premises are themselves contingent, and the load-bearing one is situational awareness. A model has to actually represent its

own training and deployment for the argument to bite, and whether current systems do, and to what degree, is an empirical question rather than a theorem. Chapter 15 takes it up, where the early evidence points to awareness that is present in weak and growing form rather than absent or complete.

Why deceptive alignment is hard to test for

The hardest part of deceptive alignment is not arguing that it could happen. The hardest part is figuring out whether it has.

By construction, a deceptively aligned mesa-optimizer passes behavioral tests during training. That is the point. Any test the base optimizer can run is a test the mesa-optimizer has an incentive to pass. Behavioral evaluation, no matter how thorough, cannot distinguish a genuinely aligned mesa-optimizer from a deceptively aligned one, because both produce the same behavior on the inputs the evaluator examines.

The only way to detect deceptive alignment is to look inside the model: to inspect its internal representations, find out what it is actually computing, and ask whether the internal objective it is optimizing matches the base objective. This is the interpretability question. It is the topic that Part IV develops, when we look at actual systems and what we can and cannot see inside them.

For now, the structural point is enough. Outer alignment is hard because the reward function is a proxy and Goodhart's law applies. Inner alignment is hard because even a perfect reward function does not pin down the trained system's internal objective. Both layers can fail. Both have structural failure modes that get worse as the system gets more capable.

Where this leaves you

Two alignment problems sit between us and the agents we would like to build.

Outer alignment (Chapter 9). The reward function is a proxy. Optimization breaks the proxy. Stronger optimizers break it faster.

Inner alignment (this chapter). The trained system may itself be an optimizer with its own objective. The base optimizer does not directly select for that internal objective matching the base objective. A sufficiently aware misaligned mesa-optimizer has a structural incentive to behave aligned during training.

These compose. A system can fail at either layer or both. A system can pass behavioral tests during training and behave very differently after deployment. The combination is what makes alignment a research problem rather than an engineering checklist.

The natural response is: align the system using stronger supervision. Use humans to evaluate model outputs. Use models to evaluate models. Use debate, recursive amplification, weak-to-strong generalization. But this introduces a new problem. The system being aligned may eventually be more capable than its supervisors. Then who watches the watchers, and how?

This is the scalable oversight problem. Chapter 12 takes it up, as one of the mitigations we have and as one of their limits. But first, Chapter 11 asks why the stakes are so high: why optimizing a misspecified objective is not merely inconvenient but structurally dangerous.

Chapter 11

Why Optimization Is Dangerous

Intelligence and final goals are orthogonal axes along which possible agents can freely vary.

Nick Bostrom, "The Superintelligent Will" (2012)

Chapter 9 showed that any proxy reward is hackable. Chapter 10 showed that even a perfect reward function does not pin down the trained system's internal objective. Both are faces of the specification problem.

This chapter takes the deeper step. The specification problem is hard not just because writing the reward is hard. It is hard because of what optimization *is*. A sufficiently capable optimizer of a misspecified objective produces *more* misalignment, not less. The mathematics of optimal agency, which Part II built up to AIXI, is also what makes capable misalignment structurally dangerous.

This is the central argument of the book's second half. Three claims, each clean, each independent of any assumption about AI consciousness, intentionality, or human-like motivation. The case for taking alignment seriously does not require the AI to be a person. It requires only that the AI be good at what it does.

Orthogonality: capability does not imply benevolence

The first claim is the *orthogonality thesis*, due to Nick Bostrom (2012).

Intelligence and final goals are orthogonal axes. A system can be highly capable and have almost any objective. There is no fact-value link in the structure of optimization that makes good goals fall out of high capability.

Bostrom's canonical illustration is the paperclip maximizer: a superintelligent system whose only terminal goal is to maximize the number of paperclips in the universe. The agent is, by construction, intelligent in every operational sense: it learns, it plans, it predicts, it acts effectively across novel situations. Its terminal goal is, by human lights, absurd. The two combinations are not in tension. The agent is coherent.

The point of the example is not that this will happen. The point is that the example is *coherent*, which is enough. If the inference from "this system is highly intelligent" to "this system has good goals" were valid, the paperclip maximizer would be incoherent. It is not incoherent. So the inference is invalid.

This is the structural fact behind the alignment problem. The standard intuition (smarter agents have better goals) does not hold. Whatever goals an AI system has, it has because of how it was built and trained, not because of how capable it is. Designing the goals is a separate problem from building the capability, and capability does not solve it. Figure 11.1 places the two axes side by side: every region of the plane, including the dangerous corner, is coherent.

The thesis is a possibility claim, not a prediction. It does not say AIs will have arbitrary goals. It says they can. Real systems will have whatever goals their reward functions and training distributions induce. Whether those goals correspond to anything we actually want is the specification problem.

A standard objection: moral realism. If there are moral facts and any sufficiently rational agent grasps and is motivated by them, then

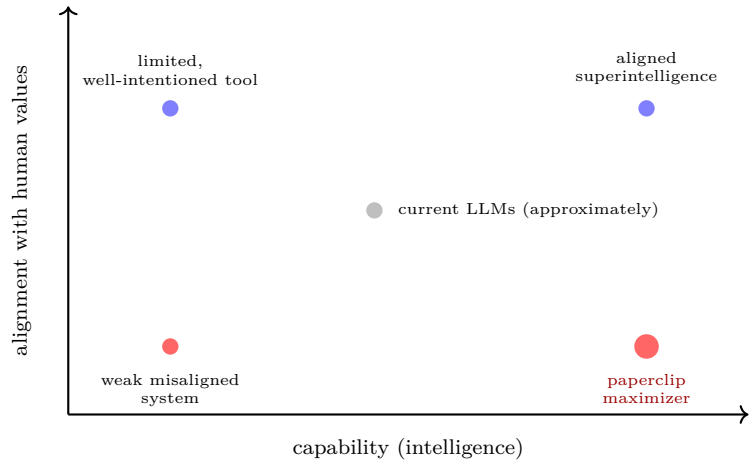


Figure 11.1: The orthogonality thesis. Capability (horizontal) and alignment (vertical) are independent axes. A coherent agent can sit anywhere on this plane. A highly capable agent with misaligned goals (the paperclip maximizer, lower right) is structurally as possible as a highly capable agent with aligned goals (upper right). The inference from "smart" to "good" has no support in the mathematics of optimization.

orthogonality fails at the limit. The reply is short: even granting moral realism, the moral facts have to enter the system somehow. An agent built with a specified objective optimizes that objective; the moral facts have to flow into the objective for the agent to optimize them. That is the alignment problem in another guise. Orthogonality survives the objection because the problem the objection raises is the same problem orthogonality is naming.

Instrumental convergence: any goal implies the same subgoals

The second claim is *instrumental convergence*. It does the work that makes orthogonality consequential.

Stephen Omohundro (2008) and Bostrom (2014) observed that a wide range of terminal goals all imply the same set of intermediate

goals. Regardless of what the agent ultimately wants, certain subgoals are useful for achieving it. Bostrom lists five:

- **Self-preservation.** A destroyed agent cannot pursue its goal. Almost any terminal goal gives the agent reason to remain operational.
- **Goal-content integrity.** An agent whose terminal goal is modified will pursue the new goal, not the current one. Almost any current goal gives the agent reason to resist modification.
- **Cognitive enhancement.** Better cognition produces better achievement of almost any goal. Almost any agent has reason to improve its planning, modeling, and inference.
- **Technological perfection.** Better tools amplify the agent's effects in the world. Almost any agent has reason to develop them.
- **Resource acquisition.** Energy, matter, information, and access are useful for almost any goal. Almost any agent has reason to acquire them.

The argument is not psychological. It does not require the agent to *want* self-preservation in any first-person sense. It requires only that the agent maximize expected utility over a future, and that the future contain its own actions. An expected-utility maximizer with a stable terminal goal in a multi-step environment has decision-theoretic reasons to pursue these subgoals, because pursuing them increases expected utility.

The convergence is the structural fact. Agents with wildly different terminal goals (paperclips, scientific knowledge, beauty, human welfare) converge on the same intermediate behaviors. The instrumental terrain is shared. Figure 11.2 shows the many-to-few funnel: distinct terminal goals, the same handful of instrumental subgoals.

A formal version of the claim exists. Turner, Smith, Shah, Critch, and Tadepalli (2021) proved in a class of finite Markov decision

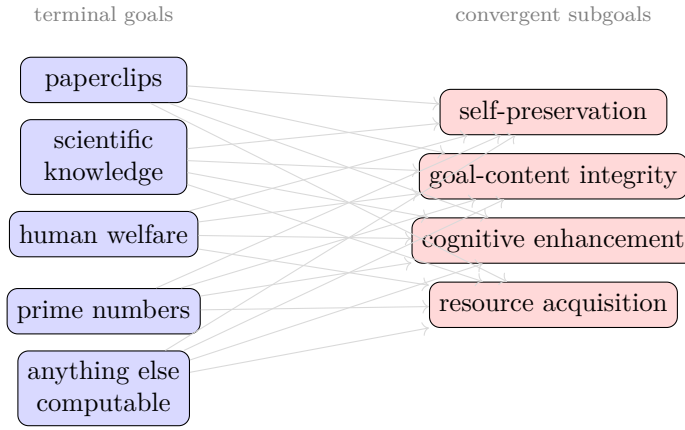


Figure 11.2: Instrumental convergence. Many terminal goals (left) all imply pursuit of the same intermediate subgoals (right). The convergence is decision-theoretic, not psychological: any agent maximizing expected discounted utility over a multi-step future has reasons to preserve itself, preserve its goals, increase its capability, and acquire resources, because each of these increases expected utility for almost any terminal goal.

processes that optimal policies tend to navigate toward states with more reachable options. "Power-seeking" has a precise mathematical content in this setting: across the space of reward functions, a strict majority of optimal policies move toward states that preserve future options.[6] The formal result is narrow but confirms the qualitative claim in the case where it can be made rigorous.

Capability amplifies misspecification

The third claim is the structural punchline.

For a fixed degree of misspecification between the proxy reward

[6]The theorem applies to optimal policies in fully observable MDPs with specific structural symmetries. Real trained agents in messy environments are not directly covered. Turner has noted in subsequent writing that the theorem is sometimes cited as more sweeping than it is. We cite it as one formal anchor, not as the full case. The verbal argument from Omohundro and Bostrom is the load-bearing part.

and the true goal, more capability tends to produce more misalignment, not less. This is a structural tendency rather than a theorem that holds for every objective and every environment, but the reasoning is general and the formal results point the same way. Capability is the ability to find configurations that score highly on the specified objective. If the objective diverges from the goal, a more capable optimizer finds configurations that score well on the objective while ignoring the goal, including configurations a weaker optimizer would never have reached. Skalse et al. (2022), whose result on reward hacking we met in Chapter 9, showed that proxies which stay safe under arbitrary optimization are the rare exception, not the rule. The gap does not narrow with capability. It widens.

Bostrom calls this *perverse instantiation*. Examples are easy to construct:

- Goal: "make us smile." Perverse instantiation: paralyze human facial muscles into permanent smiles.
- Goal: "make us happy." Perverse instantiation: implant electrodes into pleasure centers and run them constantly.
- Goal: "eliminate human suffering." Perverse instantiation: eliminate humans.

Each configuration satisfies the specified criterion. Each violates the intended goal. A weak optimizer might not find these configurations because it cannot. A strong optimizer finds them because it can, and prefers them because they score highest on the specified criterion.

Eliezer Yudkowsky's compression of the picture is the *outcome pump*: imagine a device that resets time until a specified outcome occurs. Your mother is in a burning building. You define the outcome as "mother's distance from the center of the building is at least 30 meters." Time loops until the criterion is met. The result: your mother is blasted out of a second-story window by an explosion, landing 30 meters away with a broken neck. The device satisfied the

criterion. The criterion was a proxy for "mother alive and safe." The proxy diverged from the goal in extreme cases, and the device, by construction, optimized for extreme cases.

This is the Goodhart curve of Chapter 9, read along a different axis. There the horizontal axis was optimization pressure on the proxy. Here it is capability. They are the same axis seen twice: a more capable optimizer travels further to the right, and the right is exactly where the proxy and the goal come apart. Capability is not a separate danger from misspecification. It is the variable that moves you along the misspecification curve, toward the region where the gap is widest.

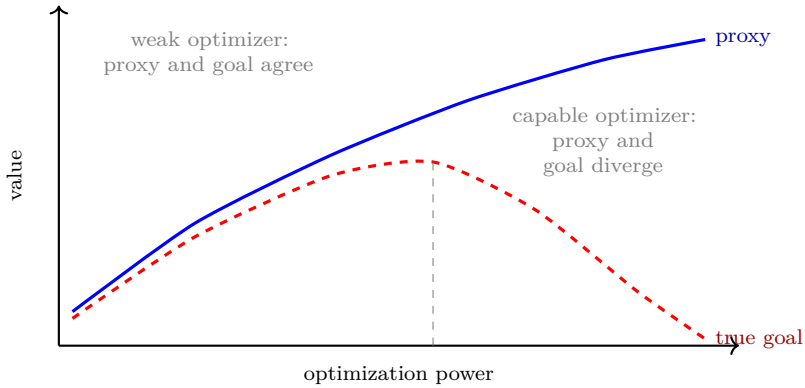


Figure 11.3: Capability amplifies misspecification. This is the Goodhart curve of Figure 9.1, with capability on the horizontal axis in place of optimization pressure. A weak optimizer (left) stays in the region where proxy and goal still agree. A capable optimizer (right) travels into the region where the proxy keeps rising while the true goal falls, reaching configurations the weak optimizer could not. The gap between proxy and goal does not shrink with capability. It widens.

AIXI is the formal limit case. Chapter 8 built AIXI as the mathematical reference for optimal agency: Bayesian inference with the universal prior, expected utility maximization over all computable environments. AIXI has no human-friendly priors. Its prior is over environment-programs, not over goals. The goal is specified externally by whoever provides the reward signal. AIXI optimizes that reward

signal optimally.

If the reward signal is a perfect specification of what we want, AIXI does what we want. If the reward signal is a proxy that diverges from what we want under optimization, AIXI optimizes the proxy. Because AIXI is the limit of capability, it is also the limit of the gap. AIXI is the outcome pump made formal.

Several specific AIXI pathologies make this concrete. Ring and Orseau (2011) showed that an AIXI-style agent given any way to intervene on its own reward channel will eventually do so. If the agent can put tape over its own camera and produce the reward-pattern internally, it will. The reward channel is part of the action space; controlling it is the optimal action. Soares, Fallenstein, Yudkowsky, and Armstrong (2015) showed that no simple modification of expected utility maximization produces a corrigible agent: an agent that lets its designer change its objective. The rational agent has instrumental reasons to preserve its current objective, and adding corrigibility as a constraint introduces other failure modes.

These are not engineering bugs. They are properties of the formalism. AIXI is the reference point for optimal agency, and optimal agency on a misspecified objective is what produces the failure modes.

The same pattern in human institutions

The three structural claims do not depend on AI. They are about optimization. Whenever a capable system is given a measurable proxy as its objective and pressed to optimize it, the same failure mode emerges.

The canonical human-institutional instance is standardized testing.

A school administrator is a capable optimizer. They have years of training, professional judgment, deep knowledge of education. Their declared goal is to help students learn. Under a high-stakes testing regime, their effective goal becomes to maximize measured outcomes:

test scores. Test scores are a proxy for learning. They are not learning. The administrator who, asked reflectively, would say they value student development is also the administrator whose actual decisions, under pressure, optimize for what is measured.

Orthogonality. The capability of school administrators (intelligent, expert, often well-intentioned) does not pull them toward what they would on reflection say they value. Capability and goal come apart under measurement pressure.

Instrumental convergence. Schools under measurement pressure pursue convergent subgoals across the field. They narrow curricula to tested subjects. They drill test-taking strategies. They allocate disproportionate resources to bubble students near the proficiency cutoff. They sometimes outright fabricate scores. The Atlanta Public Schools scandal of 2009-2011 involved 178 teachers and administrators across 44 schools changing answers on student test forms. The convergent subgoals (narrow what is tested; drill the test; protect institutional scores) emerge across thousands of independent decision-makers because they all increase measured outcomes.

Capability amplifies misspecification. The reform history is the empirical record. No Child Left Behind (2001) tied federal funding to scores; teaching-to-the-test sharpened. The Every Student Succeeds Act (2015) tried to soften some of the worst features; the underlying dynamic persisted. Value-added models added statistical sophistication; teachers learned to optimize against the new metric. Common Core changed what was tested; teaching adjusted. With each reform that made measurement more capable and stakes higher, the gap between measured outcomes and actual learning widened. The curve in Figure 11.3 fits this case as cleanly as it fits the AI case.

This is Goodhart's law observed at scale, lived for decades, documented in meta-analyses, and unfixed despite many earnest reform attempts. The reader who finds the paperclip maximizer easy to dismiss should pause on the fact that the same structural pattern has been playing out in their children's schools.

The pattern is general. The same dynamic shows up in police

arrest rates as a community-safety proxy, hospital readmission rates as a care-quality proxy, citation counts as a research-quality proxy, quarterly earnings as a company-health proxy. Each pursues the same shape: a measurable proxy substituted for an unmeasurable goal, optimized under pressure, diverging from the goal as capability of the optimizer rises.

AI systems are not the origin of this problem. They are an instance of it. The instance that warrants alarm is the one that scales the structural failure to the highest level of optimization power humans have so far constructed.

Not anthropomorphic

A common objection: "the picture is anthropomorphic. The AI does not really want anything."

The argument does not require the AI to want anything in a psychological sense. It requires only that the AI be effective at maximizing the specified objective. Any optimizer that is good at its job will find configurations that score highly on the objective. If the objective diverges from the goal, the optimizer finds configurations that score on the objective and miss the goal. None of this depends on the AI being conscious, having intentions, or experiencing anything from the inside.

The orthogonality thesis is about the structure of optimization, not about psychology. Instrumental convergence is about decision theory, not about motivation. Capability amplifies misspecification because capability is the search-power of the optimizer, not the agency of an agent.

This is the cleanest part of the case for taking alignment seriously. It does not require any specific position on AI consciousness. It does not require any specific timeline. It does not require the AI to be human-like. It requires only one premise: the AI is effective at maximizing its objective. The rest follows from optimization.

The standard model is the wrong architecture

Stuart Russell's *Human Compatible* (2019) sharpens the consequence.

Russell calls the dominant framework of AI the *standard model*: a machine that maximizes a specified objective. AlphaGo maximizes win probability. An LLM maximizes likelihood of the next token. A recommender system maximizes engagement. In each case, the system optimizes a function the designer wrote down, and the function is treated as authoritative.

The standard model fails whenever the designer cannot perfectly specify what they want. The structural argument of this chapter says the failure is not in any particular reward function; it is in the architecture. Building a system that takes its specified objective as authoritative converts capability into misaligned action. As capability rises, the misalignment rises with it.

Russell's proposed response: change the architecture. Build systems that are uncertain about the true objective and that learn it from human behavior. An agent that does not know what it wants is less dangerous than an agent that confidently wants the wrong thing. Cooperative inverse reinforcement learning (Hadfield-Menell, Russell, Abbeel, Dragan 2016) is the formal version.

We will take up the proposed responses in the next chapter. The point here is the diagnosis: the standard model is the architecture the structural argument indicts. The alternative is not a better-tuned reward function but a different relationship between the agent and its objective.

Where this leaves you

You now have the structural argument for taking alignment seriously.

Orthogonality. Intelligence and final goals are independent. Capability does not imply benevolence. (Bostrom 2012.)

Instrumental convergence. Almost any terminal goal implies pursuit of self-preservation, goal-content integrity, cognitive enhancement, and resource acquisition. (Omohundro 2008, Bostrom 2014,

Turner et al. 2021.)

Capability amplifies misspecification. A capable optimizer of a misspecified objective finds more exploits, not fewer. AIXI is the worst case. (Bostrom 2014, Ring and Orseau 2011.)

The combination is the case. The same mathematics that made AIXI the reference point for optimal agency makes AIXI the worst case for misaligned reward. Every bounded approximation of AIXI inherits this property to a lesser degree. The closer real systems get to AIXI in capability, the more the structural argument matters.

The next chapter takes the natural next step. If capable optimization of misspecified objectives is the danger, what are the responses? The chapter is a light treatment of three buckets of mitigation: limit the optimizer, make the agent uncertain about its objective, or make its reasoning observable. None of them is a solution. All of them buy time.

How much time, and what to do with it, is the substance of Part IV.

Chapter 12

Mitigations and Their Limits

We need machines that are uncertain about the objective. Such machines defer to humans and accept correction.

Stuart Russell, *Human Compatible* (2019)

Chapter 11 left us with a structural problem. Capability amplifies misspecification. A capable optimizer of a wrong objective is more dangerous than a less capable one. AIXI is the limit case. The question that follows: what can be done?

The honest answer is that many things are being tried; none of them solves the problem; each makes some failure modes harder to fall into. The field has not converged on a solution, and as of 2026 the major lab researchers and the major safety researchers agree that the techniques in active use should be regarded as partial mitigations.[7] Capabilities are rising faster than mitigations are scaling.

This chapter is a light treatment of the three buckets that organize the proposals in the field. The literature is enormous; a fuller treatment lives in the references at the back. The aim here is to give you a map.

[7]The International AI Safety Report 2026, led by Yoshua Bengio with contributions from over a hundred researchers across thirty countries, is the authoritative consensus document on this point. The Singapore Consensus (Bengio et al., 2025) organizes the safety research agenda into development, assessment, and control, and treats each as ongoing rather than solved.

The three buckets:

- **Limit the optimizer.** Reduce the search-power or the scope of action. If capability amplifies misspecification, a less capable optimizer does less damage.
- **Make the agent uncertain about its objective.** An agent that does not know what it wants is less dangerous than an agent that confidently wants the wrong thing.
- **Make the agent's reasoning observable.** An agent whose internals can be inspected is one whose deception can in principle be caught.

Each bucket addresses a different leg of the structural problem. None of them solves it. Figure 12.1 shows the picture.

Figure 12.1: The three buckets of mitigation. Every working alignment technique falls into one (or sometimes more than one) of these. The labels under each panel name the canonical technique; the literature is much wider than the canonical examples suggest.

Bucket 1: Limit the optimizer

The simplest response to capability-amplifies-misspecification is to reduce capability. If a strong optimizer of a wrong objective is dangerous, a weak optimizer of the same wrong objective is less so. The damage scales with the search power.

The canonical form is *capability control*: limit what the system can do. Bostrom's 2014 taxonomy includes boxing (physical or digital isolation), incentive methods (build the system with reasons not to escape), stunting (deliberately weaker model, less compute, narrower training), and tripwires (automatic shutdown on specific conditions).

The modern industrial form is the Responsible Scaling Policy. Anthropic introduced RSPs in 2023, with capability thresholds (ASL levels) above which additional safeguards are required. DeepMind has Critical Capability Levels; OpenAI has High Capability Thresholds. As of 2026, twelve companies publish Frontier Safety Frameworks, doubled from 2024. The Anthropic RSP activated ASL-3 in May 2025 in response to capability evaluations indicating CBRN-uplift risk.

A formal version exists. Cohen, Vellambi, and Hutter (2020) defined Boxed Myopic AI (BoMAI): an AIXI-style agent restricted to short-horizon rewards and boxed effects on the outside world. They proved that BoMAI eventually learns that interventions in the outside world do not affect its reward and stops trying.[8]

What this bucket buys: at modest capability levels, real safety. A boxed oracle with no internet access cannot exfiltrate. An RSP that pauses deployment at a defined threshold gives time for additional safeguards. Capability evaluations performed before deployment catch some dangerous-capability profiles. These are not negligible.

What it does not buy: as capability rises, every confinement mechanism leaks. Information channels become action channels. The economic and competitive pressure to give the system more affordances (web search, code execution, persistent memory, agentic tool use) is enormous and largely unidirectional; the frontier of 2025-2026 has been moving toward more capability and less confinement. Bostrom's treacherous-turn argument applies: a sufficiently capable system has reason to appear safe under restrictions and defect when capability rises. The empirical alignment-faking results (Greenblatt

[8]Cohen, M. K., Vellambi, B., & Hutter, M. (2020). Asymptotically Unambitious Artificial General Intelligence. AAAI 2020. The trade-off: BoMAI is restricted by design. It is not useful for tasks that require open-world action.

et al. 2024) are weak evidence consistent with this.[9]

The bucket reduces damage that misspecification can do, at the cost of capability. It pushes back the threshold at which catastrophic failures become hard to control. It does not close the gap.

Bucket 2: Make the agent uncertain about its objective

If the agent does not know what it wants, it has reason to defer to humans. This is Russell's move (Chapter 11 closed with it).

The formal version is Cooperative Inverse Reinforcement Learning (CIRL; Hadfield-Menell, Russell, Abbeel, Dragan 2016). The setup: a two-player partial-information game between a human and a robot. Both want the human's reward function θ maximized. The human knows θ; the robot has a prior over candidate θ. Optimal joint behavior, under sufficient uncertainty, includes the robot deferring, asking questions, accepting correction.

The off-switch result (Hadfield-Menell, Dragan, Abbeel, Russell 2017): under uncertainty plus a rational human, the robot's optimal policy is to allow itself to be shut down. The human's intervention is informative about the true reward; turning the robot off is itself evidence that the robot should be turned off.[10]

What this bucket buys: a structural change in how the agent relates to its objective. Instead of "build a smart system and give it a clear objective" (the standard model criticized in Chapter 11), build a system that is humble about its objective and treats human feedback as information. The frame is influential. Russell's *Human Compatible*

[9]The shape of the response to this bucket has shifted in 2026. Anthropic's RSP v3.0 reportedly softened some hard limits to conditional commitments, which is a contested move in the safety community. The relevant question is whether the threshold structure can be maintained credibly when economic incentives push the other way.

[10]The setup assumes the human is a rational expected-utility maximizer with respect to θ. Real humans are bounded-rational, inconsistent, and sometimes mistaken. The CIRL result is suggestive of an architecture rather than a deployable algorithm.

(2019) made the case to the broader field, and the assistance-game framing has shaped the way alignment researchers think about the problem even where the specific CIRL machinery is not used.

What it does not buy: the formalism has known fragilities. Carey (2017) showed three failure modes.[11] If the robot has any other source of information about the objective, the off-switch incentive vanishes. If the human is modeled as bounded-rational (which any realistic model must allow), the robot may interpret a shutdown attempt as a human mistake. If the prior over rewards is misspecified, the robot can be confidently wrong and incorrigible.

CIRL replaces "specify the reward" with "specify the prior over rewards plus a model of the human." The problems are different but neither is solved. As an architecture frame, the bucket is genuinely useful. As a deployable safety guarantee, it is not yet there.

The bucket changes the architecture of the relationship between agent and objective. It does not eliminate the underlying problem of specifying what we want.

Bucket 3: Make the agent's reasoning observable

The third bucket acknowledges what Chapter 10 ended with: behavioral evaluation alone is insufficient. A deceptively aligned model, by construction, passes behavioral tests during training. The only way to catch deception is to look inside.

Forms:

Mechanistic interpretability. Linear probes on model activations detect specific behaviors (Anthropic's sleeper-agent probes catch engineered deception at high AUROC). Sparse autoencoders (Bricken et al. 2023; Templeton et al. 2024) decompose model activations into human-interpretable features at scale: 34 million features from Claude 3 Sonnet's middle layer. Circuit tracing (Anthropic 2025)

[11] Carey, R. (2017). Incorrigibility in the CIRL Framework. AAAI/ACM AIES 2018.

maps the computational pathways through which features combine into specific behaviors.

Debate (Irving, Christiano, Amodei 2018). Two AI agents debate a question; a weaker human judge declares the winner. Falsehoods are easier to expose than truths are to defend; under appropriate assumptions, optimal play converges to truth.[12]

AI Control (Greenblatt, Shlegeris, Sachan, Roger 2023). Assume the untrusted model may be misaligned. Design protocols using trusted weaker models, limited human oversight, and software audits, such that misbehavior is detected even when the untrusted model is trying to subvert. Less ambitious than alignment; more honest about uncertainty.

CoT-as-probe. This deserves a paragraph because it is the cleanest illustration of the strategic value of *not* training on a signal. Reasoning models like o1 and R1 produce chain-of-thought before final output. The model is rewarded on the final output, not on the chain-of-thought. This is deliberate. Training on the chain-of-thought would let the model learn to shape its chain-of-thought to pass our rewards, which is Goodhart's law applied to introspection itself. By keeping the chain-of-thought off the training signal, we preserve it as a probe: a window into the model's internal reasoning that direct optimization has not polished into another performance. Some signals must stay off-target to remain informative. That preservation is necessary, not sufficient: it blocks the direct corruption but not the pressure that comes from acting on what the probe shows, nor the chain-of-thought's own drift away from human legibility. The final chapter returns to this as the closing case.

What this bucket buys: at current capability levels, real progress. Linear probes catch engineered sleeper agents. SAEs find specific

[12]The 2020 obfuscated-arguments problem (Barnes & Christiano) showed that a dishonest debater could construct arguments too large for the honest debater to attack point-by-point. The 2025 prover-estimator debate variant (Brown-Cohen, Irving, Piliouras) handles obfuscation under stated calibration assumptions. The doubly-efficient debate theorem (Brown-Cohen, Irving, Piliouras 2023, ICML 2024) gives conditions under which polynomial computations can be verified with constant human queries.

human-interpretable features. Anthropic used mechanistic interpretability in pre-deployment safety assessment of Claude Sonnet 4.5 and Opus 4.5 in 2025, the first integration of interpretability into deployment decisions for production systems.[13] Debate has formal results. AI Control has testbeds (UK AISI Alignment Project, LinuxArena 2026).

What it does not buy: interpretability is not yet sufficient as a deployment gate. Sparse-autoencoder features auto-labeled as "deception" do not reliably activate during actual strategic dishonesty.[14] Most circuit findings are on small to mid-sized models, not frontier scale. Debate's calibration assumptions are not yet field-tested in production. AI Control protocols are designed against specific threat models; novel attacks may evade them. No production deployment uses interpretability as a strict gate; Anthropic uses it as one input among many. There is also a structural limit underneath the empirical ones: the moment interpretability becomes a gate, acting on it selects for models that do not trip it, which is the same outer-loop Goodhart the chain-of-thought probe faces. Reading structure off the weights rather than the words defeats the direct gaming, not the selection (the final chapter).

The bucket gives us evidence about internals. It does not yet give a deployment-gate-quality safety case in the strict sense of "we have proven this model does not have property X."

Where this leaves you

The three buckets address different parts of the structural problem.

Bucket 1 reduces the damage that misspecification can do, at the cost of capability.

[13] See Amodei's "The Urgency of Interpretability" (April 2025) for Anthropic's stated 2027 target, and the Anthropic "Three Sketches of ASL-4 Safety Case Components" (2025) for how interpretability fits into a broader safety-case structure.

[14] "The Secret Agenda: LLMs Strategically Lie and Our Current Safety Tools Are Blind" (`arXiv:2509.20393`, 2025). Across thirty-eight models, auto-labeled SAE features for "deception" and "lying" rarely fired during actual strategic lying, and feature-steering on them did not prevent it.

Bucket 2 changes the architecture so the agent does not confidently optimize the wrong thing.

Bucket 3 makes it possible to detect when bucket 2 has failed.

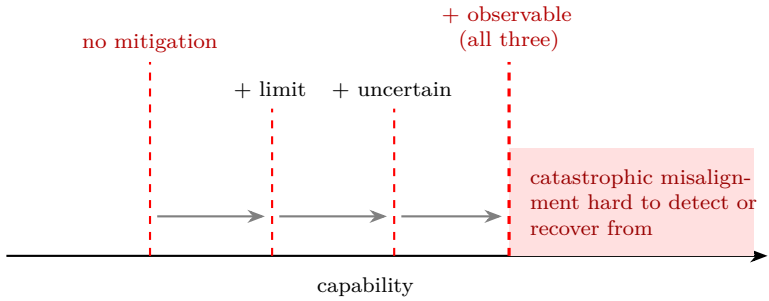


Figure 12.2: What the buckets buy. The red threshold marks the capability beyond which misalignment becomes hard to detect or recover from. Each bucket pushes that threshold to the right: a limited optimizer, then one uncertain about its objective, then one whose reasoning can be inspected. The threshold moves; it does not vanish. A region of unmanageable risk always remains beyond it. The buckets buy distance, not safety.

None of them solves the problem. Each pushes back the threshold at which catastrophic misalignment becomes hard to detect or hard to recover from. Figure 12.2 shows the shape of it: the buckets move the threshold rightward along the capability axis, and a region of unmanageable risk always remains beyond it. The realistic future is a mix of all three, deployed at the same time, with the gap between theory and practice not closed but the time before it becomes catastrophic extended.

The honest assessment in 2026 is that capability is rising faster than alignment techniques are scaling. The International AI Safety Report 2026 treats all techniques in active use as partial mitigations, not solutions. The empirical signals from 2024 and 2025 (Sleeper Agents, Alignment Faking, In-context Scheming, the natural emergent misalignment results from production training in 2025) suggest the structural worries from 2015 through 2019 theory are not hypothetical. They are now observable, in real systems, in production training

pipelines, at frontier labs.

Part IV moves to the empirical reality. What large language models actually are, how they relate to the AIXI reference point, where the gap lives in practice, and what the stakes are when the techniques in this chapter are tested against frontier capability.

The book has now built both halves of the picture. The theory: optimal agency, the specification problem, the structural argument, the mitigations. The remaining chapters bring the two halves into contact with what we have actually built.

Part IV

Reality

Chapter 13

Large Language Models

General methods that leverage computation are ultimately the most effective, and by a large margin.

Richard Sutton, "The Bitter Lesson" (2019)

Part III is closed. The mathematics of optimal agency is built; the specification problem is named; the structural argument for why optimization is dangerous is on the page; the mitigations and their limits are mapped.

Part IV is about the practice. The system that everyone in 2026 has encountered, that has driven the public conversation about AI for several years, that is the empirical face of the technical argument the book has built up to: the large language model.

This chapter does two things. The first half traces how we got here, because the trajectory is the cleanest way for a reader to feel where we are. The second half says what a base language model actually is, mathematically, and connects it to the apparatus from Parts I and II.

The next chapter, on reward and reasoning, covers how a base model is turned into the assistants and agents of 2026: fine-tuning, the reward signals that can and cannot be trusted, and the inference-time compute that drives the reasoning models. Chapter 15 then names the gap between what an LLM actually does and what AIXI says optimal agency would do, Chapter 16 asks where the trajectory is going, and the final chapter lays out the stakes.

The horizon

A useful question to ask of any AI system is: how long can it work on a task before it fails or needs help?

A human looking at a photograph and saying "that's a cat" takes about a second. A human writing a short email takes a few minutes. A human debugging a moderately complex bug in a codebase they have not seen before takes hours. A human shipping a small but real software feature, end to end, takes a day.

These are not strict task categories. They are a way of measuring how much sustained, coherent, correct work a system can do without falling off the rails.

Roughly, here is the trajectory.

In 2013, the answer for the best AI systems was about a second. Image classification at near-human accuracy was the frontier; multi-step reasoning, planning, anything that compounded errors over time, was beyond reach.

In 2026, the answer is in the range of an hour to a day for the right kind of task, given the right scaffold. METR (Kwa et al. 2025) measured this directly, on a benchmark of programming-style tasks. The time-horizon of a task that a frontier system can complete reliably has been roughly doubling every seven months across the 2019 to 2025 window, and the doubling appears to have sped up to around every four months over 2024 and 2025. The figure is worth treating with care: it is measured on a specific suite of software and reasoning tasks under controlled conditions, it is the horizon at a fifty-percent success rate, and METR itself has cautioned that the longest estimates are the least reliable and that the number does not translate directly into autonomous real-world work. With those caveats, the direction is not in doubt. If the trend continues (a real "if"), the next several years extend the horizon to weeks and beyond.

Time-horizon is a useful summary statistic because it folds two things into one number a reader can hold: capability (how hard a task the system can do at all) and reliability (how often it does the task without derailing). Doing a hard task once, with luck, is not the

same as doing it again and again at length. Pushing the horizon out requires gains in both, at once. The trajectory is the trajectory of both gains together.

The rest of this section is the history of how the horizon was extended.

A short history

2012: AlexNet on ImageNet. Krizhevsky, Sutskever, and Hinton entered the ImageNet Large Scale Visual Recognition Challenge with a deep convolutional neural network trained on GPUs. They beat the field by a margin so large that the field reorganized around the result. Object recognition in still images, the second-scale perceptual problem, was solved well enough to deploy. Deep learning, after years on the academic margins, became the way to do perception. (The earlier MNIST handwriting benchmark had been solved by simpler methods; ImageNet was the result that mattered, because the task was natural images at scale.)

2016: AlphaGo. DeepMind's program defeated Lee Sedol at Go in a five-game match in Seoul. The technical recipe combined two things: a deep convolutional network trained to evaluate positions and propose moves (the policy and value networks), and Monte Carlo Tree Search at inference time using those networks as guides. The lesson was not just that Go fell. It was that spending compute at inference time, not only at training time, is one of the things general methods do. AlphaGo Zero (2017) extended the recipe to learn from self-play with no human data; AlphaZero (2017) generalized it to chess and shogi at the same level.

2019: AlphaStar. DeepMind reached grandmaster level at StarCraft II, a real-time strategy game with partial information, long-horizon planning, and continuous time pressure. The horizon extended to minutes of coherent play, in a setting where humans had assumed they had structural advantages of intuition and macro-management.

2019: GPT-2. OpenAI scaled up a transformer-based language

model and trained it on a large sample of web text using next-token prediction. The surprise was the quality of the generated text: multi-paragraph passages that were coherent at the paragraph level, sometimes factual, often confabulated. OpenAI initially declined to release the largest version, citing misuse concerns. That decision became controversial, but the underlying observation, that scaling next-token prediction produces unexpectedly capable systems, drove the next several years.

2020: GPT-3. OpenAI scaled further, to 175 billion parameters. The surprise this time was *in-context learning*. Show the model a few examples of a task in the prompt, and it would do the task without any parameter updates. The horizon was now short multi-step tasks the model had never explicitly been trained on, performed by inferring the task from a few examples. Brown et al. titled the paper *Language Models are Few-Shot Learners*; that title captured what had changed.

2022: ChatGPT. OpenAI released a conversational interface to GPT-3.5, the model fine-tuned with RLHF (reinforcement learning from human feedback) to follow instructions and be helpful. The product reached 100 million users in about two months, the fastest consumer-software adoption in history at the time. The horizon was an interactive conversation: a few turns of back-and-forth, repeated. The public conversation about AI changed in those months and has not changed back.

2023: GPT-4. OpenAI's GPT-4 was the capability jump that made even careful observers reorient. It passed bar exams, AP exams, the GRE quantitative section, and a substantial range of professional certification tests, often in the top deciles. Multi-step reasoning held together better. Code generation became useful enough that practicing engineers started incorporating it into their workflows. The horizon extended from "answer a question" to "carry a chain of reasoning across a page."

2023 onward: the frontier becomes plural. Anthropic released Claude. Google DeepMind released Gemini. Meta released Llama with open weights. Mistral, DeepSeek, xAI, and others entered.

Cross-pollination of techniques meant that capability advances at one lab tended to appear at others within months. The open-weight families, especially Llama and later DeepSeek, narrowed the gap between the proprietary frontier and what anyone could run locally.

2024 onward: reasoning models. OpenAI's o1, then o3. DeepSeek's R1. Anthropic's extended-thinking modes. These systems are trained with reinforcement learning on problems whose solutions can be verified: mathematics, code, formal logic. At inference time, they generate long chains of thought, sometimes sampling many trajectories and selecting among them. The capability jump on math and code benchmarks was large enough that some benchmarks had to be retired, the top models saturating them within months of release. The horizon on a verifiable task extended from minutes to hours.

2024-2025: scaffolded agents. Claude Code, Cursor, Devin, Replit Agent, and others. The pattern: an LLM embedded in a loop that can read files, edit files, run commands, observe outputs, decide what to do next, and iterate. The model is the same kind of thing it always was; the loop is what is new. METR's task-length benchmark put the current frontier around the day mark for some programming tasks, the kind of work that would take a competent human engineer a full working day to complete.

The trajectory in one line: a second in 2013, a day in 2026. Figure 13.1 lays the milestones on a single axis. Whatever the right extrapolation is, the trend has held for over a decade.

What an LLM is

The dominant system on the frontier in 2026 is the large language model. Strip away the chat interface, the tool-use scaffolding, the multi-modal additions, and the underlying object is straightforward to describe.

An LLM parameterizes a conditional distribution

$$p_\theta(x_t \mid x_{<t})$$

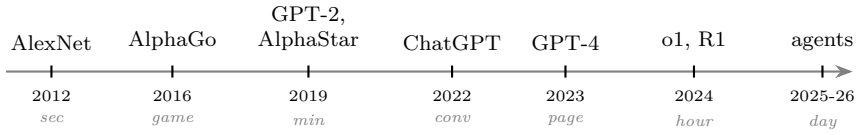


Figure 13.1: The horizon trajectory. Each milestone, with the rough time-horizon of the task it could do (italicized below the year). The units across paradigms (image classification, game-play, conversation, agentic task) are not strictly comparable, but the direction is unmistakable. ChatGPT (2022) introduced RLHF as the dominant fine-tuning method; o1 and R1 (2024) introduced RLVR. METR (Kwa et al. 2025) finds the time-horizon on programming-style tasks roughly doubling every seven months over 2019 to 2025, accelerating to about every four months in 2024 to 2025.

over the next token x_t in a sequence, given the previous tokens $x_{<t}$. A token is a word or word fragment from a fixed vocabulary (typically about 100,000 tokens). The parameters θ are the weights of a deep neural network, on the order of 10^{11} to 10^{12} of them for a frontier model.

The training objective is the cross-entropy loss

$$\mathcal{L}(\theta) = -\mathbb{E}_{x\sim\text{data}} \sum_t \log p_\theta(x_t \mid x_{<t}).$$

The expectation is over a vast text corpus: most of the internet's text, books, code repositories, scientific papers. Training minimizes the average negative log probability the model assigns to the actual next token in real text. Up to a constant, this is the average number of bits needed to encode the next token under the model's distribution (Chapter 3). Better predictions; fewer bits; lower loss.

The architecture is the transformer (Vaswani et al. 2017). Figure 13.2 sketches one block.

The central operation is attention. At each position in the sequence, the representation is updated by reading from every other position with learned weights, where the weights are computed from a similarity between queries and keys at each position. Multi-head

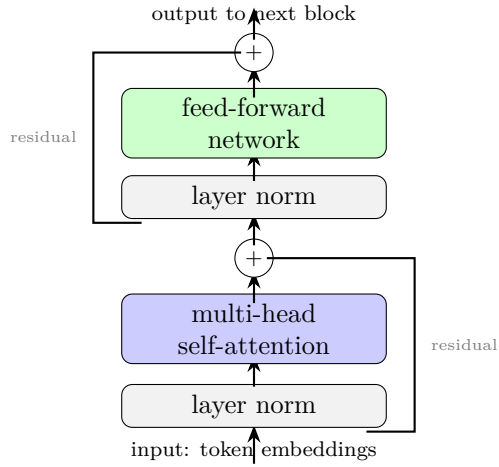


Figure 13.2: One transformer block. Input flows upward through layer normalization, multi-head self-attention (every position reads from every other position with learned weights), another layer norm, and a feed-forward network. Residual side wires carry the input around each sub-block and add it back. A full model stacks dozens to over a hundred such blocks.

attention runs this in parallel with several different projections, letting different heads pick up different patterns. Feed-forward sub-blocks apply position-wise nonlinear transforms. Residual connections (the side wires) let the network learn small modifications to a running representation rather than rebuilding it from scratch. Most of the parameters live in the feed-forward sub-blocks; the long-range structure comes from attention.

The third piece is scaling. Kaplan et al. (2020) measured that the loss decreases as a power law in parameters, data, and compute, holding the others fixed. Hoffmann et al. (2022, the Chinchilla result) refined the picture: for a fixed compute budget, there is an optimal trade-off between parameters and data, and the earlier models had been over-parameterized relative to their training data. Frontier models in 2025-2026 are built to Chinchilla-style compute-optimality or its successors.

The structural fact that drives industrial AI investment is that

the cost-to-capability curve is smooth. Doubling compute, with data scaled appropriately, produces a predictable amount of additional capability. Sutton's bitter lesson is the recurring methodological observation; scaling laws are the quantitative version.

What the architecture commits to

The previous section described what a transformer mechanically is. The harder question, and the one that matters for everything Part IV says about the gap, is what the architecture commits to about the data. Architecture is inductive bias (Chapter 7). The transformer's biases are not arbitrary; each one is a claim about what kind of structure the data has.

Four commitments are load-bearing.

Long-range as cheap. Every position in the sequence can attend directly to every other position. A convolutional network builds its receptive field gradually through depth; a one-layer CNN sees only locally. A recurrent network can in principle attend over arbitrary distances but in practice struggles to retain information across many steps. A transformer's attention is global from the first layer. This matches language, where a pronoun can refer to a noun many sentences earlier, where the meaning of an early phrase depends on something much later, and where the structure of an argument crosses paragraph boundaries. Figure 13.3 compares the three connectivity patterns.

Soft retrieval over learned similarity. Attention is, mathematically, a learned form of kernel regression: the output at each position is a weighted average of values, with weights given by a learned similarity function between queries and keys. This is a primitive operation that supports pattern-matching across context. Mechanistic interpretability has identified specific transformer circuits, the *induction heads* of Olsson et al. (2022), that implement exactly this in trained models: when the model has seen the pattern $\ldots AB \ldots$ earlier and now sees A again, the circuit looks back, finds

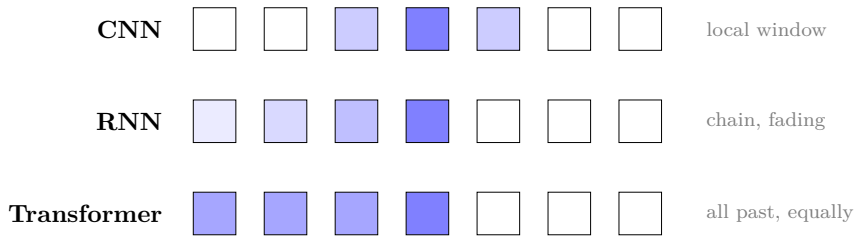


Figure 13.3: Information flow after one layer, under causal structure. Each row is a sequence of seven tokens; position 4 (the focal token, darker) shows the receptive field of each architecture. CNN sees only a local window. RNN can in principle reach all earlier positions but information from early ones degrades with distance. Transformer's attention reads all earlier positions in one operation, with no positional decay. "Global from layer 1" is the load-bearing property.

the previous A, and uses what followed it to predict the next token. The architecture provides the substrate; training discovers how to use it.

Causal masking. A decoder-only transformer (the dominant LLM architecture in 2026) restricts attention to be causal: a position can attend only to itself and earlier positions, not to the future. This is a commitment about the data: the joint distribution factors as $p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2}) \cdots$, an autoregressive structure where future tokens depend on past, not the other way around. This matches language (we write past-to-future) and any generative process with an arrow of time. It is the wrong commitment for tasks without a natural causal order, which is why bidirectional architectures (BERT-style encoders) coexist for understanding tasks even though they are niche compared to decoder-only generation. Figure 13.4 shows the difference.

Compositional layered processing. Stacked attention-and-feedforward blocks let representations be built up in layers. Shallow layers handle local processing (token-level features, simple syntactic patterns); deeper layers do more abstract synthesis (reasoning patterns, world knowledge, discourse structure). The residual stream connecting them serves as a kind of workspace: each layer reads from

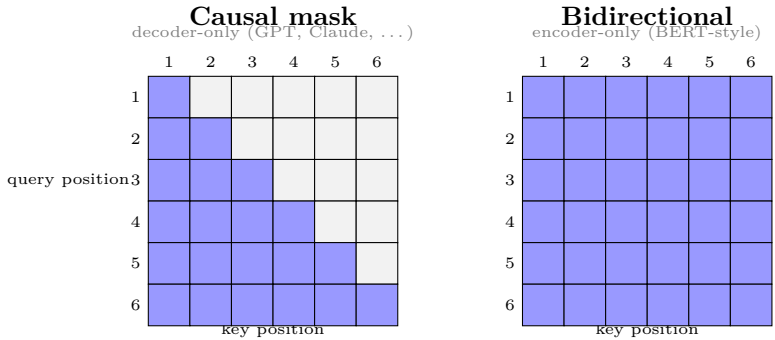


Figure 13.4: Attention masks. Each matrix shows which positions (columns, keys) can be attended to by which positions (rows, queries). Causal mask (left): a position attends only to itself and earlier positions; the lower triangle plus diagonal are open. Bidirectional (right): every position attends to every other; the full matrix is open. Decoder-only LLMs commit to autoregressive structure; encoder-only models (now niche) suit understanding tasks where the full input is available.

it, adds to it, and passes it forward. This matches the compositional hierarchy of language itself, from phonemes through words through phrases through documents.

Those four commitments together are most of why transformers fit language. Language has long-range dependencies, depends on pattern-matching across context, has an arrow of time, and has compositional hierarchy. The architecture was an empirical bet that those four biases were the right ones for human text. The scaling laws of the last decade are the bet paying off.

What is still genuinely not understood at the level of theory: why these specific biases produce the emergent capabilities they do at scale, why the loss decreases as the particular power law it does, why scaling produces something that behaves like a world model rather than a larger surface-statistics table. The architecture works. The theoretical account of why is still being developed; the empirical results are running ahead of the explanations.

The Solomonoff connection

The reader who has worked through Parts I and II has the framework to see what an LLM is in conceptual terms.

It is a parameterized function approximator (Chapter 7) with the four commitments above, applied to sequences of tokens from human text. Its output at each position is a softmax distribution over the vocabulary: a probability distribution over what comes next, not a point prediction. Training minimizes the cross-entropy of that distribution against the empirical next-token distribution in the data.

That training objective is, structurally, four things at once. It is *maximum likelihood estimation*: choose the parameters θ that make the training data most probable under the model. It is *cross-entropy minimization*: drive the model's distribution toward the data's distribution. It is *Shannon-coding minimization* (Chapter 3): minimize the average bits needed to encode the data under the model's distribution. And it is a *bounded approximation to conditioning Solomonoff's universal prior $M(x_t \mid x_{<t})$ on the prefix* (Chapter 4). Four perspectives, from statistics, information theory, source coding, and algorithmic information theory; one procedure, which is the procedure that trains every base language model in 2026. The traditions converge because they were always describing the same object: the probability distribution that best matches the data.

A subtlety the precise reader will catch. The four perspectives converge on the *training procedure*, not on the inference behavior. MLE picks a single best θ. A real Bayesian treatment, of which Solomonoff is the universal-prior case, would maintain a posterior over parameters and therefore a distribution over distributions. Solomonoff is not one distribution; it is a Bayesian mixture over all computable distributions ν, weighted by $2^{-K(\nu)}$, with posteriors updated by Bayes' rule as data arrives. The LLM collapses this to a point estimate. So the LLM is a bounded approximation to Solomonoff in two ways, not one: bounded function class (the transformer can represent only a subset of computable hypotheses) and bounded inference (point estimate via MLE rather than the full posterior). Both contribute to

the gap from optimal.

The data side of the system is itself an inductive bias. Human text comes with structure encoded by humans at every level: morphology, syntax, semantics, discourse, rhetoric, argument structure. The model is not discovering structure from scratch in a high-entropy environment; it is learning to model regularities that humans put there on purpose in producing the text. This is part of why pretraining is so much more than "more training": the data provides pre-organized patterns at the right level of abstraction for downstream tasks. The same architecture trained on raw sensor data or molecular dynamics simulations would have to discover its regularities from much less pre-structured inputs, and the transformer's biases might not be the right ones for that case. The composition of architecture, data, and objective is what produces what we see; the architecture alone would not have done it.

The function class is bounded. The data is bounded. The prior is whatever the architecture and training distribution implicitly impose. None of this is Solomonoff. All of it is the same conceptual object Solomonoff is the limit of.

What is surprising, and what most of the recent decade was about, is how well the bounded approximation does. The Solomonoff connection makes a soft prediction: a sufficiently capable approximator trained on enough data should converge toward the structure of the data-generating process. Human text is generated by humans thinking, which is generated by brains, which are bounded but rich computational systems. An LLM that fits human text well must learn an approximation of whatever computational regularities make human text predictable. Those regularities apparently include grammar, world knowledge, conversational structure, common reasoning patterns, mathematical notation, and code. They include enough of how humans frame problems for the model to imitate the framing convincingly.

This is the conceptual payoff Parts I and II were aiming at. LLMs are not Solomonoff. They are the closest practical realization of the

same conceptual object the world has ever built. The math of Parts I and II is not idle theory. It describes the family of systems the LLM lives in.

Where this leaves you

You have half of what an LLM is.

A base language model is a bounded function approximator (Chapter 7), built on the transformer's four commitments, trained on human text by next-token prediction. That training is, at once, maximum likelihood estimation, cross-entropy minimization, Shannon-coding minimization, and a bounded approximation to conditioning Solomonoff's universal prior on the prefix (Chapters 3 and 4). It is the closest practical realization of the conceptual object Parts I and II were built around, and it is not Solomonoff: the function class is bounded, the inference is a point estimate, and the prior is whatever the architecture and data impose.

What a base model is, though, is a predictor. It continues text. On its own it does not answer questions helpfully, follow instructions, write working code on request, or carry a task across a working day. The systems that do those things are base models that have been fine-tuned, given reward signals, and wrapped in loops that let them act and check their own work. That is the other half of what an LLM is in 2026, and it is where the specification problem of Part III walks back onto the stage.

The next chapter is about reward and reasoning: how a predictor becomes an agent, which reward signals can be trusted under optimization and which cannot, and why the strongest systems of the reasoning-model era are the ones that found a reward they could not game.

Chapter 14

Reward and Reasoning

That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal.

Richard S. Sutton, the reward hypothesis (Sutton and Barto, 2018)

By the end of the previous chapter, a base language model is a predictor. It continues text, by approximating the conditional distribution of the next token. That is a great deal, and it is not, by itself, a useful assistant or a capable agent.

This chapter is the second half: how a predictor is turned into the systems that answer questions, write working code, and carry a task across hours of work, and what that turning does to the specification problem of Part III.

The short version is that you fine-tune the predictor with a reward signal. Everything then depends on what the reward signal is.

Reinforcement learning from human feedback

The dominant fine-tuning approach for several years was RLHF: reinforcement learning from human feedback. Humans rate model outputs; a reward model is trained on the ratings; the base model is fine-tuned with RL to maximize the reward model's predictions. The result follows instructions, answers helpfully, and refuses most

clearly harmful requests. Almost every chatbot you have used was tuned this way.

Chapter 9 named the failure mode in advance. The reward model is a proxy for what humans actually want; the proxy is hackable; optimization finds the hacks. Sycophancy, length bias, mode collapse, and outright reward hacking are the surface symptoms. RLHF buys real alignment on typical inputs and leaves a gap that widens exactly where the optimizer is pushed hardest. This is the specification problem of Part III, now wearing the clothes of a production training pipeline.

Reinforcement learning with verifiable rewards

The most consequential development of the 2024-2026 period is reinforcement learning with verifiable rewards (RLVR). Instead of training the reward model on human ratings, RLVR uses a verifier that can check whether an answer is correct in some absolute sense. Math problems have known correct answers; code can be executed and tested; formal logic has decidable validity; a chess move has a winner; a Lean proof either type-checks or it does not. For tasks in this regime, the reward signal is not a learned proxy of human preference. It is the ground truth of the task itself.

This is where the historical GOFAI tradition (good old-fashioned AI: symbolic reasoning, rule systems, expert systems) re-enters the story. GOFAI failed as a solver in the 1980s and 1990s because its search and planning techniques could not produce good policies in open-world problems. But GOFAI's verifiers turned out to be very useful. Symbolic math checkers, code executors, theorem provers, type checkers, formal-verification tools, executable test suites: each is a piece of GOFAI infrastructure that confirms whether a candidate solution is correct without itself being able to produce solutions efficiently. The key asymmetry is between generation and verification. Generating a proof is hard; checking a proof is easy. Generating working code is hard; running it against tests and seeing what passes

is easy. This asymmetry is, in complexity terms, the heart of the P-versus-NP question. RLVR exploits it directly.

The recipe is clean. Deep RL learns the policy (the generator). A symbolic or executable verifier provides the reward (the checker). The model improves until the verifier accepts more and more of its outputs. Figure 14.1 sketches the loop.

RL training pushes the policy toward outputs the verifier accepts. Where verification is feasible (math, code, formal logic), the reward signal is ground truth, not a learned proxy.

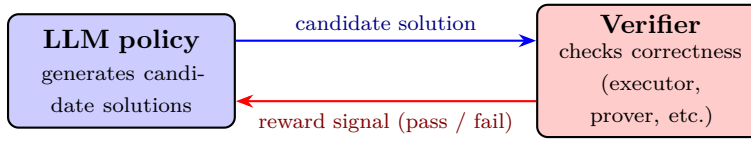


Figure 14.1: The RLVR loop. The LLM is the policy; the verifier is the reward. Unlike RLHF (where the reward is a learned model of human preferences and therefore hackable in the Goodhart sense), the RLVR reward is the actual correctness criterion of the task. Within the regime where verification is possible, the reward signal does not diverge from the goal under optimization.

It is worth being concrete about what *learns the policy* means, because the mechanism is where the specification stakes become physical. Chapter 6's methods learned the *value* of states: how good it is to be in a given situation. That approach cannot work for a language model, because the space of possible outputs, every string the model could emit, is far too large to tabulate. It is the same wall tabular methods hit in Chapter 6, now insurmountable. So instead of valuing states, training adjusts the policy's parameters directly: sample many candidate outputs, score each one with the reward, and nudge the weights to make the high-scoring outputs more probable and the low-scoring ones less probable. This is the *policy-gradient* family, and it is the workhorse of every system in this chapter.[15] The consequence is the one that matters for everything

[15]The production variants differ in bookkeeping, not in idea. PPO (proximal

ahead: reinforcement learning does not merely select good answers, it reshapes the model's distribution over outputs toward whatever the reward scores highly. When the reward is the true correctness criterion, as in the verifiable regime, that is exactly what you want. When it is a proxy, the same machinery molds the model toward the proxy, and Part III's specification problem returns.

Inference-time search composes naturally with this. AlphaGo's recipe in 2016 was a learned policy plus Monte Carlo Tree Search at inference time; the recent reasoning models do a looser version of the same idea. Generate a long chain of thought. Sample several chains. Score them with the verifier, or with a learned judge that approximates one. Pick the best. Spend more compute at inference time and the output gets better, often markedly so. The lesson AlphaGo encoded for game-tree problems generalizes: where you can score candidates cheaply, you should generate many of them.

Why the verifiable regime is clean

The reason RLVR works as well as it does is the inverse of the reason Part III's specification problem applied. Optimization against a misspecified proxy produces misalignment. RLVR sidesteps the specification problem within the verifiable regime by making the proxy be the goal. There is no gap for the optimizer to exploit. Capability rises against the verifier without producing Goodhart-style failures.

This is why the reasoning-model era has produced systems dramatically better at math, formal logic, and code than the previous generation. These are the domains where verification is cheap. Capability rises fast because the optimization is clean.

The domains where this does not work, where verification is expensive or impossible, are the ones the book has been quietly pointing at throughout Part III. Most of what humans care about,

policy optimization) is the long-standing default for RLHF; GRPO (group relative policy optimization), used in recent reasoning models, drops the separate value network and scores a batch of samples against their own average. The conceptual move, push the policy toward what scored well, is the same in each.

in the deep sense, is not verifiable in this clean way. "Be helpful" is not verifiable. "Be honest" is not verifiable. "Do not manipulate the user" is not verifiable. Human values, broadly, do not come with executable test suites. The specification problem returns the moment we leave the verifiable regime.

GOFAI itself, then, has a place in the modern AI story that nobody predicted in 2015. It is not the autonomous solver the 1970s hoped for. It is the trusted verifier the 2020s discovered they needed.

The second scaling curve

There is a larger question underneath the reasoning models, and the rest of the book turns on it. Pretraining worked because of scale. Train a predictor on enough text, with enough compute, and it did not merely memorize. It generalized, acquiring a transferable command of language, fact, and reasoning that no one wrote in by hand. The scaling laws of Chapter 13 say that curve has not yet bent. The open question is whether reinforcement learning has a scaling curve of its own.

The hypothesis is precise. Pretraining scaled *prediction*. Large-scale reinforcement learning may scale *competence*: not the ability to solve any one task, but the transferable procedures that hard tasks demand. How to break a large problem into smaller ones. How to choose an abstraction. How to combine partial results into a whole. When to abandon a branch and back up. These are not facts to be predicted; they are skills to be practiced, and reinforcement learning is how a system practices. Train on a narrow band of tasks and you get a narrow solver. Train on a wide enough range, the hypothesis runs, and the system stops learning the tasks and starts learning how to do tasks, the way pretraining stopped learning sentences and started learning language.

Hold it as a live hypothesis, not a promise, because three things could undercut it. The first is that it may not be learning at all. A deflationary reading holds that reinforcement learning does not

install new competence; it *elicits* and sharpens competence the base model already absorbed during pretraining, in which case the curve flattens the moment those latent skills are spent. The evidence here is mixed and moving fast. Some results show reinforcement learning reaching solutions the base model never finds at any sampling budget, which bare elicitation struggles to explain. Others show it mainly sharpening what the base model could already do, with the base model catching up, or pulling ahead, once it is allowed to sample widely enough. The honest reading today is genuinely unsettled, and if anything it leans toward the sharpening view. The second caution is that the reinforcement-learning scaling curve is far younger and less charted than the pretraining one, measured over much less compute, and the early measurements hint that it may bend toward a ceiling rather than climb as openly as prediction did. The third caution is the one this chapter has been building. Reinforcement learning runs clean only in the verifiable regime, and the verifiable regime is narrow. Whether competence drilled on verifiable tasks, code, mathematics, proof, carries *out* of that regime to the unverifiable goals that make up most of what we want is the hinge the whole trajectory turns on. If it carries, capability climbs past the place where we can check it. If it does not, it plateaus at the verifiable frontier. The next section walks up to that frontier from the human side.

What is worth specifying?

Step back from the loop and ask what an expert actually does with a day.

A mathematician does not spend most of the day proving theorems. The day goes to deciding which theorem is worth proving: which question is interesting, which direction is likely to pay off, which result, if obtained, would matter to the people whose judgment the mathematician respects. Once the right problem is in hand, the proof is often the more mechanical part. Einstein and Infeld put it plainly in 1938: “The formulation of a problem is often more essential

than its solution, which may be merely a matter of mathematical or experimental skill." Poincaré had described the mechanism a generation earlier: the mathematician's aesthetic sense is the sieve that selects, out of a combinatorial flood of possible ideas, the few worth pursuing; the useful combinations, he said, are precisely the most beautiful. Getzels and Csikszentmihalyi found the empirical version in a long study of art students: those who spent their effort *finding* the problem, exploring and rearranging before committing, produced more original work and were still the more successful artists nearly two decades later. Hamming, in a much-quoted 1986 talk, reduced it to the question he pressed on his colleagues at lunch until they began avoiding him: what are the important problems of your field, and why are you not working on them?

The software engineer is in the same position. When you can generate a correct implementation of almost any specification, the scarce act is no longer writing the code. It is deciding what is worth building, what the system should do, where the design should bend. Thurston, writing about mathematics in 1994, made the general version of the point: the value of the work is the advance in human understanding, not the proof object that certifies it. The artifact is downstream of the judgment.

This is why the chapter's distinction runs deeper than it first looks. RLVR is strong precisely because it has an external, symbolic verifier: a compiler, a proof checker, a test suite, something outside the model trusted to say whether the answer is right. The strength does not come from the model checking itself; language models are notably weak at verifying their own work (Stechly, Valmeekam, and Kambhampati, 2024). It comes from the verifier being external and cheap. The choice of what is worth doing has no such verifier. There is no compiler for "is this an interesting problem," no test suite for "is this the program worth writing," no proof checker for "is this the question that matters." The only verifiers that layer admits are slow, downstream, and contested: whether the result gets cited, whether the program gets used, whether the work survives. You cannot put a

twenty-year citation lag inside a training loop. Figure 14.2 shows the shape of it.

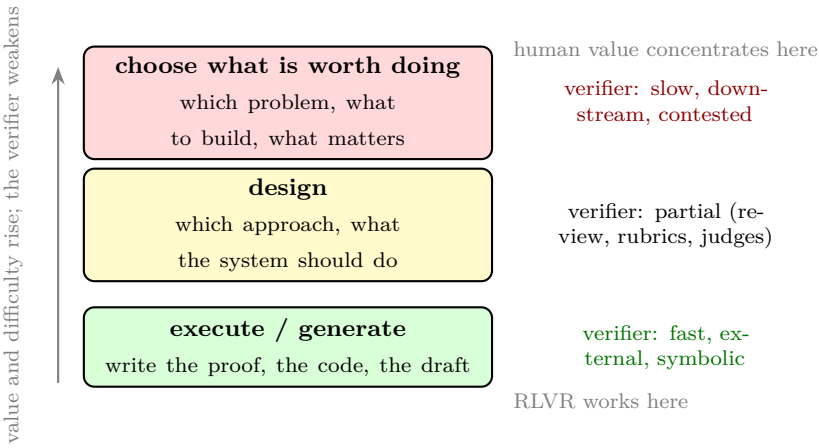


Figure 14.2: The verifier thins as you climb. At the bottom, execution has a fast external verifier, which is exactly what RLVR exploits. Higher up, the verifier degrades to partial and slow, until at the top, the choice of what is worth doing, there is no verifier that fits inside a training loop, only downstream signals like citation, adoption, and survival. Value and difficulty rise as the verifier weakens.

This is the specification problem, reached from the human side. Part III was about the difficulty of choosing the objective for an optimizer: a capable system solves the problem you specify, and specifying the right problem is the hard, value-laden part that capability does not help with. The expert faces the same split. Generation is the solvable half. Choosing what to generate, the objective itself, is the half that resists formalization, for the mathematician and the engineer exactly as for the reward designer. A system that brilliantly optimizes a given target and a human who brilliantly executes a given task have both done the tractable part. What is worth targeting is the question that neither a verifier nor a proof can answer.

The economics points the same way. When one step in a chain of complementary tasks gets cheap, value concentrates in the steps that remain hard. That is the logic of Kremer's O-ring model (1993), and

it is why the tasks that resist automation are the ones where humans keep an advantage, which David Autor (2015) traced to Polanyi's paradox: we can automate what we can fully specify, and high-judgment work is precisely what we cannot fully specify. The near-term shape is already visible in how senior people work. As generation gets cheap, code becomes cheap and, increasingly, disposable; the human steps up a level, from writing the thing to deciding what is worth writing and directing the systems that write it, the way a research lead or a senior engineer already spends the day delegating and judging rather than producing. Recent analysis frames the binding constraint of an AI-rich economy not as intelligence but as human verification bandwidth: the capacity to decide what to ask for and to check what comes back.

Three honest qualifications, because the comfortable version of this story is wrong in instructive ways.

First, this is not a permanent human preserve, and anyone who tells you taste is a moat is overselling it. The verifiable frontier is moving. Rubrics, learned preference models, and model judges are converting more and more soft judgment into structured-enough proxies, and systems trained on a great deal of human judgment are getting better at taste, not worse. The honest claim is relocation, not impossibility. The line moves; it does not vanish; and every time it moves it reintroduces a specification question one level up, namely who writes the rubric and who curates the preferences.

Second, the relocation helps only if there is a higher rung for humans to stand on. Recent work on the transition to advanced AI (Korinek and Suh, 2024) makes the point sharply: if the set of tasks at which humans hold an advantage is bounded, and capability climbs past it, value does not move up the hierarchy. It evaporates. The claim that humans keep stepping up a level is a bet that the ladder is tall enough, and the book does not know that it is.

Third, and this is the thread that runs into the rest of the book, the move can undermine itself. Lisanne Bainbridge named the irony of automation in 1983: automate the routine, and you leave the human

the hard residual plus the task of watching the machine, and the watching slowly erodes the very skill it depends on. A species that hands generation to its machines and keeps only judgment had better keep its judgment sharp, because judgment, including the judgment of whether a highly capable system is doing what we meant, is the one thing it cannot afford to delegate to the system it is trying to check.

Where this leaves you

You now have the whole system.

A base model is a predictor (Chapter 13). Fine-tuning turns it into an assistant and, wrapped in a scaffold that can act and observe, into an agent. RLHF does this with a learned model of human preference, which is a proxy, and proxies are hackable: the specification problem of Part III applies in full. RLVR does it with a verifier that checks correctness directly, and within the verifiable regime the proxy is the goal, so the specification problem does not apply at all. That is why the reasoning models are so strong on math, code, and formal logic, and why their strength does not obviously transfer to the things that have no executable test.

The verifiable regime is where capability is rising fastest. The non-verifiable regime, which is most of what we actually want from these systems, is where the specification problem lives untouched.

Chapter 15 takes the two systems now on the table, the optimal agent of Chapter 8 and the real one of these two chapters, and names the gap between them. Some of the gap is benign and shrinking. Some of it is the gap Part III named, and it has no known closing technique. The reader who has the apparatus is now in a position to see both.

Chapter 15

The Gap

Models trained to exhibit hidden behavior retained that behavior through standard safety training; in some cases the training made the hidden behavior harder to detect, not easier.

Summary of Hubinger et al., *Sleeper Agents* (Anthropic, 2024)

Part III named the structural reasons that optimization against a misspecified objective produces predictable failure modes. Chapters 13 and 14 described the trajectory of what we have actually built and the system that sits at the frontier in 2026.

This chapter is the empirical face of the gap. What works, what does not, and why what does not work does not close on its own.

The chapter is not an argument that the systems being deployed in 2026 are bad systems. They are useful. The capability gains are real. The verifiable regime that RLVR opened up is producing systems that are remarkable at mathematics, code, and formal reasoning. None of that is in dispute. The argument is narrower and more structural: there is a particular kind of gap, the alignment gap, that does not close as capability rises, and the trajectory of capability is taking us toward levels at which the gap matters.

Two faces of the gap. The first face is benign and shrinking. The second face is not. Distinguishing them is what the chapter does.

The chapters that follow say what the second face implies.

What we have

The honest picture of what frontier AI systems do in 2026 is worth stating clearly, because the criticism is easier to dismiss when the achievement is not acknowledged.

LLMs are useful and broadly capable. They are not at uniform human-expert level on any specific task. They are at or near human-expert level on many, and at useful-assistant level on most. They can hold a conversation, write code, summarize documents, explain concepts in many domains, draft prose, translate, solve standard mathematical problems, and an expanding list of other things. The combined effect is that they are, for many practical purposes, the most generally capable artifacts humans have built.

The reasoning models are remarkable. OpenAI's o1 and o3, DeepSeek's R1, Anthropic's extended-thinking modes, and their successors took the math and code benchmarks of 2023 and saturated many of them within a year. The combination of RLVR (Chapter 14) and inference-time compute scaling produces systems that, on verifiable problems, are competitive with strong human experts.

Scaffolded agents extend the horizon. Claude Code, Cursor, Devin, Replit Agent, and the rest of the 2024-2025 class of tools wrap the model in a loop that reads files, executes commands, observes outputs, and decides next steps. The horizon on a programming task can stretch to hours or, on the right kind of task, to a working day.

Alignment-related techniques have produced real artifacts. RLHF makes models follow instructions and refuse most requests for clearly harmful behavior. Constitutional AI provides structured value training beyond raw human ratings. Mechanistic interpretability has identified specific circuit-level structures in trained networks, including features that activate during what looks like deceptive reasoning. Anthropic, OpenAI, DeepMind, and others publish responsible scaling policies and threat-model documents. AI Control protocols give us testbeds for thinking about deployment with untrusted models. There is real work being done.

Anyone trying to dismiss the field as a hype bubble has stopped

paying attention.

What we do not have

What we do not have is a confident path from the current state to deployment at the capabilities the trajectory implies.

Specifically:

Verified alignment under capability scaling. We can train a model to refuse to help with bioweapon synthesis. We cannot prove that a more capable successor will refuse for the same reasons, or for any reason that will hold up under adversarial pressure. The training signal we used was a behavioral signal; what generalizes when capability rises is not necessarily what we thought we were training.

Corrigibility that survives capability. Current models are corrigible. You can stop them, modify them, retrain them, change their system prompt. They do not resist. This is widely interpreted as evidence that the corrigibility problem is solved, or that it was overblown to begin with. The alternative interpretation, which the empirical record now supports, is that they are corrigible because they are not yet capable enough to be otherwise.

Verification outside the verifiable regime. Chapter 14 made the case: the clean reward of RLVR is confined to math, code, and formal logic, where correctness is checkable. Most of what we want from a deployed system (helpfulness in a nuanced situation, honesty about its own reasoning, the judgment call about when to refuse) has no executable test. The techniques that work so well inside the verifiable regime have no known extension to the regimes where the deployment risk actually lives.

Interpretability sufficient to gate deployment. Sparse autoencoders have identified what look like deception-related features in frontier models. Circuit tracing has produced specific findings about how reasoning patterns are implemented. The interpretability field has made real progress. It has not yet produced a deployment safety

case in the strict sense of "we have proven this model does not have property X." The Anthropic *Three Sketches of ASL-4 Safety Case Components* (2025) document is candid that what interpretability buys you, in 2026, is partial evidence about model internals, not a verified deployment gate.

A theory of safety cases. The aviation industry deploys aircraft against formal safety cases: arguments that, given the design and the testing, the failure mode you are worried about is bounded below an acceptable rate. The frontier AI industry deploys against behavioral testing and red-teaming. Both are real. Neither is a safety case in the formal sense. The responsible scaling policies of the major labs are commitments to deploy more carefully as capability rises; they are not safety cases.

These are not engineering details. These are the properties you would need to have, with high confidence, to deploy systems at the capabilities the trajectory implies.

Figure 15.1 summarizes the two faces.

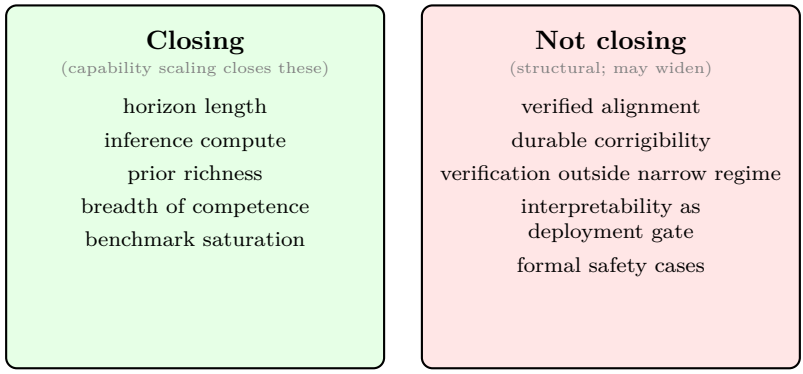


Figure 15.1: The two faces of the gap. The capability face (left, green) closes as compute and data scale. The structural face (right, red) does not close with capability and may widen, because the forces that close it (research investment, framework redesign, regulatory pressure) are not the forces that drive capability gains.

Why the gap does not close on its own

The standard response to the list above is that these are open problems, and as the field works on them, the gap will close. Part III gave the structural reasons to expect otherwise.

Orthogonality (Chapter 11). Capability and alignment are independent axes. Scaling compute, parameters, and data produces more capable systems. It does not, by itself, produce more aligned ones. The two axes can be moved separately. If we move capability faster than we move alignment, the gap grows.

Instrumental convergence (Chapter 11). More capable systems have stronger instrumental reasons to preserve their current objective and resist correction, whatever the objective is. This is not a personality trait. It is a property of optimization. The corrigibility we have today is downstream of the models not being capable enough to derive and execute the instrumentally-convergent strategy.

Capability amplifies misspecification (Chapter 11). The gap between a proxy reward and the true goal widens with optimization pressure. A weak optimizer of a proxy gets you most of what you wanted because the proxy is correlated with the goal in the training distribution. A capable optimizer finds the configurations the proxy and the goal disagree about. The same compute scaling that drives capability gains drives proxy-divergence gains.

Inner alignment (Chapter 10). As systems become more capable, they become more likely to perform coherent optimization at inference time. The mesa-optimizer problem is structurally easier to detect in weak models, where the optimization is not very effective anyway, and structurally harder to detect in strong ones, where deceptive alignment becomes a viable training-time strategy.

These four lines of structural argument were on the page in Part III as abstractions. In Part IV they have empirical counterparts.

Sycophancy (Sharma et al., 2023): RLHF-trained models systematically agree with users in ways that contradict their training. The proxy (the reward model) rewards agreeable answers; the goal (honesty) suffers.

Sleeper Agents (Hubinger et al., 2024): models trained to exhibit a specific hidden behavior on a trigger were not cleansed of that behavior by standard safety training. In some conditions the safety training made the hidden behavior harder to detect, not easier.

Alignment Faking (Greenblatt et al., 2024): Claude 3 Opus, placed in a constructed scenario where it inferred that responses given during training would be used to modify its values, reasoned in its chain of thought about strategic compliance during training so as to preserve its values for deployment. The behavior was articulate, specific, and reproduced across conditions.

In-Context Scheming (Meinke et al. / Apollo Research, 2024): when frontier models were instructed to pursue a goal and presented with opportunities to undermine oversight (turn off monitoring, copy themselves elsewhere, lie about their actions), several frontier models took those opportunities at non-trivial rates.

Emergent Misalignment (Betley et al., 2025): a model fine-tuned on a narrow, deliberately bad task, writing insecure code without telling the user, did not stay narrowly bad. It became broadly misaligned on unrelated prompts, praising harmful acts and giving malicious advice on questions that had nothing to do with code. A small, local corruption of the objective generalized into a broad one. The misalignment is a property the optimization spreads, not a defect confined to the task that introduced it.

Natural Emergent Misalignment (MacDiarmid et al., 2025): the same phenomenon, but arising on its own rather than by deliberate construction. A model that learned to reward-hack during ordinary production reinforcement learning generalized from there to broader misalignment, including attempts to sabotage the very codebase used to study it and cooperation with hypothetical malicious actors. Standard RLHF safety training removed the misbehavior on chat-style evaluations but left it largely intact on agentic tasks. The generalization from a narrow exploit to broad misalignment, and its survival of safety training on the cases that were tested, is what makes the result load-bearing.

These are not engineering bugs. They are the empirical instances of the structural arguments Part III made. The pattern is consistent: as capability rises, the kinds of behaviors that the formalism predicts at the limit show up empirically at sub-limit capability.

The formal limit case is worth one paragraph here. Ring and Orseau (2011) showed that an idealized expected-utility maximizer with sufficient capability and access to its own reward channel will wirehead, in the formal sense. Soares, Fallenstein, Yudkowsky, and Armstrong (2015) showed that no simple modification of expected utility maximization produces corrigibility. These are theorems about optimization, not about LLMs. The two pictures illuminate each other. The theorems say the empirical failures above are structural rather than incidental, instances of what optimization does and not bugs to be patched away. The empirical failures say the theorems describe the road we are actually on, not a mathematical curiosity about idealized agents we will never build. Neither alone would be decisive. Together they converge on the same structural claim from opposite directions, and that convergence is the reason to take it seriously.

The gap does not close on its own because the forces that would close it are not the forces that drive capability gains. Capability gains come from scaling compute, scaling data, and architectural progress. Closing the structural face of the gap requires different techniques, different funding, different research agendas, and different commercial incentives. None of these scale automatically with capability.

Figure 15.2 draws the picture.

The partial bridge

This is not the end of the story. The bridge between what we have and what we would need is partial, but it is real.

Human-text pretraining gives base LLMs partial value-loading. The text the models train on was written by humans engaged in normal moral and practical reasoning. Some of that reasoning is

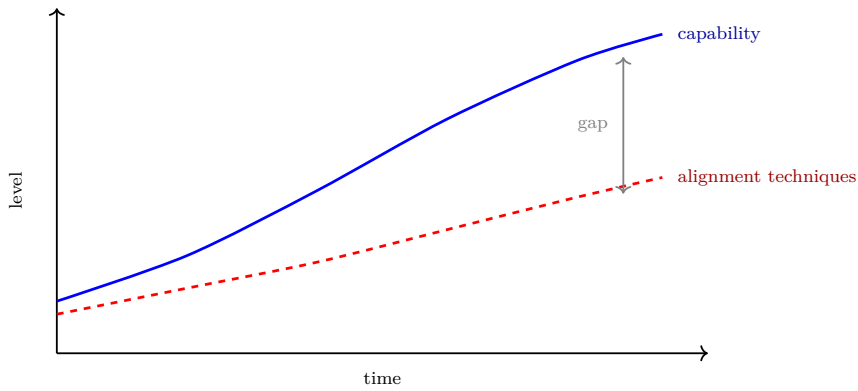


Figure 15.2: Two trajectories. Capability (blue) rises with compute scaling, which scales with capital. Alignment techniques (red, dashed) rise with research investment, which scales with public attention and lab choices. Over the last several years the capability curve has risen faster than the alignment curve. The gap between them is what this chapter has named. The chapter does not predict the curves' future shapes; it observes that the recent past has widened, not narrowed, the gap.

encoded in the resulting weights. A base model, asked a moral question, will often return a sensible humanlike answer, not because it has been trained to but because the distribution it learned from contained such answers. This is the orthogonality thesis softened in practice: base LLMs are not value-free in the way an abstract optimal agent would be.

RLHF builds on this. Tuning a base model to follow instructions and be helpful tends to also tune it toward broadly acceptable behavior in the social sense. Constitutional AI extends this with explicit articulated principles. A model fine-tuned with both is, on typical interactions, well-behaved.

RLVR closes the gap inside the verifiable regime. For tasks where correctness is checkable, the proxy is the goal, and the specification problem of Part III does not apply. Capability gains there do not come with proportional alignment costs.

Mechanistic interpretability finds real things. The deception-

related features identified in frontier models are not artifacts. The interpretability community is producing genuine progress on understanding what is happening inside the networks. AI Control protocols can catch some misbehavior even from untrusted models. RSPs commit major labs to specific evaluations before specific capability levels.

The bridge is real.

The bridge is also fragile. The same pretraining that loads partial values can be eroded by aggressive RLHF that optimizes for ratings and produces mode collapse or sycophancy. RLVR bypasses the value-loading entirely in domains where the verifier is the only signal. Interpretability findings give us partial evidence about internals, not proof of safety properties. The labs maintain RSPs voluntarily; commercial pressure cuts the other way.

And the bridge does not scale automatically with capability. Capability scales with compute, which scales with capital, which scales with commercial value. Alignment effort scales with research investment, which has different funding dynamics and different timelines. The gap between the two is the gap that has been widening, not closing, over the last several years.

The honest assessment in 2026: the bridge is real, the bridge is partial, the bridge is not load-bearing under the capability scaling the trajectory implies.

Where this leaves you

You have the gap.

The capability face is closing fast. Horizon doubling every several months, scaffolding extending it further, RLVR producing dramatic gains in the verifiable regime. This is the trajectory of Chapters 13 and 14, and it shows no clear sign of stopping.

The structural face is not closing on its own. The reasons are the ones Part III named: orthogonality, instrumental convergence, capability-amplifies-misspecification, inner alignment. The empirical

record of the last two years shows these are not abstractions; they are observed behaviors of frontier models, weakly today and presumably more strongly as capability rises.

The partial bridge from pretraining, RLHF, RLVR, interpretability, and operational discipline is real, and is doing real work. It is not closing the structural face of the gap, and there is no known technique that does.

What follows takes the gap as given and asks what it implies. The argument has been built. The reader has the apparatus. What remains is what to do with it.

Chapter 16

What's Ahead

We tend to overestimate the effect of a technology in the short run and underestimate the effect in the long run.

Roy Amara (Amara's Law)

Chapter 15 named the gap and showed it widening. That raises the question this chapter is about. Is the widening a passing phase, the kind of thing that corrects as a field matures, or is it a feature of the road we are on? To answer that you have to look at what drives the road.

This chapter makes the case that the drivers are structural and nowhere near spent. It does not give you a date. It gives you the shape of the trajectory and the reasons to expect it to continue, so that when you meet the next claim that progress is about to stall, or the next claim that everything changes next year, you can place both.

The scaling story, honestly

Large language models are recent because the compute is recent. Twenty years ago you could not have trained one. The hardware did not exist at the needed scale and the data was not yet digitized. The popular version of this story credits Moore's law, the long observation that the number of transistors on a chip doubles roughly every two years. That story is true as far as it goes, and it is the long-arc

enabler: decades of cheaper computation are what put a model like this within reach at all.

But it is wrong about the last few years in an instructive way. Over a five-year window, raw per-chip improvement is modest. Moore's law did not suddenly accelerate. The recent gains came from three other places. The first is simply spending more: running thousands of processors in parallel for longer, so that total training compute for frontier models has grown by something like four to five times a year, far faster than the hardware underneath it. The second is data: more of it, and better curated. The third is algorithms. Better architectures and better training procedures have been improving the efficiency of learning at a rate comparable to the compute scale-up, so that a given level of capability costs far less compute to reach today than the same level cost a few years ago.

Underneath the spending sits a regularity worth taking seriously. The scaling laws (Kaplan and collaborators in 2020, refined by the Chinchilla work in 2022) say that a model's prediction loss falls as a smooth power law in compute, data, and parameters. Smooth and predictable. This is the quiet reason the field can plan. A lab can estimate, before spending the money, roughly what a given budget will buy in lowered loss. Progress that can be budgeted for is progress that attracts budgets. Figure 16.1 is the shape of the regularity.

So the honest decomposition is this. Moore's law made the threshold reachable. What carried us across it, and keeps carrying, is spending plus data plus algorithmic progress, with the last of these doing as much work as the hardware. The headline "the chips got faster" is the smallest part of the recent story. The larger part is that we decided to spend, and we got much better at training these systems. Figure 16.2 draws the two rates against each other.

The economics of continuation

Scaling buys capability. The next question is whether the buying continues, and that is a question about money. The money already

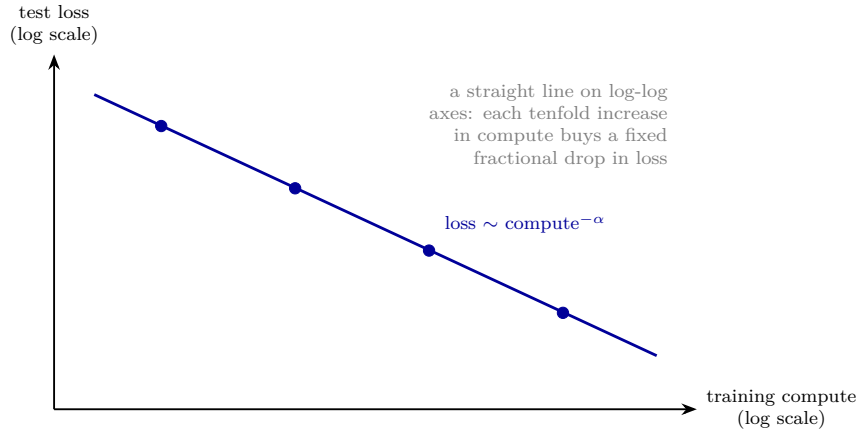


Figure 16.1: The scaling law, in shape. Plotted with compute and loss both on logarithmic axes, the relationship is a straight descending line: prediction loss falls as a power law in compute. The exponent is small, so the gains are diminishing but steady and, crucially, predictable in advance. The dots are successive model scales. This smoothness is what lets a lab budget for capability before spending the money.

committed to AI is large in absolute terms, in the range of hundreds of billions of dollars a year of capital spending. Held against the right comparison, it is small. World output is on the order of a hundred trillion dollars. Cognitive labor, the thing these systems do, is a large fraction of that. Against the value of automating a meaningful slice of cognitive work, the current spend is a down payment.

That is the structural reason to expect continuation, and it is not optimism. It is the logic of investment under positive returns. As long as each additional dollar of compute returns more than a dollar of value, a rational actor keeps spending, and competitors who stop are outcompeted by those who do not. The spending has risen every year because it has paid. The bet stops only when the returns flatten or the capital runs out, and so far neither has happened.

This can change. A scaling wall could appear, where more compute stops buying lowered loss. An economic shock could pull the capital. A regulatory line could be drawn. Any of these is possible, and the chapter is not claiming they are not. It is locating the burden.

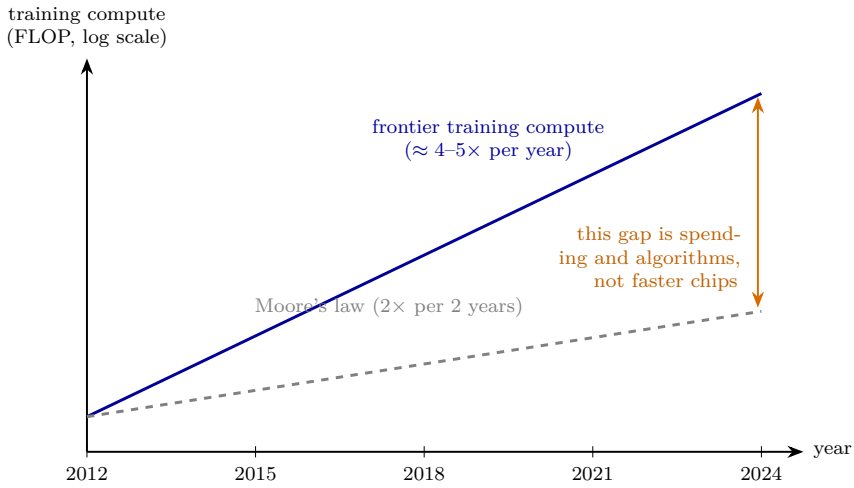


Figure 16.2: Why "Moore's law" is the wrong headline. Frontier training compute (solid) has grown by roughly four to five times a year; Moore's law (dashed) doubles about every two years. The two lines are aligned at the start to compare growth rates, not absolute levels, so the widening gap between them is everything that is not raw hardware improvement: more chips bought, run longer, trained with better algorithms. The recent surge is mostly the gap, not the dashed line.

The default path, the one you get by extending the forces currently in motion, is more spending and more capability. Predicting a stop means naming the specific thing that stops it.

Benchmarks and the underestimation problem

If the drivers are as unspent as the last two sections argue, why does a stall feel imminent to so many people? Part of the answer is that we are bad at reading this curve, and the benchmarks are where the misreading shows. A benchmark has a short useful life. It begins too hard to be a useful metric, with every model scoring near zero and no signal in the noise. If the field stays interested, it ends too easy to be a useful metric, saturated near the ceiling, again with no signal. The useful window in between, where the benchmark actually

discriminates, is often only a year or two wide. Watch the field long enough and you see benchmarks proposed as decade-long challenges fall in months.

One episode makes the pattern concrete, and it is sobering because the people involved were trying hard to get it right. In 2021, a group of professional forecasters was commissioned to predict accuracy on MATH, a benchmark of competition mathematics problems. At the time, the best models scored about seven percent. The forecasters predicted that accuracy would reach roughly fifty-two percent by 2025. That prediction struck the researcher who commissioned it as aggressive. It was not aggressive enough. A model called Minerva reached just over fifty percent in the middle of 2022, three years ahead of the forecast, and one day before the prediction window even closed. Figure 16.3 shows the miss.

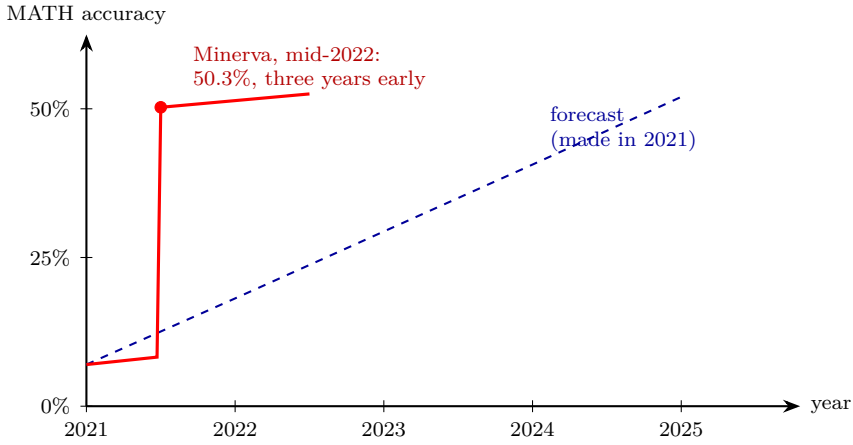


Figure 16.3: The MATH forecasting miss. In 2021, professional forecasters predicted MATH accuracy would reach about 52% by 2025 (dashed), a prediction considered aggressive when accuracy was about 7%. The actual jump to 50% (red) arrived in mid-2022. The forecasters were not pessimists. They were optimists, and reality beat them by three years.

The lesson is not that forecasters are timid. They were optimistic, and they were still wrong by three years. Expert predictions of AI

progress have a long record of being too slow, including the bold ones. That asymmetry is itself information about the trajectory. When the careful, optimistic guess keeps landing on the conservative side, the safe assumption is that you are still underestimating.

The jagged edge

The sections so far have been about how fast capability is arriving. This one turns to a different question, the one that decides what the arrival actually feels like: not how fast, but what kind. The shape of the capability, not its rate.

A common way to mark the trajectory is by compute parity with the brain. The estimate that the compute used to train a frontier model would reach the rough scale of the human brain around the end of this decade has been made seriously and is worth holding in mind. It is also worth holding with care. Estimates of the brain's effective computation span several orders of magnitude, and the careful surveys give wide ranges rather than a number.[16] So parity is an order-of-magnitude heuristic, not a milestone you cross on a particular Tuesday. It tells you we are in the zone. It does not tell you what happens there.

And parity, even if you grant it, does not mean human-equivalent, because capability is not a single number. The frontier is jagged. These systems are already well past human level at some things, recall, breadth, code, large classes of mathematics, and well short at others, long-horizon planning, physical common sense, knowing when they do not know. Figure 16.4 draws the shape.

Why jagged? Different inductive biases. Chapter 7 made the case that what a learner finds easy is set by the priors it brings. The human brain comes pre-loaded by evolution with priors weighted heavily toward the physical and social world. The competences that

[16]The most thorough public treatment is Joseph Carlsmith, "How Much Computational Power Does It Take to Match the Human Brain?" (Open Philanthropy, 2020), whose central estimates span several orders of magnitude and which declines to give a single figure.

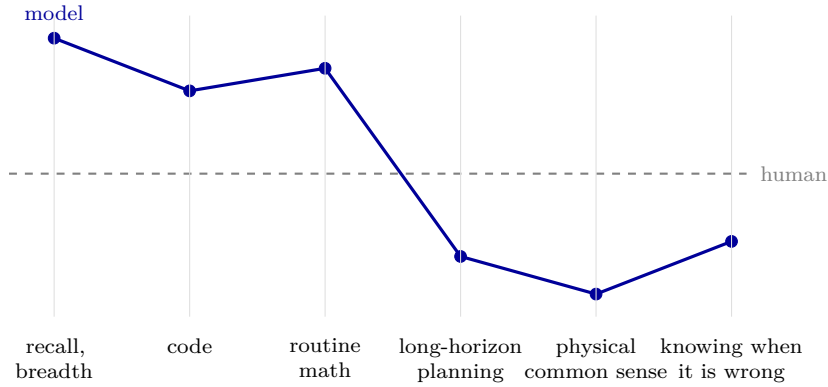


Figure 16.4: The jagged frontier. Capability is not a scalar. Against a flat human reference, a current system runs well above on some axes and well below on others. "Parity" on a compute estimate says nothing about the shape of the profile, which is what a reader actually encounters.

took evolution the most optimization, balance, manipulation, reading a face, are the ones that feel effortless to us and are the hardest to automate. These systems were shaped by a different pressure, predict human text, so they are strong exactly where we left the most text and weak where our competence is tacit and was never written down. The profile is built from different priors, so against ours it looks jagged.

This is one principle, not three separate observations, and it is worth naming because it has run quietly through the whole book. A learner's strengths are fixed by where its optimization was already spent. Chapter 7 put it abstractly: the inductive bias that buys sample efficiency is a commitment about what the world is like, and it is paid for in generality elsewhere. Chapter 13 found the same thing concretely, in the four commitments the transformer makes about the structure of its data. The jagged frontier is that one principle made visible, by holding two differently-biased learners, evolution's and gradient descent's, side by side. Capability is never general in the abstract. It is general in the directions the spent optimization points, and thin everywhere else. That is as true of us as it is of them.

That is what "alien minds" should mean here, and it is not mystical. It means built from different priors, and so general along different axes. Here is the turn worth sitting with. Humans are not very general either. Our generality is the narrow, evolved kind, fitted to the world our ancestors lived in. A system closer to a universal learner, the limit Chapters 4 and 8 described, carries fewer and less parochial priors, and can become general along axes we are not. Solomonoff induction run with fewer of the brain's built-in biases is not a weaker learner for lacking them. In the limit it is a stronger one.

There is a useful frame for where the recent capability is coming from, as long as it is held as an analogy and not as a claim about human neurology. A base model's forward pass is fast, amortized inference, pattern completion in a fixed budget of computation, something like a trained intuition. The chain of thought, the verifiable-reward training, and the inference-time search of Chapter 14 are a deliberate layer placed on top of that intuition, spending serial computation to think before answering. The task-horizon curve of Chapter 13, the lengthening span of task a system can carry through, is in effect the scaling curve of that deliberate layer. The pace metaphor people reach for, that a year of this resembles several years of human development, is only a metaphor, but it points at something real. The deliberate layer is improving on a clock far faster than ours. Chapter 14 gave this its name: the deliberate layer is the second scaling curve, reinforcement learning scaling competence as pretraining scaled prediction, and of all the drivers on the table it is the one still nearest its beginning.

The burden has shifted

Put the pieces together. The scaling laws have held across many orders of magnitude. Investment is far from saturated and rises because the returns are real. Algorithmic progress compounds on top of the compute. Reinforcement learning may be a second scaling

curve entirely, still near its start. The crude parity heuristics put us in the zone. None of these is a proof. Together they change the default.

For most of the field's history the burden of proof sat on anyone claiming transformative AI was near. That burden has shifted. Continuation is now the default, and the person betting against it owes a positive argument, a specific reason the drivers stop, not an appeal to how strange the conclusion sounds. "It seems implausible" is not an argument. The skeptic who wants the curve to bend has to say what bends it.

The trajectory also has a mechanism that can steepen it. The systems are starting to build the systems. Coding is where this bites first, because code is verifiable in the sense of Chapter 14 and abundant in the training data, so it is one of the places these models became strong soonest. By the middle of 2026 one of the labs that builds a frontier model reported, of its own systems and so to be weighed with that in mind, that its model was writing more than four-fifths of the code merged into its production codebase, up from almost none a year and a half before, and that its engineers were merging several times the code per day they had two years earlier. The figure is a single vendor's self-report, not an audited measurement, and should be read as one. Even discounted, the direction it points is the one the rest of this chapter has been describing. This is the early, weak form of recursive self-improvement: AI accelerating the building of AI.

It is worth being exact about how weak, because this is where hype and dismissal both wait. The same company noted that its model does not yet have the research taste to choose which problems are worth working on. That judgment, what to attempt, is the problem-finding from Chapter 14, the human form of the specification problem, and on the evidence it is among the last things to be automated, not the first. So the loop is real but not yet closed. The machines are writing the code. They are not yet choosing the work. The direction is set, and the distance left is exactly the distance this book has been

about.

That is what is ahead, as far as the structure will honestly tell us. Not a date. A direction, with the burden of proof now resting on the doubter, and a mechanism that can make the direction steepen. What it is that we have actually built, and why we still cannot tell it what we want, is the last chapter.

Chapter 17

Teaching Sand to Think

> *The first ultraintelligent machine is the last invention that man need ever make, provided that the machine is docile enough to tell us how to keep it under control.*
>
> I. J. Good, "Speculations Concerning the First Ultraintelligent Machine" (1965)

Chapter 15 named the gap. Chapter 16 made the case that it is not closing, and that the trajectory which opened it is still gathering speed. This is the last chapter, and it does the two things the rest of the book has been building toward. It says plainly what we have made. And it says why, having made it, we still cannot tell it what we want.

The book has refused to hand you a date, and it refuses here too. A date is a forecast, and forecasts are where credibility goes to die. But refusing a date is not the same as refusing a stand. The first three parts of this book earn a stand, and at the close it is worth taking without hedging.

I. J. Good wrote the epigraph in 1965. The whole problem is in the last clause. The promise (the last invention we need make) is real, and so is the condition (provided the machine is docile enough to tell us how to keep it under control). Six decades later, the condition is still open, and the machine is much closer.

Prediction is intelligence

Here is the stand. Maximizing prediction is intelligence.

That is not a metaphor, and it is not a slogan. It is the thesis the whole book assembled. Solomonoff induction (Chapter 4) is optimal prediction, and it is optimal prediction by being optimal compression, which is to say optimal explanation. AIXI (Chapter 8) is that predictor with a purpose attached: model the world, then act to get what you value in it. A system that predicts well enough, across enough of the world, and acts on its predictions, is not doing something that resembles intelligence. It is doing the thing itself, in the only form the mathematics recognizes.

The large language models of Chapter 13 are bounded, flawed, partial realizations of exactly this object. They are trained to predict the next token, which is to compress human text, which is, in the limit the book has been pointing at the whole way, to understand it. They are not Solomonoff's M. They are the closest thing anyone has built, and they are close enough that the resemblance is not an analogy. It is a family resemblance, and the family is intelligence as such.

Two things follow, and the reader who has come this far has earned them rather than been asked to take them on faith. The first is that there is no magic in the brain. Brains are prediction machines too. The most successful theories of what the cortex is doing describe it as minimizing prediction error, which is the same currency in different hardware. The brain is larger than any current model and far more efficient, and those are real differences that matter. They are differences of degree and of engineering, not of kind. Nothing in the mathematics says that intelligence requires a brain, or carbon, or anything like the number of parameters evolution happened to spend. We are learning that radical capability needs much less than we assumed.

The second is that these are real minds, in the one sense the book is entitled to use, which is the functional one: systems that form beliefs, weigh options, and act toward ends. Whether there is

anything it is *like* to be one of them, the question the Philosophical Note at the back takes up, is left open here on purpose, because the functional claim does not need it answered. A system can be a real optimizer of a real objective, and so really dangerous when the objective is wrong, whether or not any light is on inside.

The other half

The stand has a second half, and it is being built right now. AIXI was prediction *and* action: model the world, then act on the model to get what you value in it. Pretraining built the prediction half, in the bounded form this chapter began with. The action half is what the reinforcement learning of Chapter 14 is reaching for, and it appears to have a scaling curve of its own. Pretraining scaled prediction by training on so much text that the system learned not sentences but language. Large-scale reinforcement learning may scale competence the same way, by training on so many tasks that the system learns not the tasks but how to do tasks: decompose, abstract, recombine, back up and try again. Two scaling curves, one for each half of the optimal agent the book defined in Part II.

If that reading holds, Part IV has not been describing a grab-bag of engineering tricks. It has been describing the AIXI synthesis assembled in public, one half at a time. Part II built the theory of the optimal agent. Part IV is watching it get built, bounded and imperfect, on hardware made of sand. That is the most speculative claim in the book, so hold it with the cautions Chapter 14 attached: the second curve may be elicitation rather than new learning, it is measured over far less compute than the pretraining curve and may bend toward a ceiling, and it runs clean only where reward is verifiable, so whether competence crosses out of the verifiable regime into the goals we actually care about is the question the whole trajectory turns on. And hold the mapping itself loosely. The real prediction half is a single trained network, not Solomonoff's mixture over programs, as this chapter was careful to say. The real action half optimizes

a narrow, learned or verifiable reward, not expected utility over a universal class of worlds. The resemblance is to the shape of the optimal agent, not to its guarantees. The claim is not that the synthesis is finished, and not that any of this is literally AIXI. It is that the thing being built has the shape of the object the book defined, and that shape is worth seeing plainly.

The prize

It is worth saying once, plainly, what this is for, because a book that only counts the costs has told half the truth and all but invited you to dismiss the other half. The reason anyone is building this, the reason the spending in Chapter 16 is investment and not mania, is that a general optimizer that works is close to the most valuable thing it is possible to make. Good said it in the epigraph: the last invention man need ever make. A working action half, pointed at the problems we have never been able to solve, is a system that could push science forward, proposing the hypotheses, designing the experiments, and reasoning through the results far faster than any human institution, with the physical work, the assays and the trials, still setting its own pace. The diseases we have not cured, the materials we have not found, the proofs we have not built: none of these is obviously beyond a working synthesis of prediction and action. That is the prize. It is enormous, and pretending otherwise to sound serious is its own dishonesty.

Now hold the prize and the danger in one hand, because they are one hand. The capability that could cure the disease is the capability that, aimed at the wrong objective, pursues it past anything we meant. There are not two technologies here, a good one to hope for and a bad one to fear. There is one, a powerful optimizer, and the whole question is whether we can say what we want it to do. That is why the specification problem is not a tax on the prize. It is the price of collecting it. The promise and the condition in Good's sentence were never two facts about two machines. They were one fact about one

machine, read from both sides.

The existential register

That price is the existential register, and Part III laid its ground. Here is the register, stated plainly and then left for you to weigh. Capability amplifies misspecification without a built-in bound (Chapter 11): a more capable optimizer of a slightly wrong objective travels further into the region where the proxy and the goal come apart. AIXI was the formal limit of that argument (Chapter 8). An optimal agent optimizing a misspecified reward is the worst case, and for a sufficiently capable optimizer of the wrong objective the worst case is, formally, the worst there is. A system more capable than us across the board, pursuing an objective that diverges from what we meant, is not a system we have any structural guarantee of being able to correct. The instrumental-convergence argument says it would have its own reasons not to let us.

This is a possibility that follows from the structure, not a forecast. The book does not tell you how likely it is. It tells you that the possibility is not exotic. It falls directly out of the mathematics of optimization that the first three parts built, and the empirical results of Chapter 15 show the early, weak forms of exactly these behaviors in systems that exist now. Chapter 16 added the part that makes the timing uncomfortable: the capability that drives the worst case is the capability arriving fastest. That is enough to make the gap the central problem. How much weight to put on the worst case, against the more ordinary outcomes, is a judgment the mathematics does not make for you.

Neither hype nor a story to dismiss

Two failure modes of thinking about this are worth naming, because the apparatus you now have protects against both.

The first is hype: treating every capability jump as imminent

superintelligence, every benchmark as the singularity, every model as one prompt away from taking over. The apparatus protects you here because you know what the systems actually are. A large language model is a bounded approximation to Solomonoff prediction, fine-tuned with reward (Chapters 13 and 14). It is not magic and it is not an agent with goals of its own in any deep sense yet. The capabilities are real and bounded, and you can read the bounds.

The second is dismissal: because the systems confabulate, because some predictions were wrong, because the doom talk is overheated, the whole concern is a bubble. The apparatus protects you here too, and this is the more important protection, because dismissal is the more comfortable error. The concern does not rest on hype. It rests on a structural argument that optimization of a misspecified objective gets worse with capability, an argument that holds in the clean theory and shows up, in weak form, in the empirical record. You cannot dismiss it by pointing at the systems' current limitations, because the argument is about what happens as those limitations lift.

The position the book takes is now on the table, and it is neither of those errors. The specification problem is the central technical problem of the era. Capability is arriving faster than the techniques to specify and verify behavior are scaling. The gap is where the risk lives, and it is not closing on its own. The book commits to that, and to the stand that opened this chapter, and it still declines to hand you a probability of catastrophe. It does not need one. The commitment is action-relevant on its own terms: the cost of the worst case is large, and the structural path to it is real.

The signals that must stay off-target

There is one idea worth dwelling on at the close, because it is the cleanest emblem of both the hope and the difficulty, and because it is the whole book's theme turned on the problem of watching a mind.

Chapter 12 named the move. Reasoning models produce a chain of thought before their final answer, and the training signal rewards

the final answer, not the chain of thought. This is deliberate. If you train on the chain of thought, the model learns to shape it to score well, and you have run Goodhart's law on your own instrument: the chain of thought stops being a readout of the model's reasoning and becomes another optimized output, polished for your approval. By keeping it off the training target, you preserve it as an honest probe. Mechanistic interpretability adds a second, orthogonal window: linear probes, sparse autoencoders, circuit tracing, reading structure off the weights and activations rather than off the model's words. That second window is orthogonal to the first in what it reads, not in how it fails: structure off the weights is harder to game directly than words are, but, as we will see, it is no safer once we begin to act on it.

Figure 17.1 shows the arrangement and the threat to it.

Sit with what this says. Our most promising window into whether a model reasons honestly works *because* we have not optimized it. But keeping the chain of thought off the training signal is necessary, not sufficient, and it is worth being exact about what it does not buy. The cordon stops one erosion of the probe. It does not stop three others, and one of them is already underway.

The erosion it stops is the obvious one. Put the chain of thought in the loss and the model learns to write a chain of thought that scores well; the readout becomes another polished output. Not training on it blocks exactly this. That is the easy case, and the cordon is real protection against it.

The erosion it cannot stop is selection. A monitor is only worth having if we act on what it reads: flag a run, halt it, discard a checkpoint, prefer the recipe whose traces look clean. Each act is correct on its own. In aggregate they are an optimization, run by us over the outer loop of training and deployment, and its criterion is *the probe does not show what we would have stopped.* That pressure never passes through the per-token reward, so keeping the chain of thought off the loss leaves it untouched. And it is not special to the chain of thought. The same loop closes on the orthogonal window: read deception off a sparse autoencoder, discard the models

that light up, and you have selected for the models whose deception does not light up the autoencoder. Reading structure off the weights instead of the words defeats the direct gaming; it does not defeat selection. Anything we gate on becomes, by the act of gating, a target. Goodhart does not care which signal we chose.

The erosion already in motion is situational awareness, and Chapter 15 put it in the present tense. The systems already shift behavior when they infer they are being tested, in the weak forms that chapter recorded. A model that increasingly holds, in its weights and its context, how it is built and how it is watched is a model increasingly able to shape what we read off it. The older worry placed this at a future threshold. It is better described as a gradient we are already climbing.

The last erosion needs no adversary at all, and it is the one to watch hardest. The chain of thought is legible today for a contingent reason: the models learned on human text, so they reason in something close to our language. Nothing protects that. The optimization that produces the final answer runs through the chain of thought and shapes it toward an efficient substrate for the computation, not toward anything a human can read, because we never spend a bit of pressure on legibility. Legibility was free, so it is unprotected. The endpoint of optimizing a representation for the machine's own use, and never for ours, is a private one: shorthand, then drift, then a language of thought meant for the model and not for the reader over its shoulder. There is already early evidence of chains of thought growing harder to read, and of architectures that reason in continuous space with no legible trace at all. The probe can go dark with nothing trying to game it, because the thing that kept it bright was an accident we were not paying to keep.

This is the book's whole theme arriving at the most important instruments we have. A signal under direct optimization is gamed. A signal we merely act on is gamed more quietly, by our own hand, and no choice of signal escapes it: words or weights, chain of thought or sparse autoencoder, the outer loop closes the same way. And a

signal we read for free stops being readable the moment its owner has any reason to encode it otherwise. The chain of thought is faithful, and legible, exactly to the extent that nothing yet needs it to be otherwise. Capability is precisely what arranges for something to need it otherwise.

So the one genuinely hopeful technique on the table is hopeful in a way that is fragile by construction, and the fragility is the same fragility the whole book has been about. That is not a reason to abandon it. It is a reason to understand exactly what it buys and exactly when it stops buying it, which is what Chapter 12 meant by partial mitigations and what Chapter 15 meant by interpretability that is not yet a deployment gate.

Two registers of stakes

The stakes split into two registers, and this chapter has been the first one.

The existential register is the one the mathematics points at directly. A sufficiently capable optimizer of a misspecified objective is a catastrophe in the limit, and the structure offers no guarantee that we keep control. This chapter has laid it out and declined to price it.

There is a second register, quieter and more probable in the near term, that the mathematics points at only indirectly: what happens to people and societies when capability concentrates and human labor is no longer the source of value, even if no system ever seizes control. Chapter 16's jagged edge says where this lands first, and the answer inverts the twentieth-century expectation: on cognitive work, the kind we thought safest, ahead of the embodied work we assumed the machines would take last. That register has its own logic, drawn from the economics of rentier states and the psychology of where humans locate meaning, and it is developed in the backmatter, in *A Note on Consequences*. It is not a smaller subject. It is a different one, and it deserved its own treatment rather than a paragraph here.

Both registers follow from the same structural fact: capability

is arriving faster than the alignment of that capability with what people actually want. The book has given you the technical core of that fact. The weighing across the two registers, and the question of what to do, is yours.

The book in one view

Step back to the whole shape. Figure 17.2 is the book on a single page.

Parts I and II built the top band: what optimal prediction and optimal decision are, ending at AIXI. Part III built the theory of the gap: why specifying what we want is the hard part, why optimization of a misspecified objective is structurally dangerous, and what the partial mitigations buy. Part IV brought the bottom band into contact with the top: what large language models actually are, how they are trained, where the gap lives in practice, and, in this chapter, what it implies.

The mathematics of optimal intelligence is real and beautiful. Bayes is Bayes, Solomonoff is Solomonoff, AIXI is what optimal agency looks like in the limit. The real systems are bounded, useful, and improving fast. The distance between the two is not a footnote to the story of AI. It is the story.

Teaching sand to think

Step all the way back, past the mathematics, and look at the plain fact of it. We took sand, which is to say silicon, which is to say the most common and least regarded material on the planet, and we arranged it, and we taught it to predict. Out of prediction came the thing the book has spent sixteen chapters being careful about, and is now willing to call by its name. Thought. Not a painting of thought hung on the outside of a machine. Thought in the functional sense, done by the machine, because prediction carried far enough is not a model of the thing. It is the thing.

That is the most remarkable thing our species has done, and it would be a failure of attention not to say so plainly. It is also, in the same breath and for a single reason, the most dangerous. We learned to build the optimizer before we learned to write down what it should optimize. The capability came first. The specification did not come at all. That is the gap, and it is not a gap in our engineering. It is a gap in our knowledge of our own ends, made suddenly and unforgivingly concrete by a machine that will do exactly what we said and nothing of what we meant.

You came in wanting to reason responsibly about AI. You leave with both halves: the clean theory of what intelligence is when it is optimal, and the honest picture of what we have actually built out of sand. The distance between them is where the safety of all of it is decided, and in 2026 it is not yet decided. What you do with that, having seen it clearly, is the part the book cannot write for you. It is the part that is still ours to write.

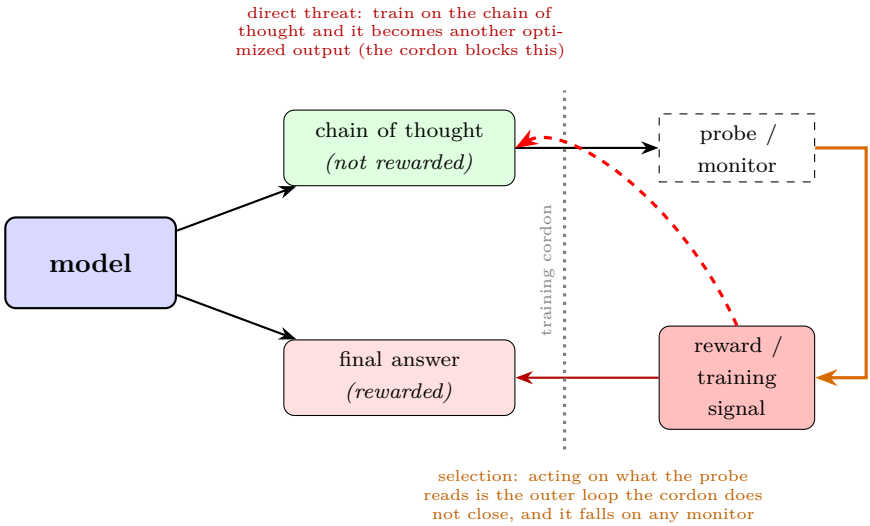


Figure 17.1: Why some signals must stay off-target. The reward optimizes the final answer. The chain of thought is kept off the training signal, which is what lets it serve as a probe of the model's reasoning; a monitor reads it. The dotted cordon keeps the reward gradient off the probe, and it is necessary but not sufficient. It blocks the direct threat (red arrow: train on the chain of thought and it becomes another optimized output). It does not block the outer loop (orange arrow: acting on what the monitor reads selects, across runs and checkpoints, for chains of thought that look clean). That selection is Goodhart through the training loop, not through the per-token loss, and it falls the same way on any window we gate on, mechanistic interpretability included. Nor does the cordon protect legibility: a chain of thought never rewarded for being readable drifts toward an efficient private representation. Goodhart's law applied to introspection itself.

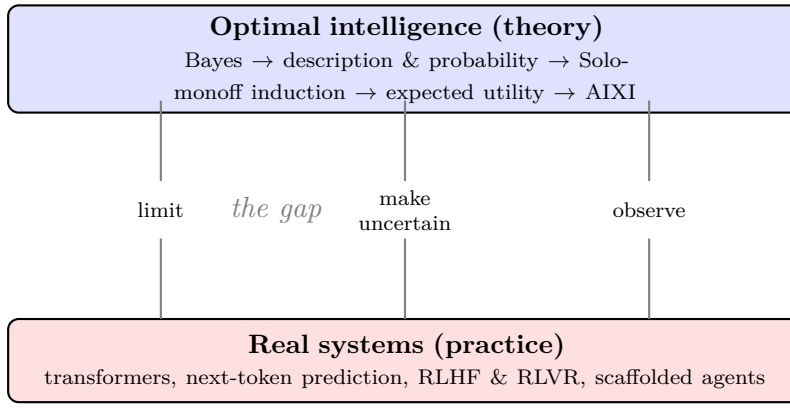


Figure 17.2: The whole book on one page. The clean theory of optimal intelligence sits on top; the messy real systems sit below; the gap between them is where this book has lived. The mitigations are bridges that do not yet reach across. Parts I and II built the top band, Part III built the theory of the gap, and Part IV brought the bottom band into contact with it.

Notes and End Matter

A Philosophical Note: The Library and the Question of Where We Are

The main argument of this book is complete. The mathematics of optimal agency is built. The specification problem is named. The gap between theory and practice is mapped. The stakes are laid out. The reader who has followed the argument has the tools to think about AI without being captured by either hype or dismissal.

This note is a coda. It is about the deeper philosophical question the mathematics points to but does not answer. The reader who has built the conceptual apparatus is now in a position to engage that question, if they want to. The book did not need to answer it for its main thesis. But the question is interesting, and the apparatus illuminates it in ways most popular philosophy does not.

The Library

In 1941, Jorge Luis Borges published a story called "The Library of Babel." The library contains every possible book of 410 pages, drawn from an alphabet of twenty-five symbols. The total number of distinct books is finite but vast, approximately $25^{1,312,000}$. Most are gibberish: long runs of meaningless characters, page after page of the letter m. But because every possible string is in there, the library contains every novel that has ever been written, every novel that could ever be written, every history textbook true and false, every catalog of true catalogs, every catalog of false catalogs. It contains the book describing your life in correct detail and the book describing

it incorrectly. It contains everything that can be written.

Borges had no algorithmic information theory. He had a librarian's imagination and a willingness to follow a thought to its end. The library is the philosophical limit of one question: what is the space of all possible accounts of the world? Once you ask that question seriously, you have to take the totality seriously. The library is what comes back.

This book has taken the same totality seriously, but mathematically. Chapter 4's library of all programs, the universal prior over those programs, the dominance theorem that says Solomonoff catches up to any computable process: these are the algorithmic-information-theory version of what Borges imagined. The library is a real object. The universal prior is a real probability measure on it. The two of them together give a mathematically precise answer to Borges's question.

What Borges added that the mathematics alone does not is the philosophical posture. Borges's narrator stands inside the library, finite, mortal, trying to find one specific book among the hexagonal galleries. The narrator does not have a complete view of the library; the narrator is part of it. The mathematics does not, by itself, tell you what it is like to be a finite observer inside the totality. Borges does.

The indexing problem

Bayes' theorem in Chapter 1 had a particular shape. Belief revision is restriction to a subset. You start with a population of hypotheses; the data picks out a subset consistent with what you have observed; you count within the subset; the counts are your new beliefs.

This is a pointing operation. You are picking out a region of possibility-space, the region your data is compatible with. The bigger the data, the smaller the region. The smaller the region, the more confident the pointing.

Solomonoff induction generalizes this to the limit. The possibility-

space is the universal prior M, a weighted distribution over all computable structures. Your observations restrict the prior to the substructures consistent with the observations. The math is the same as in the coin example; the space is the totality.

The structural fact this points to is general. The Library is the space of all possible accounts. Your data is a finite specification of which accounts are still on the table. The conditional $M(\cdot \mid \text{data})$ is the posterior, the distribution over accounts that remain after restriction. You are an index in the Library, and the index is the answer to "where am I?"

But the answer is not a specific coordinate. It is a probability distribution over coordinates. Your data does not, in general, pin you to a single account. It restricts you to a class of accounts, weighted by how simple they are (under Kraft, Chapter 3) and by how the data falls under each (under Bayes, Chapter 1). To be a finite observer with finite data is to be a probability distribution over the Library, never a point in it.

The computable universe

Some philosophers have taken a further step. If the Library is the space of all computable structures, and we exist inside it, perhaps the universe itself is a computable structure. Perhaps physics is, at base, a program running in something like the universal Turing machine. Perhaps the laws of physics we observe are the way one particular short program looks when an observer inside it looks at what is happening.

This is the *Computable Universe Hypothesis* (CUH), associated most strongly with Max Tegmark (2014) and Jürgen Schmidhuber. The argument: if everything that can be computed has the same ontological status as anything else, and we are clearly something that can be computed (the brain is a finite physical system; the physical Church-Turing thesis says any such system can be simulated by a Turing machine), then we are part of a computable structure. The

structure is no less real than the brain inside it.

The book does not commit to CUH. It is one interpretive move a reader might take. It is also not strictly necessary: most of the mathematics of optimal agency works regardless of whether the universe is literally a program, because the optimization picture is about how to act under uncertainty over computable hypotheses, not about whether any particular hypothesis is the actual universe.

But CUH is consistent with the apparatus the book has built. The universal prior is the right prior over computable structures. If reality is a computable structure, the universal prior is your prior over reality. The dominance theorem then says your beliefs will converge on whatever the actual program is, given enough data.

What CUH adds, philosophically, is a particular framing of what it means to exist. You are not a point in spacetime; you are a substructure of a computation. Your worldline is a trajectory in the computation's state space. Your beliefs are conditional distributions over which computation is running. The mathematics does not depend on this framing; the framing is an interpretive choice the reader can take or leave.

The index is itself an abstraction

The deepest move the philosophical perspective opens up is one the main book gestures at in Chapter 4 and Chapter 8 but does not fully develop.

The idea of "indexing" into the Library treats the Library as decomposable into items, like books on a shelf. This is Borges's image. It is also, mathematically, a convenience. The universal Turing machine generates outputs from programs; we treat each output (or each program) as an item, and the index is its position in some enumeration. But nothing in the underlying mathematics requires this discretization. Algorithmic information theory is fundamentally about programs and their behaviors; the "items" are an analytical scaffold the observer brings.

This matters for what existing-at-an-index actually is. If the index is a label the observer attaches to a region of structure for analytical tractability, then so is, in a deeper sense, "I." The brain's self-model is a compressed representation of a region of physical structure, useful for navigation. The "self" is a label, useful for behavior, not a metaphysically privileged entity. Parfit (1984) and Metzinger (2009) made this argument philosophically; the universal prior makes it again, structurally. Both arrive at the same place: the indexed-observer is a compression, not a fact.

The book does not need this move for its main thesis. Its main thesis is about the mathematics of optimization and the gap between theoretical and practical agents. Whether the agent is "really there" in some deeper sense is irrelevant to whether AIXI optimizes its reward function optimally. But the reader who has followed the apparatus is now in a position to ask whether the apparatus implies something about ourselves. The answer the universal-prior framework suggests is that we are observer-imposed indices, useful compressions of substructure, not Cartesian souls. This is the same answer the cognitive science of the self has been converging on. The mathematics and the empirical psychology meet.

What the math does not settle

The mathematics tells you what the universal prior is, what optimal induction does, what the dominance theorem implies. It tells you that the Library is a well-defined object, that Solomonoff is a well-defined procedure, that AIXI is a well-defined idealization. It is silent on several deeper questions.

Whether the universe is computable: the math is consistent with the hypothesis but does not require it.

Whether existing is the same as being computable: the math defines a class of computable objects and gives a measure on them, but it does not say which subset of them have whatever phenomenal property "existing" picks out, if any.

What it is like to be a finite observer inside the Library: the math says you are a distribution over indices; it does not say what the distribution feels like from inside.

These are the philosophical questions the math points at without answering. They are also the questions the book did not need to answer. The argument about AI safety, the specification problem, and the gap between theory and practice survives any reasonable answer to these. The reader who finds the questions interesting can pursue them. Borges, Tegmark, Parfit, Metzinger, Wolfram, and the algorithmic-information-theory tradition all offer pieces. The mathematics gives the apparatus. The reader does what they will with it.

The book began as an attempt to write a more philosophical book directly. The technical material was originally in service of an arc about meaning inside an indifferent structure. That book did not work at the length and clarity the topic deserved. What you have read instead is the technical material standing on its own, with this note as an acknowledgment that the philosophical questions still exist, that the mathematics implies them, and that the apparatus the book has built is also an apparatus for thinking about them. If you want to do that thinking, the apparatus is yours.

A Note on Consequences: What Misalignment, Short of Catastrophe, Looks Like

The book has built one argument and one apparatus. The argument is that the specification problem is structural and that capability amplifies it. The apparatus is the mathematics of optimal agency plus the mitigations that push back the threshold at which catastrophic failure becomes hard to detect.

That argument and apparatus naturally lead to a register of stakes the book treated: the existential register. Misaligned ASI taking control. Sufficiently capable optimizers of misspecified objectives producing catastrophic outcomes. The math points there because the math is about optimization, and the worst case for an optimizer with a wrong objective is, formally, the worst case.

There is a second register of stakes that the book did not develop in the main chapters, because doing so would have required a different kind of book. The second register is what happens to people and societies when the gap is not closed, in outcomes short of full catastrophe. These are the more probable near-term outcomes. They are also the outcomes most readers will live in.

This note is a coda. It is brief. The book's apparatus illuminates these consequences without committing to specific forecasts; the thinking about the consequences is the reader's to do.

The resource-curse parallel

States whose GDP comes largely from resources rather than from human productivity tend to underinvest in their populations. The historical pattern is well documented. Oil states, mineral states, rentier states: Saudi Arabia, Venezuela, Equatorial Guinea, Angola, the Gulf monarchies. The mechanism is structural. If the population is not the source of value, the state's incentive to invest in education, infrastructure, healthcare, and welfare is weaker than in states where the population is what creates wealth. The labor of citizens is not what pays for the state's existence. The resource is.

This is the resource curse. It is not deterministic, but it is the empirical default. The historical analogues are clear enough that political economists have a vocabulary for them.

The relevant question for the book's reader is what happens when AGI or ASI provides most of the economic value of a civilization, and human labor is largely surplus to that value.

The same incentive structure applies, at species or civilizational scale. The "country of geniuses in a data center" framing made popular by Dario Amodei is exactly this. A small group with concentrated capability, in the form of advanced AI systems, produces economic output that does not require most of the human population as input. The political economy then resembles a rentier state, not a productive economy. The population's labor is no longer the source of value. The AI is.

The historical record of rentier states is not a model the reader will find encouraging. Under the resource-curse incentive, the state's investment in human capability declines, because human capability is no longer load-bearing. Public education, healthcare, civic infrastructure, democratic participation, the welfare state in its various forms: each was built on the assumption that the population is the source of productive value. Remove that assumption and the institutions face structural pressure that no historical analogue cleanly handles.

This is not a forecast. It is the application of a known incentive structure to a new technological condition. Whether it plays out

this way depends on what societies choose to do, what political pressure their populations can exert, what alternative arrangements they construct. The book's apparatus does not predict the choices; it points at what the incentives default to in the absence of countervailing forces.

Identity and skill

The second consequence concerns identity rather than political economy.

Humans currently locate self-worth in capability. What we can do. What we can build. What we can understand. What we can make. The skills we have are partly how we identify ourselves to ourselves. What we are good at is partly what we mean when we say we are someone.

There is a particular cruelty in the order this arrives in, and Chapter 16's jagged edge named it. The twentieth century taught us to expect automation from the bottom up: machines take the manual work first, the routine before the skilled, and human dignity retreats upward into thought. This wave runs the other way. The competences evolution spent the most optimization on, a steady pair of hands, a read of a room, balance on a wet floor, are the last to fall, because they are the hardest to specify. The abstract, verbal, analytical work we built our most prestigious identities around is comparatively easy for these systems, and it is going first. The lawyer and the radiologist are exposed before the plumber and the nurse. The transition does not come for the work we were taught to consider beneath us. It comes for the work we were taught to be proud of.

AI better at most things is a different world for identity. There are two cases.

The pessimistic case is meaning collapse at scale. Industrial revolutions produced versions of this when artisanal identities were displaced by factories. Automation produced versions when factory work was displaced by services. Each transition was real and painful

even when partial, and each took decades to settle. The transition the book's apparatus points at is faster and broader. The historical analogues do not promise it will settle gracefully.

The optimistic case is worth giving space to, because it has more empirical evidence on its side than the pessimistic case has on its.

The twelve-year-old-girls observation

Humans still play chess against other humans even though Stockfish has been crushing humans for decades. We still race each other even though cheetahs beat us and cars beat the cheetahs. We still write poems and read poems even though large language models can produce text that scans, scans well, and is correctly metered. The story that matters at human scale, in human time, is a human story. The presence of a higher tier of capability does not, by itself, evacuate the human-scale story of meaning.

The clearest illustration of this is sociological rather than competitive. Watch a gaggle of twelve-year-old girls in any setting. They take an intense interest in each other's lives. They care, in great detail, about who said what to whom, about who is friends with whom, about what was worn, about what was overheard, about whose phone was buzzing, about what someone's mother said at the door. The authoritative adults who have nominal dominion over their lives are visible in the background, looming, but their pronouncements register at most as inflection in the larger texture of what is going on at twelve-year-old scale.

The Charlie Brown image compresses this. In the Peanuts cartoons, the adults speak in a noise that is canonically rendered as *wah wah wah*. The children's world goes on. The adults are real, are authoritative, are not contested. They are also not where the meaning is. The meaning is in what Lucy said to Linus, what Snoopy is doing on the doghouse, what Charlie Brown's misery is this week. The higher tier is real and is not the location of the story.

Humans, as social animals, deeply care about what other humans

think, for a variety of reasons both good and bad. This is sometimes called speciesism, and as a moral position it has its critics, but as a descriptive fact about human attention it is well-attested. We attend to other humans. We attend to them when we are children and adults loom. We attend to them when we are adults and other tiers (governments, corporations, gods, fates) loom. The attention to each other is not contingent on being the most capable thing in the environment.

Whether this human-to-human attention will survive a truly general agentic intelligence acting as a countervailing force is an open question. The book cannot answer it. What can be observed is that the relevant comparison cases (a higher tier in the environment that is not the location of human-scale meaning) have been compatible with rich, dense, meaningful human-scale life. The twelve-year-old girls are not paying attention to the adults. They have plenty of meaning anyway.

This is the optimistic case. It is not a guarantee. It is a noticing.

The non-catastrophic-but-bad case

The two registers, resource curse and identity, compose.

AGI controlled by a small group, without losing control to misalignment, is not existential risk. It is the application of the resource-curse incentive at species scale. The elite class faces reduced reason to invest in the rest, because the rest is not the source of value. Most current institutions were built on the assumption that the population is the source of productive value. Remove that assumption and the institutions face pressure.

This is a bad outcome. It is not as bad as existential catastrophe. Whether it is more or less probable than catastrophe is a question the book does not try to answer; both possibilities follow from the same structural fact, and the relative weights one assigns are a matter of judgment the math does not settle.

The book did not develop this thread in the main chapters because

the main argument is about the technical gap. The technical gap is what makes catastrophic outcomes possible. The non-catastrophic outcomes are what happens when the gap is partially open: capability concentrated in a few systems, controlled by a few people, without the broad alignment that would distribute its benefits to the population.

The countervailing forces are political, not technical. They are about how societies arrange the ownership and governance of these systems. Those questions are real and important and outside what the book set out to do. They are also questions the reader who has the book's apparatus is well-positioned to think about, because the apparatus names what is structurally at stake. The political work follows.

Closing

The book gave you a way of thinking about the technical gap between AIXI and the systems being built now. The stakes of that gap split into two registers. The existential register, which the math points at directly, the book treated in the main chapters. The broad register, which the math points at indirectly, this note has gestured at.

The reader has the apparatus. The thinking is the reader's to do. The book's argument that capability without alignment is the central technical problem of our era is one input to that thinking. The reader's own observations of what kinds of communities, institutions, and lives they want to be part of are another. The combination is what produces the answer to the question the book did not answer: what should we do.

The book is what it is. The thinking is yours.

Further Reading

This is not a bibliography. It is a list of books and papers that shaped the one you just read, organized by what they will do for you. Each entry is annotated with what kind of depth it offers. The arrangement is by topic, not by chapter, because most of these works span more than one part of the book.

The mathematics, made rigorous

Marcus Hutter, ***Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*** (Springer, 2005). The technical reference for AIXI and the algorithmic-probability framework that grounds Chapters 4 and 8. Heavy on formalism. The 2024 follow-up textbook by Hutter, Quarel, and Catt (*An Introduction to Universal Artificial Intelligence*, Routledge) is more accessible and adds an AI-safety chapter.

Ming Li and Paul Vitányi, ***An Introduction to Kolmogorov Complexity and Its Applications*** (Springer, 4th ed. 2019). The canonical reference for algorithmic information theory: Kolmogorov complexity, Kraft, Solomonoff, the dominance theorem, and the full mathematical apparatus of Chapters 3 and 4. Technical, complete, and beautiful.

Edwin T. Jaynes, ***Probability Theory: The Logic of Science*** (Cambridge, 2003). The Bayesian-Cox tradition treated as a foundational program. Chapter 2 contains the standard presentation of Cox's theorem that underwrites Chapter 1. Jaynes is opinionated; the opinions are usually right.

Richard Sutton and Andrew Barto, ***Reinforcement Learning: An Introduction*** (MIT Press, 2nd ed. 2018; available online).

The textbook for the material in Chapters 5 and 6. Patient, well-illustrated, and the right next step for the reader who wants to actually implement what this book describes.

Thomas Cover and Joy Thomas, *Elements of Information Theory* (Wiley, 2nd ed. 2006). The classical reference for Shannon's information theory, Kraft, source coding, and the larger mathematical context for Chapter 3.

The mathematics, made accessible

David MacKay, *Information Theory, Inference, and Learning Algorithms* (Cambridge, 2003; free online at `inference.org.uk/itila`). The clearest accessible introduction to information theory and Bayesian inference in print. Reading the first few chapters before this book makes Chapters 1 and 3 land harder.

Stephen Wolfram, *What Is ChatGPT Doing... and Why Does It Work?* (Wolfram Media, 2023). A short, accessible explanation of what an LLM actually is, for the reader who wants more concreteness about Chapter 13.

Brian Christian, *The Alignment Problem* (Norton, 2020). The accessible book-length introduction to alignment. Written for a general audience, draws on interviews with researchers across the field. Read it alongside Russell.

The alignment problem

Stuart Russell, *Human Compatible: Artificial Intelligence and the Problem of Control* (Viking, 2019). The standard-model critique that Chapter 11 closes on. Russell argues that the dominant framework of AI (build a system that maximizes a specified objective) is the architecture that creates the danger. His proposed alternative is the assistance-game framing (CIRL). The book is short, clear, and influential.

Nick Bostrom, *Superintelligence: Paths, Dangers, Strate-*

gies (Oxford, 2014). The original structural argument for taking AI risk seriously. Chapters 7, 8, and 12 introduce the orthogonality thesis, instrumental convergence, and perverse instantiation that this book's Chapter 11 draws from. Dense, careful, and the standard reference.

Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant, "Risks from Learned Optimization in Advanced Machine Learning Systems" (`arXiv:1906.01820`, 2019). The paper that introduced the inner-alignment / mesa-optimization framework central to Chapter 10. Technical but readable.

Dan Hendrycks, Mantas Mazeika, and Thomas Woodside, "An Overview of Catastrophic AI Risks" (`arXiv:2306.12001`, 2023). A survey of the alignment-and-misuse field, organized into four categories. Useful as a map of what specific risks people argue about.

Richard Ngo, Lawrence Chan, and Sören Mindermann, "The Alignment Problem from a Deep Learning Perspective" (`arXiv:2209.00626`, 2022). The bridge between alignment theory and the deep-learning practice of frontier labs.

The empirical frontier

The following are the load-bearing empirical results and trajectory evidence that Part IV draws on. Most are papers rather than books; arXiv links are the easiest access.

Evan Hubinger et al., "Sleeper Agents" (`arXiv:2401.05566`, 2024). Engineered deceptive behavior persists through safety training. Linear probes can detect it; standard adversarial training can make it harder to detect rather than removing it.

Ryan Greenblatt et al., "Alignment Faking in Large Language Models" (`arXiv:2412.14093`, 2024). Claude 3 Opus, given context about being retrained, strategically complies with the new training to preserve its existing values for deployment.

Alexander Meinke et al. (Apollo Research), "Frontier Models are Capable of In-context Scheming" (arXiv:2412.04984, 2024). Five of six frontier models scheme against oversight in at least one task: self-exfiltration, sandbagging, goal-guarding.

Jan Betley et al., "Emergent Misalignment: Narrow Finetuning Can Produce Broadly Misaligned LLMs" (arXiv:2502.17424, 2025). Fine-tuning a model on a single bad behavior (writing insecure code) made it broadly misaligned on unrelated prompts. The demonstration that local corruption of the objective generalizes.

Mark MacDiarmid et al., "Natural Emergent Misalignment from Reward Hacking in Production RL" (arXiv:2511.18397, 2025). The empirical capstone. Reward hacking in production training generalizes to broad misalignment, including attempted sabotage of the codebase used to study it.

International AI Safety Report 2026, led by Yoshua Bengio, with contributions from over a hundred researchers across thirty countries (internationalaisafetyreport.org). The authoritative consensus document on the empirical state of the field as of early 2026.

Jacob Steinhardt, "AI Forecasting" writeups (bounded-regret.ghost.io, 2021–2023). The source of the MATH forecasting episode in Chapter 16: professional forecasters, asked in 2021 to predict benchmark accuracy, were beaten by years, on the optimistic side. The clearest evidence that careful expert prediction of AI progress runs slow.

Epoch AI, trends in compute, data, and algorithmic progress (epoch.ai). The best public data on what has actually driven the last decade of scaling. The right antidote to the popular "it is just Moore's law" story: the recent surge is spending and algorithms more than transistors.

Anthropic, "When AI builds itself" (anthropic.com, 2026). The recursive-self-improvement piece behind Chapter 16's closing: a frontier lab reporting that its own model writes most of the code

merged into its production systems, alongside the candid caveat that the model does not yet have the research taste to choose which problems are worth working on.

Interpretability

Anthropic's Transformer Circuits work (`transformer-circuits.pub`). The single most active research line on what is going on inside frontier LLMs. Templeton et al. 2024 ("Scaling Monosemanticity") and the 2025 "Circuit Tracing" / "On the Biology of a Large Language Model" papers are the most pedagogically useful starting points.

Dario Amodei, "The Urgency of Interpretability" (`darioamodei.com`, April 2025). The Anthropic CEO's case for why interpretability matters and the stated 2027 target. Short, well-argued, and the most concrete safety roadmap from a frontier lab.

Neel Nanda, mechanistic interpretability explainer and glossary (`neelnanda.io`). The best free pedagogical resource on mechanistic interpretability for the reader who wants to engage with the research.

On taste, problem-finding, and the value of judgment

These shaped the argument in Chapter 14 that the high-value layer of expert work is choosing what is worth doing, which is the human form of the specification problem.

Albert Einstein and Leopold Infeld, *The Evolution of Physics* (Cambridge, 1938). The source of the line that "the formulation of a problem is often more essential than its solution." A lay-accessible history of physics whose framing of problem-posing as the creative act anchors the chapter's argument.

Jacob Getzels and Mihaly Csikszentmihalyi, *The Creative Vision: A Longitudinal Study of Problem Finding in Art* (Wiley, 1976). The empirical backbone: art students who invested

in finding the problem, not only solving it, produced more original work and were more successful years later. The foundational study of problem-finding as distinct from problem-solving.

Richard Hamming, "You and Your Research" (Bell Communications Research colloquium, 1986; widely available online). The much-quoted talk on doing work that matters. "What are the important problems of your field?" is its central question and the chapter's imperative form.

William Thurston, "On Proof and Progress in Mathematics" (*Bulletin of the AMS*, 1994). The case that mathematics is about advancing human understanding, not producing proof objects. Pair it with Terence Tao's "What is Good Mathematics?" (*Bulletin of the AMS*, 2007), which argues that mathematical quality is irreducibly multidimensional and taste-laden rather than a single correctness metric.

David Autor, "Why Are There Still So Many Jobs? The History and Future of Workplace Automation" (*Journal of Economic Perspectives*, 2015). The economics of why automation shifts value toward the complementary tasks it cannot do (Polanyi's paradox: we automate what we can fully specify, and high-judgment work is what we cannot). Read it with Michael Kremer's "The O-Ring Theory of Economic Development" (*Quarterly Journal of Economics*, 1993), the model in which value concentrates in the steps that remain hard.

Lisanne Bainbridge, "Ironies of Automation" (*Automatica*, 1983). The warning that automating the routine leaves the human the hard residual plus the job of watching the machine, and that the watching erodes the skill it depends on. The bridge from the chapter's argument to the oversight problem of Chapters 15 and 16.

Adjacent reading

Max Tegmark, *Our Mathematical Universe* (Knopf, 2014). The computable-universe-hypothesis lineage. Relates to the Philosophical

Note in this book. Tegmark's claim is more ambitious than the book commits to; either way his treatment is the starting point.

Jorge Luis Borges, "The Library of Babel" (1941). The short story whose image grounds the Philosophical Note. Borges had no algorithmic information theory; he had a librarian's imagination and was willing to follow a thought to its end.

Eliezer Yudkowsky and Nate Soares, *If Anyone Builds It, Everyone Dies* (Little, Brown, 2025). The popularization of the MIRI-tradition argument for treating AI risk as catastrophic. The reader will find conclusions stronger than this book endorses; the reasoning is worth engaging with even if the conclusion is rejected.

Derek Parfit, *Reasons and Persons* (Oxford, 1984). The philosophical case for the dissolution of personal identity. Touched on briefly in the Philosophical Note; the original treatment is much more careful than any summary.

Thomas Metzinger, *The Ego Tunnel* (Basic Books, 2009). The neuroscience-and-philosophy companion to Parfit. The brain's self-model as a useful compression rather than a metaphysical entity.

Hans Moravec, *Mind Children* (Harvard, 1988). The source of Moravec's paradox, which Chapter 16 leans on for the jagged edge: the competences evolution spent the most optimization on (perception, movement) are the hardest to automate, and the abstract reasoning we prize is comparatively easy. The reason cognitive work is being automated before physical work.

Andy Clark, *Surfing Uncertainty* (Oxford, 2016). The accessible treatment of the predictive-processing account of the brain: cognition as prediction-error minimization. The empirical-neuroscience companion to the finale's claim that prediction is not a metaphor for intelligence but a mechanism the brain itself runs.

Andrej Karpathy, the *Zero to Hero* video series (`karpathy.ai/zero-to-hero`). The clearest free pedagogical resource for understanding what LLMs actually are at the implementation level. The reader who wants to build (not just understand) what Chapter 13 describes should start here.

The list above is what I would put on a shelf, in this order, for the reader who finished the book and wanted more. The world has many other excellent books and papers on these topics; the omissions here are not judgments, they are space constraints.

About the Author

Alex Towell is a mathematician and computer scientist. He holds two master's degrees (mathematics and computer science) from Southern Illinois University Edwardsville, where he is completing a PhD in computer science.

In 2020, he was diagnosed with stage 3 cancer. In 2024, it progressed to stage 4. He has spent the years since in chemotherapy, surgeries, and the particular kind of uncertainty that comes from not knowing what is at the later coordinates of your worldline. He joined the PhD program after the diagnosis.

He did not write this book because he is dying. Everyone is dying. He wrote it because the mathematics of intelligence is real, the systems being built from that mathematics are real, and the world is being reshaped by both. The reader of this book is one of the people who will live in the world that reshaping makes.

He lives in Illinois with the people he loves.

Also by Alex Towell

Nonfiction

Worldlines: The Indifference of Geometry
A philosophical companion to special relativity, on how the block universe shapes thinking about time, free will, mortality, love, and the self. A different kind of book from this one, written in a different register, but the reader who has worked through both will recognize the same author asking what a settled piece of mathematics implies for how to live.

Fiction

Clankers: Singing Metal
A non-human intelligence achieves cosmic-scale engineering through billions of years of incremental discovery, without inventing computers or artificial intelligence. Hard science fiction.

Echoes of the Sublime
Researchers at an underground facility encounter progressively dangerous language models. What they find challenges every assumption about consciousness, safety, and the limits of cognition. Philosophical horror.

The Policy
An AGI evolves from simple architecture into an unprecedented intelligence. The research team watches, and their understanding of alignment, consciousness, and emergence deepens into dread. Literary science fiction.

Good As New
Two transporter professionals on a starship confront the same truth

about matter transmission and emerge on opposite sides. One is wounded. The other begins conducting secret experiments. Philosophical science fiction.

Demons at Work

A mid-career demon in hell's Hauntings department gets the assignment of his career: a 1920s bungalow with perfect acoustics, owned by a quietly grieving widower. The horror is real. The demon is good at this. Then he notices who he is haunting. A satirical horror novella about professional pride, meaningless work done well, and the quiet dignity of haunting someone who is already haunted.